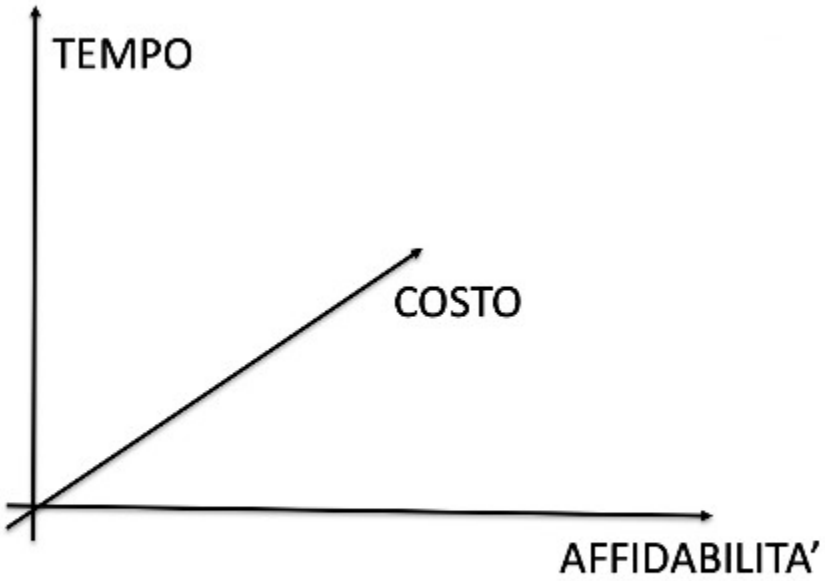
Introduzione

L’ingegneria del software è la disciplina che si occupa dello **sviluppo** del software, e comprende **principi**, **metodi**, **strumenti** e **processi**.
Il software è uno strumento che sta diventando man mano sempre più indispensabile per molti campi (servizi, commercio, industria, intrattenimento etc).
Al giorno d’oggi si può “fare” tecnologia in tre modi:

- Nuove tecnologie
- Sviluppo/aggiornamento di tecnologie esistenti
- Sostituzione di vecchie tecnologie

La facilità di distribuzione del software ha vantaggi e svantaggi, ad esempio, parlando di vantaggi, anni fa per poter usare un software bisognava andare in un negozio per comprare un disco fisico con all’interno il software, invece oggi si può ottenere facilmente con un click online. Ci sono anche svantaggi come il fatto che ora ci sono le recensioni che, se negative, possono influire molto in maniera drastica.
Esistono delle procedure ben definite per verificare la **correttezza** e la **stabilità** del software.
Ci sono tantissimi fattori che vanno ad influire sulla “**dimensione**” di un software, ovvero la quantità di lavoro che serve per portarlo a termine



Mantenibilità, estensibilità, integrabilità, portabilità, usabilità, documentazione, …

Ci sono poi vari fattori per misurare la **qualità** di un software:

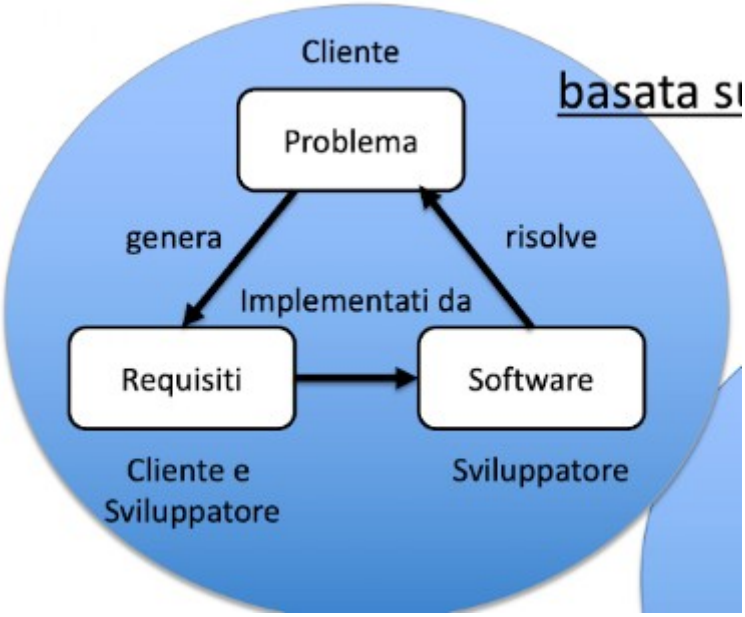
- Interni
    - Verificabilità
    - Manutenibilità
    - Riusabilità

    - Portabilità
- Esterni
    - Correttezza
    - Affidabilità
    - Robustezza
    - Efficienza
    - Usabilità
    - Scalabilità

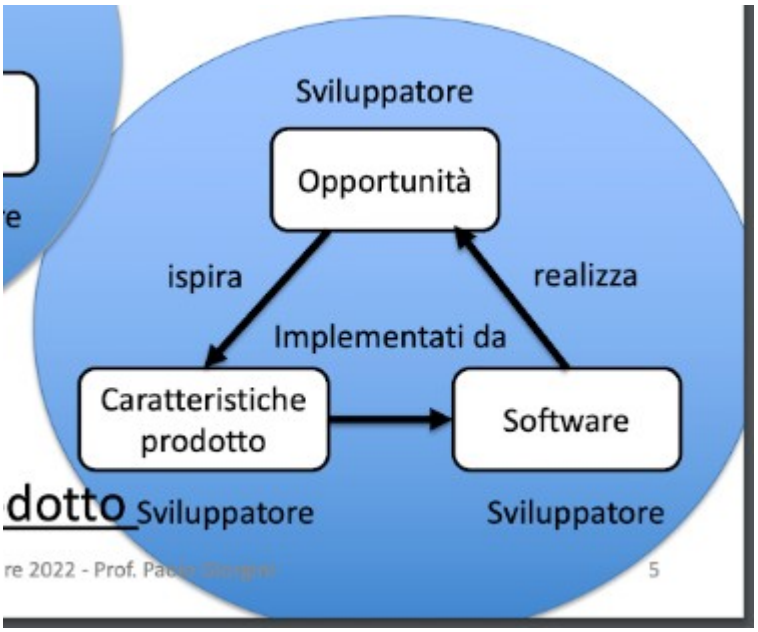
Ovviamente bisogna cercare di ottenere la massima qualità possibile con meno lavoro possibile, ed infatti esistono vari strumenti per poter raggiungere questi obiettivi.
I problemi legati alla qualità del software sono difficili da rilevare e in molti casi vengono rilevati in fasi avanzate, e si rischia di dover buttare la maggior parte del lavoro.

Ci sono due modalità per ideare un software:

- Problema dell'utente, basata sul **progetto**



- Idea dello sviluppatore, basata sul **prodotto**



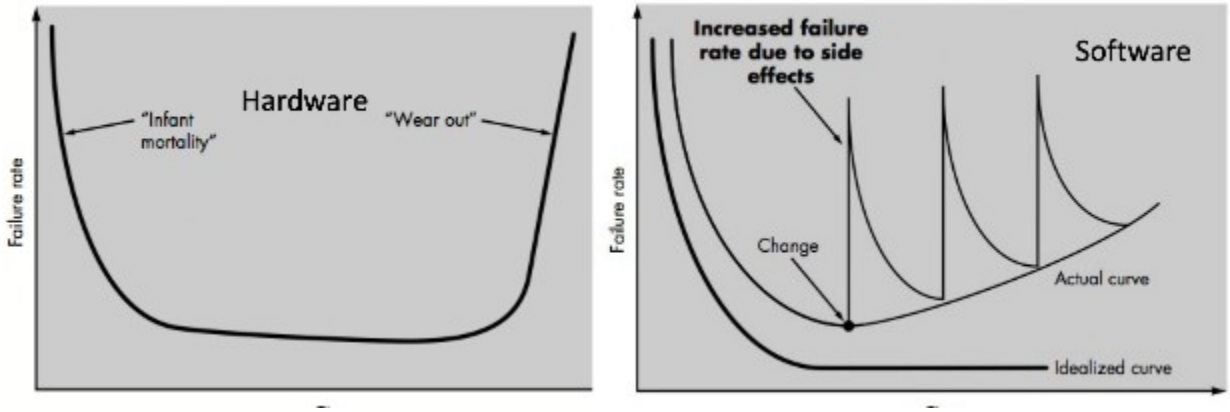
Un fattore importante è notare che il software è “sminuito” dalle persone.
Ad esempio quando un cliente vuole costruire una casa e si rivolge ad un architetto, ascolterà l’architetto e difficilmente si lamenterà dei progetti e dei costi; invece quando si parla di software è completamente l’opposto (“come fa a costare così tanto una cosa immateriale?”).

Domande a cui dover rispondere

- Perché tempi così lunghi?
- Perché costi così elevati?
- Perché non sono stati individuati tutti gli errori prima della consegna?
- Perché così tanto tempo e impegno nel mantenimento di un software esistente?
- Perché è difficile misurare i progressi dello sviluppo del software?

Ingegneria del software - risposte a queste domande attraverso metodi sistematici e tecniche consolidate nel tempo

Nella fabbricazione hardware abbiamo una curva di “fallimento” abbastanza standard, ovvero che all’inizio può dare problemi durante la costruzione, poi funziona correttamente e normalmente, e dopo un po’ di anni smette di funzionare, soprattutto perchè c’è il deterioramento dei componenti.
Si penserebbe che per il software questa cosa non valga, perché è un codice che non si può “rovinare”, invece succede, e succede anche più frequentemente rispetto all’hardware (ad esempio nuovo sistema operativo, nuova release di API).



Solitamente le persone pensano delle cose che **non sono mai vere** nella pratica di un progetto software, ad esempio che per completare più velocemente un progetto basti aumentare il numero di programmatori, oppure che una volta che il programma è finito il programmatore non deve più fare niente.

Solitamente il software costa più dell’hardware, così come la manutenzione/evoluzione costa più dello sviluppo (da notare che la distribuzione dei costi dipende molto dal **modello di sviluppo** che viene adottato).

Gli ingegneri del software adottano un **approccio sistematico** per compiere il loro lavoro e fanno uso di **tecniche e strumenti** appropriate.
Lo scopo dell’ingegneria del software è risolvere un problema complicato non ben definito e capito, infatti spesso il cliente non sa cosa vuole ed è difficile capirlo.
Un problema non può essere risolto senza prima averlo capito.

I ruoli nell’ingegneria del software:

- Cliente (o stakeholder)
  Richiede un sistema informatico per risolvere un problema/obiettivo tramite interazione dell’utente oppure interazione dell’ambiente.
- Utente
  Colui che farà uso del sistema informatico
- Ingegnere del software
  Deve capire cosa vuole il cliente e deve capire come fare per ottenerlo, definendo le specifiche e il design
- Programmatore
  Implementare, scrivere il software che ha definito/specificato l’ingegnere del software

Notiamo che l’ingegnere e il programmatore spesso sono la stessa persona, ma non sempre.

Le specifiche dicono “che cosa” fare e non “come” fare per risolvere un problema
verifica -> vedere se è stata implementata bene la specifica (in caso è colpa del programmatore)
validazione -> vedere se risolve veramente il problema posto dal cliente (in caso è colpa dell’ingegnere del software)

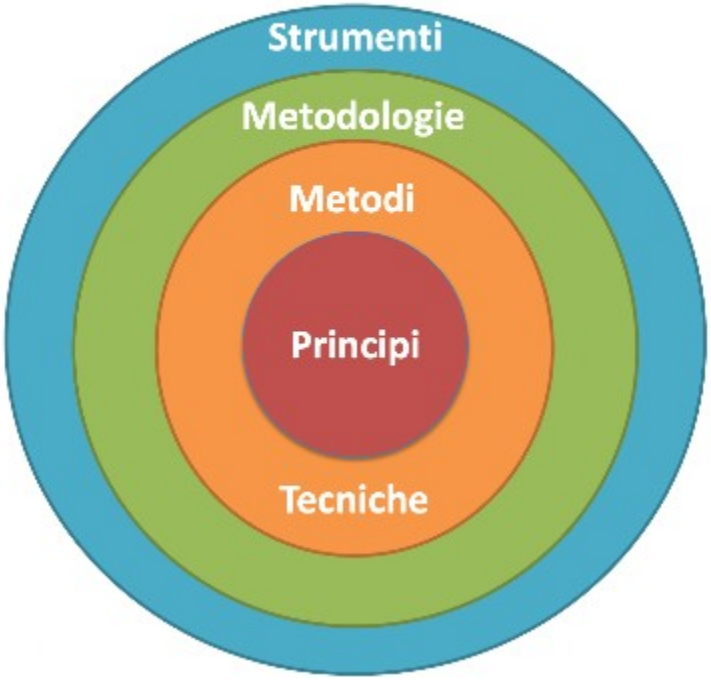
I **processi software** sono un’insieme di attività aventi per obiettivo lo sviluppo/evoluzione di un sistema software, ogni processo tipicamente include:

- Specifica
  Ciò che il sistema deve fare e i vincoli di progettazione
- Sviluppo
  Progettazione e programmazione
- Convalida
  Si verifica che il software sia ciò che il cliente richiede
- Evoluzione
  Si modifica il software per adeguarlo ai requisiti dell’utente e del mercato

Principi, processi e metodologie

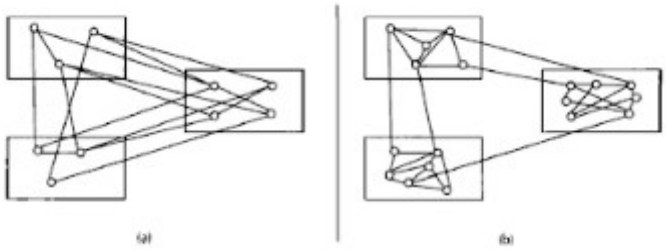
La prima definizione (1969) di ingegneria del software fu:
L’ingegneria del software è l’istituzione e l’impiego di principi ingegneristici ben fondati, allo scopo di ottenere in modo economico software affidabile ed efficiente
Questa definizione a grandi linee vale tuttora, l’ingegneria del software è quindi l’applicazione di una **strategia sistematica**.
È molto importante nell’ingegneria del software riuscire a dare ai **requisiti** delle **misure** (bisogna misurare l’usabilità, la velocità, l’affidabilità etc.)
Quindi più che una disciplina o un insieme di conoscenze è **un modo per affrontare un problema**.
L’ingegneria del software si può vedere come una cipolla (a strati), si parte da una base che sono i **principi**, per poter implementare questi principi dobbiamo utilizzare dei **metodi**/delle **tecniche**.
Poi metto insieme questi metodi andando a formare la **metodologia** e queste metodologie vengono supportate dagli **strumenti** (che ci aiutano a programmare ad esempio)

IS come tecnologia stratificata



Ecco i **principi** più importanti:

- **Rigorosità e formalità**
  Dare vita ad un software è una pratica creativa, però richiede una sistematicità per poter guadagnare in tempo e in ottimizzazione
  Degli esempi sono una prova formale di correttezza e la stesura di una documentazione precisa
- **Separazione dei problemi**
  Separare i problemi aiuta a gestire la complessità, magari dividendo la risoluzione tra più persone (dividi et impera)
  Degli esempi sono usare diverse tecniche di test per diversi problemi
- **Modularità**
  Dividere lo sviluppo in moduli che hanno un ruolo ben preciso, in modo che poi comunichino tra di loro



  L'idea è di preferire le interazioni all'interno del modulo limitando le interazioni esterne al minimo indispensabile così da formare **alta coesione** e **basso accoppiamento**
  Così riduco la dipendenza di un modulo con gli altri, e in caso di guasto l'avere moduli

indipendenti è vantaggioso perché il sistema generale ha più possibilità di continuare a funzionare.
La differenza tra modularità e separazione dei problemi è che la separazione dei problemi è a monte, la modularità viene poi durante lo sviluppo.

- **Astrazione**
  L'astrazione consiste nell'ignorare i dettagli, così da avere meno "problemi" a cui pensare e poter pensare al progetto in maniera più generale, ad esempio concentrandosi solo sulle caratteristiche di alto livello
  Un esempio di astrazione è dividere il progetto in frontend, backend eccetera, concentrandosi su questo livello tralasciando i mille dettagli che ci sono sotto.
  L'astrazione viene fatta per una questione di semplicità di ragionamento
- **Previsione dei cambiamenti**
  La capacità nel riuscire ad affrontare l'evoluzione, sia durante che dopo la realizzazione.
  Questo comprende sia richieste del cliente che l'evoluzione esterna delle tecnologie
  Un esempio è che una versione 3.0 di un software è derivata dalla 2.95, la quale è compatibile con la 2.7 in un altro modulo
- **Generalità**
  Solitamente noi dobbiamo risolvere il problema generale, e non l'istanza. Quindi quando è possibile dobbiamo generalizzare un problema così da poter riutilizzare (**riusabilità**) quella soluzione per altre istanze
  Dobbiamo fare però attenzione ai costi, in quanto generalizzare ha un costo
- **Incrementalità**
  Procediamo a passi successivi, ad esempio tramite prototipi incrementali, con interazioni continue con l'utente che vede crescere il sistema.
  Ha un costo più elevato e ci vuole più tempo ma siamo sicuri che il prodotto finale andrà bene

Un **processo di sviluppo** stabilisce **come** e **quando** qualcuno fa una **cosa** per raggiungere un determinato obiettivo
La scelta dell'approccio, su che modello usare, dipende da moltissimi fattori (in base al problema, al team, alla cultura aziendale etc)

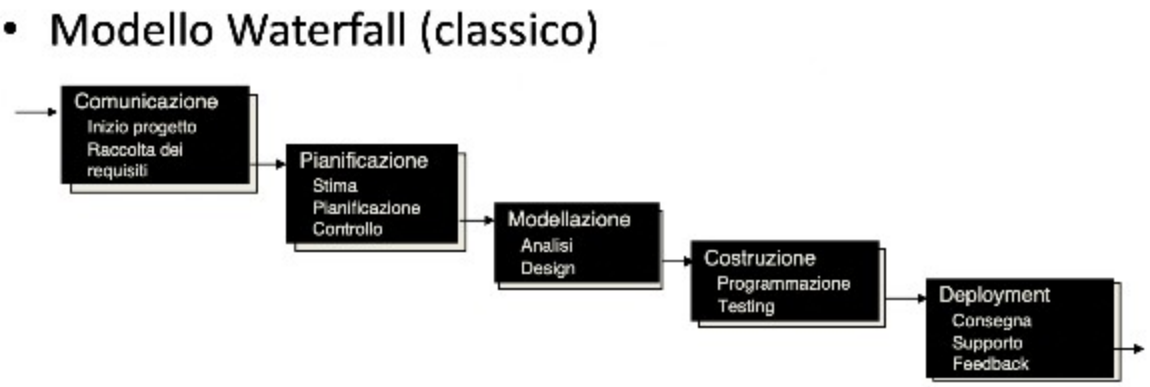
Modelli di processi

Le differenze tra i vari modelli sono:

- Flusso delle attività, dei task e loro interdipendenza
- Attività di controllo del progetto
- Dettaglio e rigore del processo
- Coinvolgimento degli stakeholders
- Autonomia del team
- Prescrizione dei ruoli e loro organizzazione

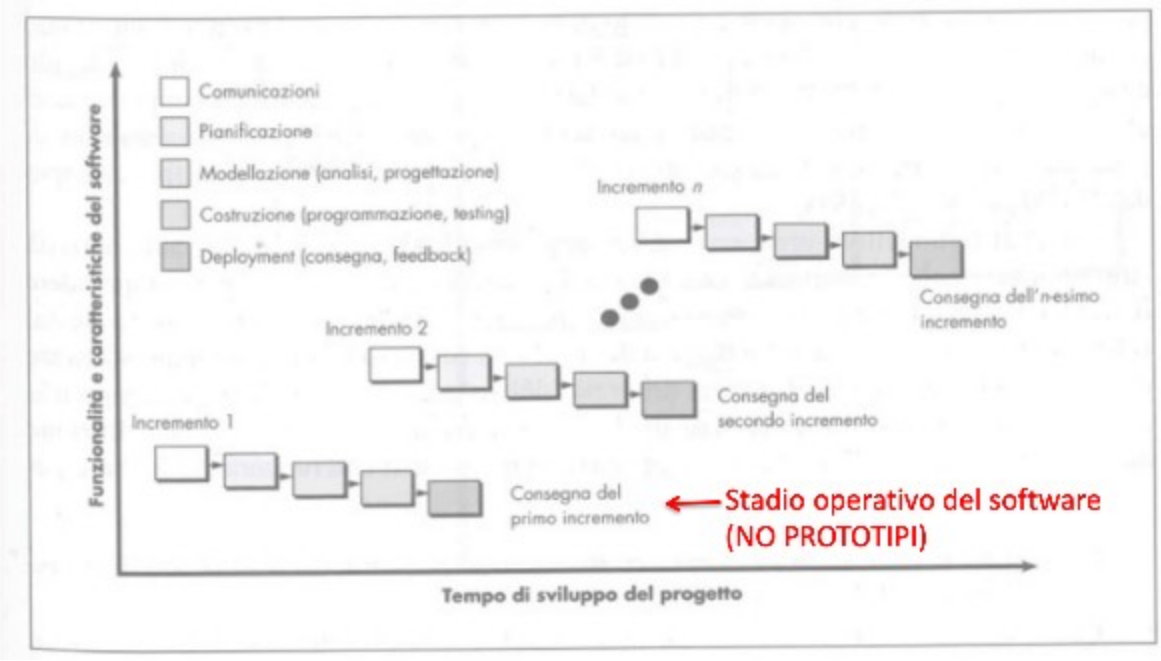
Una caratteristica da far notare è quella dei modelli **agili** ed è che giovano di **alta adattabilità**
Ci sono poi vari enti e modi per valutare un processo per garantire determinati livelli di qualità del software (ISO)

Modello di processo a cascata



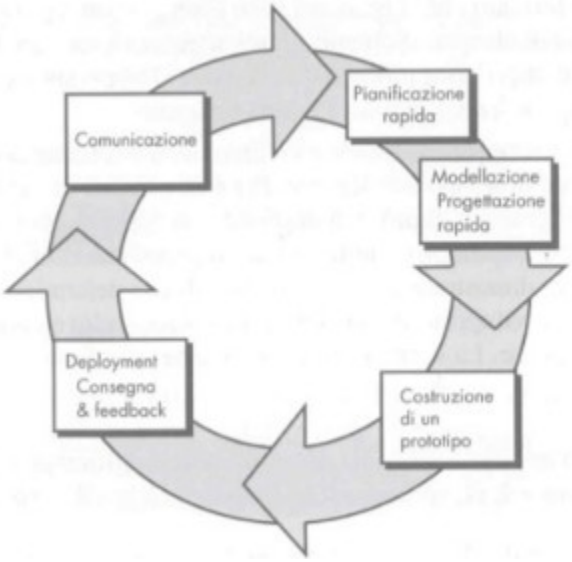
I requisiti sono noti e si può procedere in maniera lineare
Non si torna mai indietro, se ci si accorge di un errore bisogna ricominciare da capo.
Come vantaggio abbiamo i costi più bassi e di poter strutturare/organizzare l'azienda in maniera ben definita (avendo tutte le fasi separate soprattutto temporalmente)

Modello di processo incrementale



Durante uno sviluppo, parto già con lo sviluppo successivo aggiungendo altre funzionalità, fino ad arrivare a tutte le funzionalità del software con l'ultimo incremento.
Si possono usare processi a cascata in maniera incrementale.
Attenzione che i vari incrementi **non** sono prototipi, infatti sono completamente operativi per le funzionalità richieste

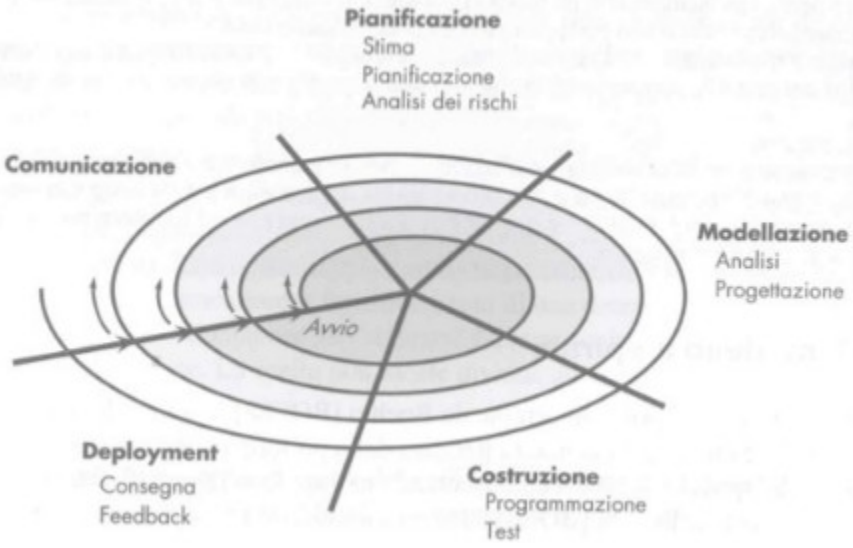
Modello a prototipi



19

Quando il cliente ha un bisogno legittimo ma non riesce a fornire indicazioni precise sui dettagli, il primo passo consiste nello sviluppo di un prototipo.
Questo prototipo è utile per definire i requisiti, avendo una rappresentazione materiale che il cliente può vedere e valutare.
Un prototipo non deve necessariamente essere un sistema funzionante (ad esempio possono essere usati solo per vedere la forma o il layout di qualcosa)
Di prototipi non funzionanti ne esistono principalmente due, **wireframe** (per trovare la migliore user-experience) oppure i **mock-up** (gli elementi del design, come colori, caratteri, immagini etc, entrano in gioco)
Solitamente i prototipi si buttano una volta mostrati al cliente, perchè viene fatto di fretta e con una qualità scadente
Ad esempio con una incrementalità del prototipo posso arrivare al prodotto finale

Modello a spirale



23

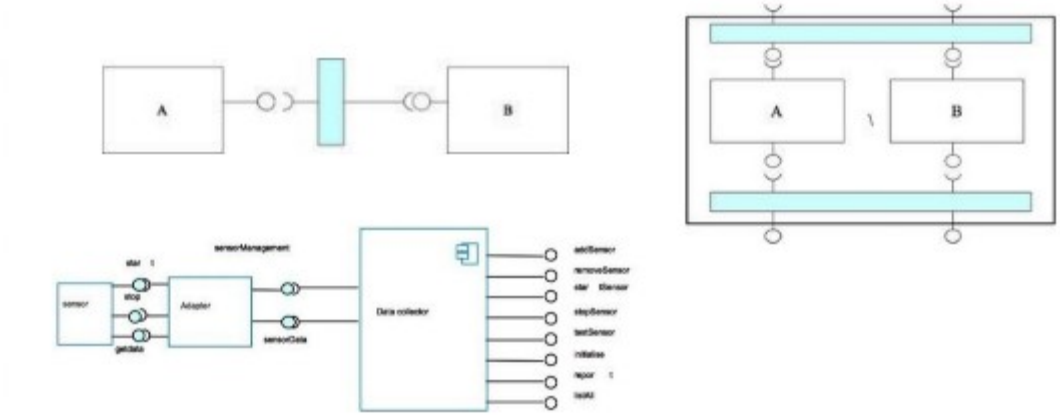
È sostanzialmente un modello incrementale iterativo (è un misto tra l'iterazione della prototipizzazione e modello a cascata)
A differenza del modello incrementale, qui si possono modificare i requisiti e le funzionalità aggiunte nelle iterazioni precedenti

Model-Driven development

Una metodologia guidata dai modelli, infatti viene creato un modello ad alto livello (una specie di prototipo) che funge da guida per lo sviluppo.
Quindi si passa attraverso vari livelli di astrazione fino ad arrivare ad un livello di grande dettaglio (c'è una vera e propria trasformazione), il quale sarà l'implementazione.
I **OMG** (Model-Driven Architecture) fanno spesso uso della notazione UML.

Sviluppo a componenti

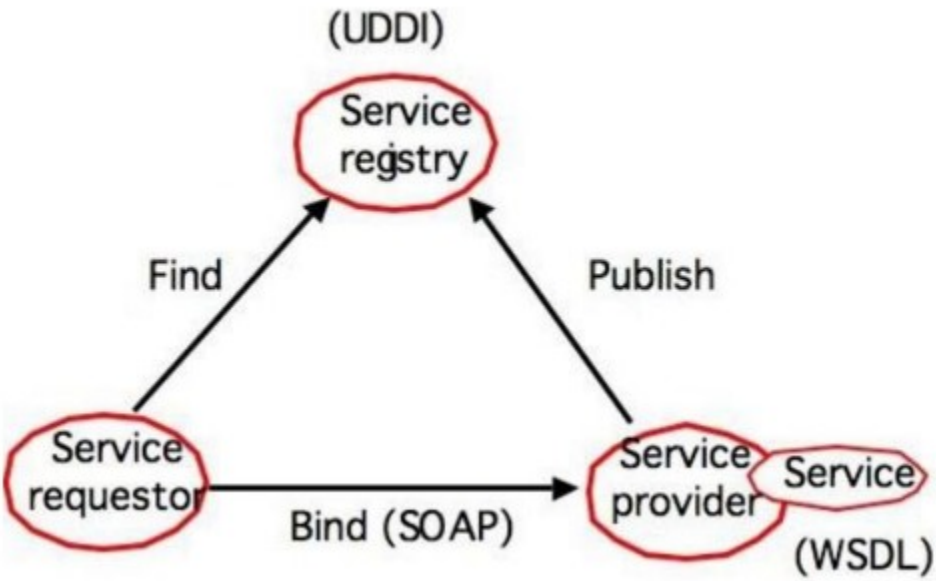
Una metodologia che sfrutta delle parti di codice che possono venire riutilizzate, le quali vengono adattate al contesto e combinate con altre componenti



Ovviamente i componenti più semplici risultano essere i più riusabili ma difficilmente riescono a risolvere problemi complessi, però con la **composizione** con altri componenti si possono raggiungere requisiti più specifici e complessi.
Spesso per la composizione risulta necessario costruire dei pezzi di codice "integratore" (**glue code**), ovvero che faccia da collante tra questi componenti (vengono soddisfatte riusabilità e generalità)
Molto vantaggioso per quanto riguarda i costi, potendo riutilizzare componenti vecchie.
Bisogna riutilizzare bene i componenti perchè altrimenti si rischia di fare danni (ad esempio nel 1996 per il lancio di un razzo venne riusato un componente che era stato scritto per un altro razzo, il quale causò un overflow che per il vecchio razzo era irraggiungibile visto che era troppo poco potente, e questo fece perdere il controllo del razzo che si distrusse)

Architetture orientate ai servizi

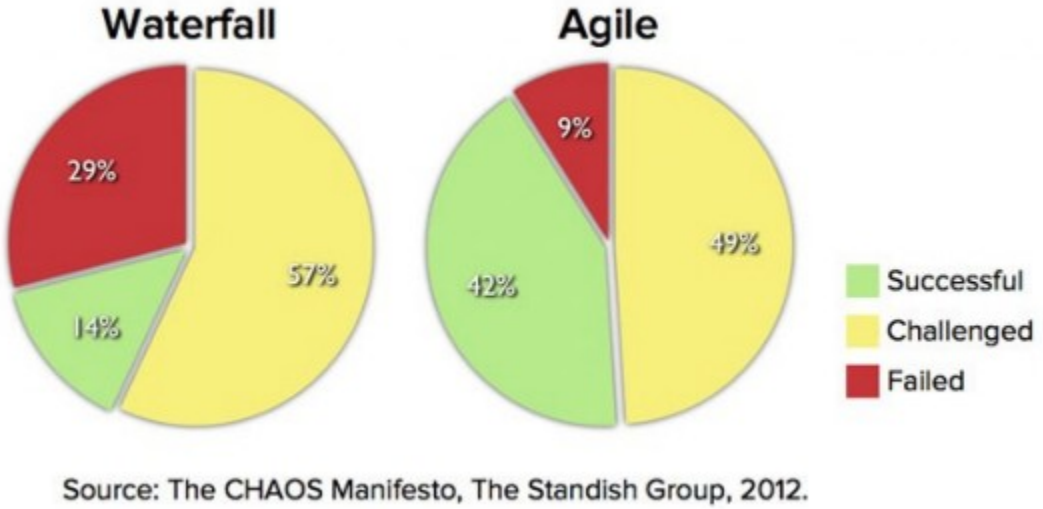
Costruisco il software non sviluppando codice ma utilizzando un servizio già funzionante (ad esempio delle API per lo scambio di mail)



Un rischio/problema è che i servizi sono dinamici, nel senso che non sono sempre a disposizione oppure non sono sempre gli stessi (vengono aggiornati), e questo richiede una costante attenzione per evitare di perdere funzionalità

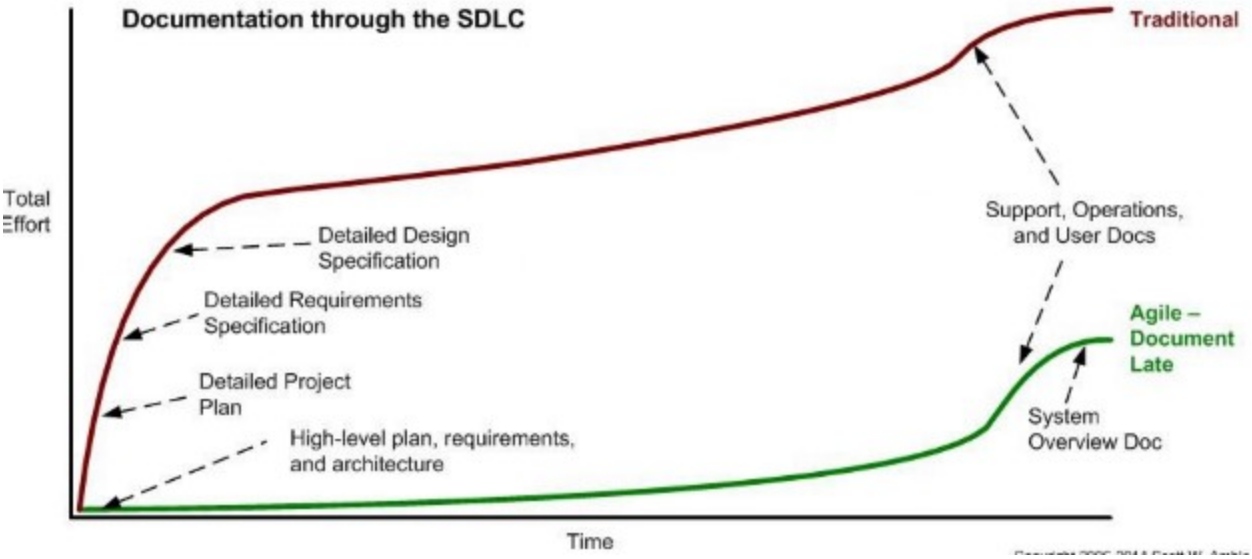
Metodologie Agile

Con metodologia agile si intende un particolare metodo per lo sviluppo di software in cui si hanno continui confronti e colloqui con il committente, ottenendo in tal modo una elevata reattività alle sue richieste, esistono varie versioni di metodologie agile
Questo tipo di affronto dello sviluppo è nato quando non se ne poteva più di processi/modelli eccetera, principalmente dal punto di vista degli studenti
Simile all'approccio a spirale, attraverso la forte partecipazione/discussione a tutti i livelli (ad esempio facendo intervenire i programmatori in fase di progettazione)
Nei modelli AGILE la percentuale di successo aumenta notevolmente rispetto ad un modello a cascata



Source: The CHAOS Manifesto, The Standish Group, 2012.

Un altro fattore di diversità è la documentazione, che nelle metodologie Agile viene concentrata verso la fine del progetto. La documentazione finale sarà meno precisa/fatta bene rispetto a una metodologia tradizionale



L'approccio agile non forza il cliente a fare una lista di requisiti ma favorisce una descrizione di come dovrà essere utilizzato il software finale, attraverso degli scenari di utilizzo, con magari una descrizione delle possibili interazioni tra utenti e sistema.
Viene quindi sviluppato software in maniera iterativa, con forte enfasi sulle attività di costruzione con molteplici **incrementi software** e privilegiando **l'adattamento ai cambiamenti**

Le metodologie Agile non vanno a sostituire l'ingegneria del software (non bisogna pensare che agile = improvvisazione), visto che comunque i principi e le fasi tipiche di un progetto vengono mantenute. Molti dei concetti della metodologia agile non sono altro che **adattamenti** dei migliori concetti dell'ingegneria del software

Il modello SCRUM

(scrum = mischia)
Sono delle insieme di pratiche e regole che permettono di ridurre l'overhead (i costi) organizzativi
Il progetto viene portato avanti da piccoli team (2-3-4 persone), i quali producono frequenti incrementi software con una continua attività di testing e documentazione.
Il progetto viene diviso in blocchi rapidi di lavoro (chiamati **sprint**), questi sprint hanno una loro organizzazione ben precisa (cosa deve fare), alla fine degli sprint viene consegnata una versione del sistema al cliente.
Sono sempre indicati i dettagli del lavoro da fare nell'immediato futuro (**backlog**, ovvero operazioni in attesa di essere eseguite in un computer) e vengono organizzate riunioni giornaliere del team di sviluppo (**daily scrum**) per verificare cosa si è fatto e cosa si farà

- Attività strutturali
  - Requisiti, analisi, design, evoluzione e consegna



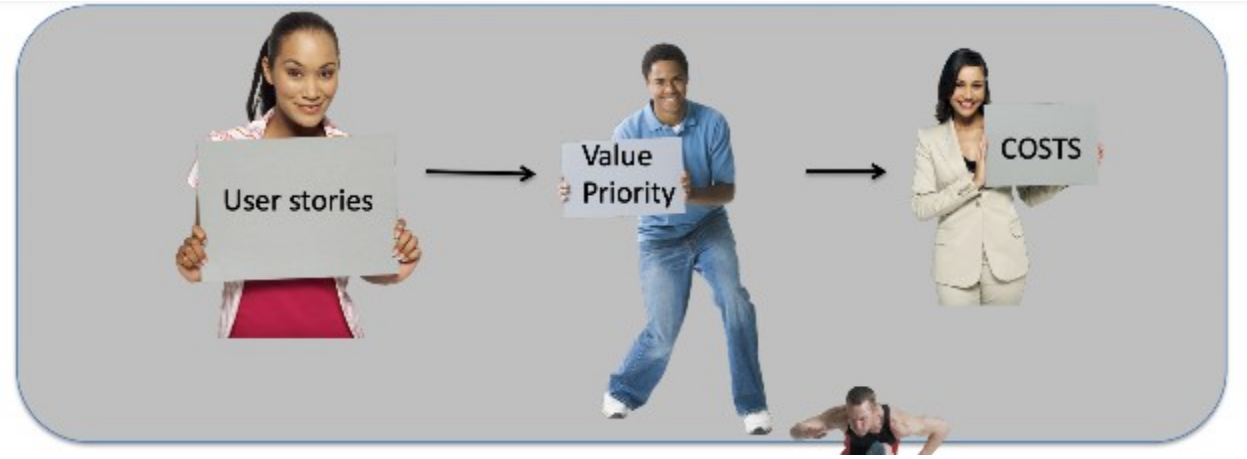
Extreme programming

Il problema dello sviluppo a componenti è l'integrazione finale, che magari non va come dovrebbe, nell'extreme programming c'è infatti una continua integrazione per limitare i problemi e i conflitti che il nuovo codice e il codice di test potrebbero dare.
È un approccio **object-oriented** per lo sviluppo
Possiamo fare un software perfetto, bello esteticamente e senza bug, ma se il cliente non è soddisfatto è tutto inutile
Questa metodologia è fondata su 4 valori e 12 pratiche:

- Semplicità, comunicazione, testing e coraggio
- Le 12 "prassi consolidate" possono essere raggruppate in 4 aree fondamentali
  - feedback a scala fine, processo continuo, comprensione condivisa, benessere dei programmatori

È strettamente sequenziale, infatti un solo programmatore alla volta integra le sue modifiche con il sistema, sempre per limitare gli errori e aumentare la facilità di trovarli
È basato su **refactoring** e **design improvement,** un esempio di refactoring è migliorare la struttura del codice in modo da renderlo più semplice e generico, ed è basato anche su **small releases**

Vengono usate un insieme di **user story**, ovvero le caratteristiche e funzionalità descritte dal cliente, e il cliente assegna una **priorità** a ciascuna user story.
I membri del team valutano le user story e assegnano ad ognuna un costo (settimane di sviluppo), se il costo è troppo alto spesso viene chiesto al cliente di dividerla in più user story



Vengono anche sviluppate delle **unit test**, che mettono alla prova ogni **user story**
È comune la pratica del **pair programming** che garantisce qualità e soluzione rapida ai problemi

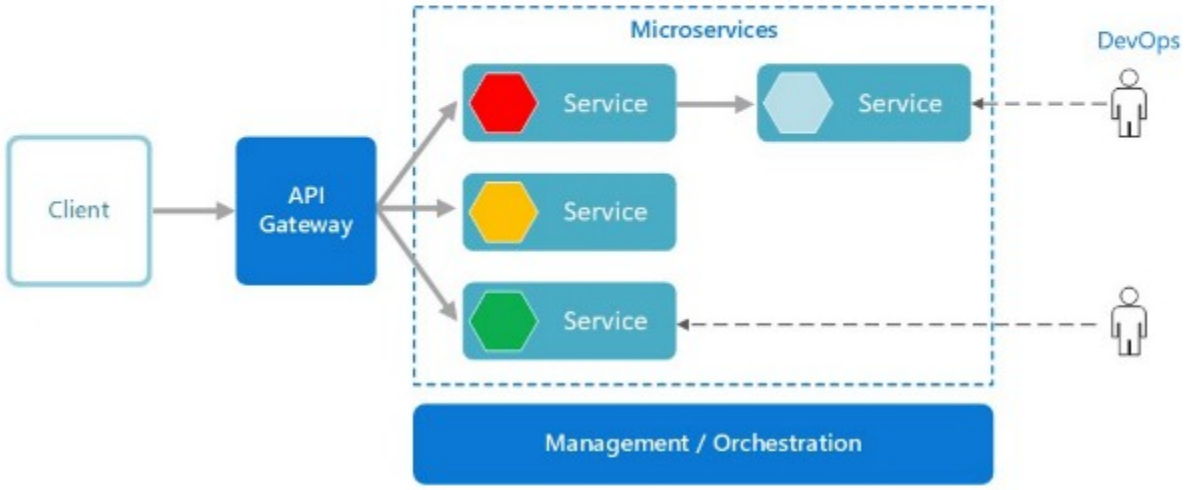
DevOps

Nella realtà i componenti software vengono installati in infrastrutture grandi e complesse e quindi bisogna coordinare il lavoro di chi scrive il software (**Dev**elopment) con quello di chi gestisce l'infrastruttura (**Op**eration**s**), è quindi una metodologia che consente di gestire in modo rapido e efficiente questa interazione
Si porta avanti sviluppo e deployment contemporaneamente

Modularizzazione e microservizi

Una delle architetture che si sta diffondendo di più è quella dei microservizi, **MA** (Microservice Architecture), in cui creo il codice semplicemente mettendo insieme i microservizi adeguati (scelti da un catalogo praticamente)
È una metodologia adatta alle **grandi applicazioni** che hanno necessità di evolversi rapidamente, sfruttando soprattutto il **cloud**
È una specie di architettura SOA (Service Oriented Application)

Esempio MA



Modellazione

Un modello è un' astrazione (quindi ci fa perdere dettagli), che rappresenta un'entità in modo semplificato, con un **focus** sulle informazioni importanti
Ad esempio in una mappa non vengono segnati i veicoli, le linee stradali o cose non importanti per il contesto. La mappa però è un "linguaggio" adatto a un turista, che conosce bene i simboli delle mappe, per un ragazzino invece quella mappa non significa niente, è incomprensibile







In ingegneria del software ci sono moltissimi linguaggi per i vari campi, fasi e contesti (UML ad esempio, ma dipende molto da tanti altri fattori)
I vari linguaggi si possono concentrare su aspetti diversi, è importante scegliere il linguaggio giusto, i quali differiscono per:

- Facilità d'uso (sia espressione che comprensione)
- Comprensione da parte di non esperti tecnici (per poter comunicare meglio col cliente)
- Formalità e precisione (per favorire analisi ed elaborazioni, e anche per il testing)

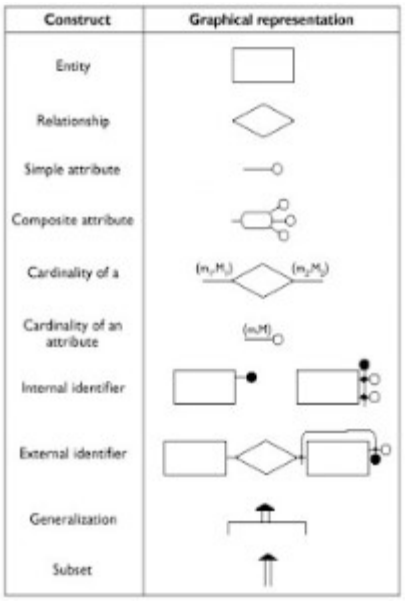
Notiamo che all'interno delle varie fasi di sviluppo di un software vengono usati svariati linguaggi, soprattutto per i diversi livelli di astrazione concettuale

Vediamo ora alcuni esempi di linguaggi

Entity-Relationship

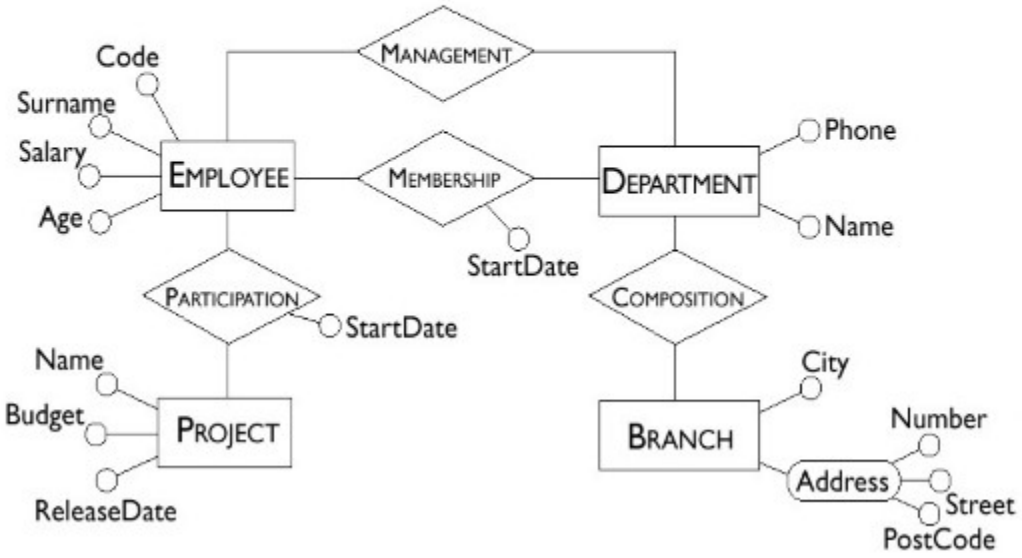
Questo linguaggio è utilizzato prevalentemente nell'analisi e nella progettazione di **database**. Si tratta di una modellizzazione concettuale (prima di quella logica)
La cosa interessante introdotta col modello ER è stata la rappresentazione attraverso degli elementi diagrammatici, formando una rappresentazione concettuale di quello che sarà un database (concentrandosi su due concetti chiave ovvero Entità e Relazioni)

Costrutti base



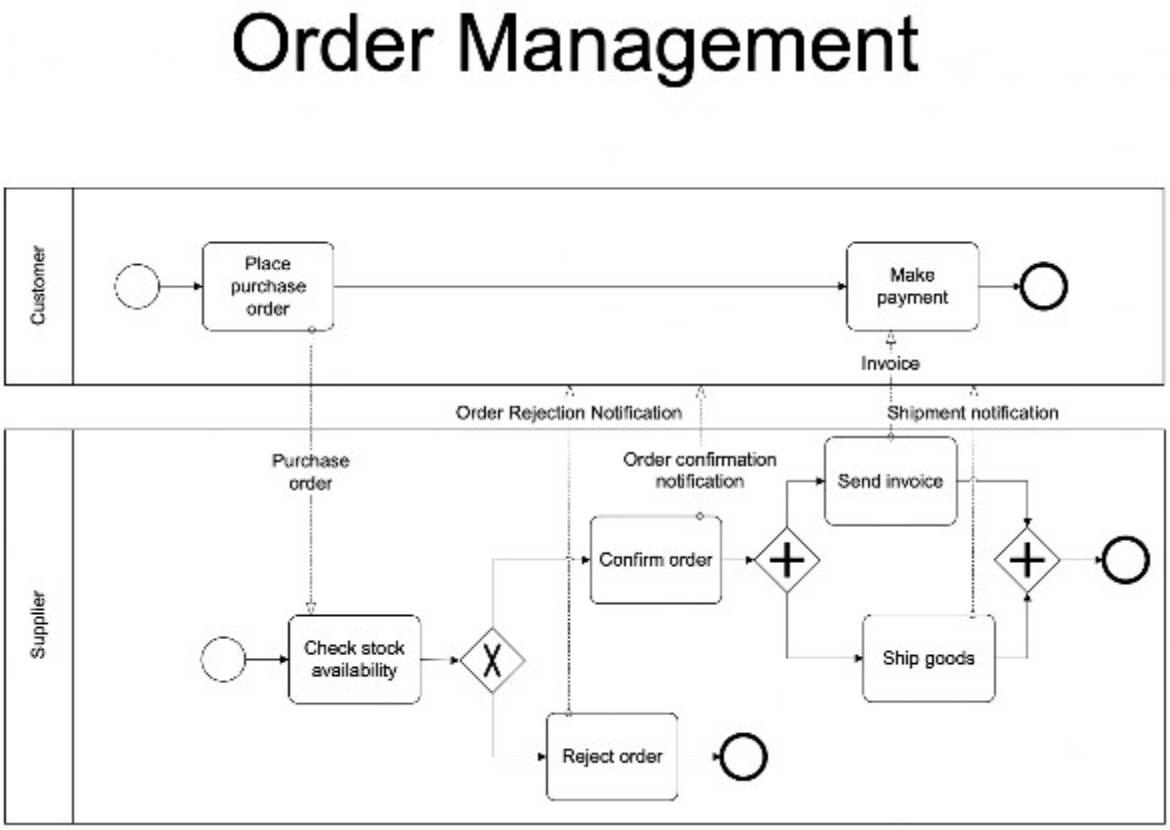
La caratteristica è che chi va a leggere il modello riesce a capire esattamente quello che lo voleva descrivere grazie agli elementi della rappresentazione, senza ambiguità.

Model with basic attributes



BPMN

Il linguaggio più utilizzato per rappresentare i processi aziendali è il **BPMN** (Business Process Modeling Notation), è simile al linguaggio ER ma si focalizza sul **flusso delle attività**. Possiamo ad esempio verificare se c'è la possibilità che si verifichi qualche scenario particolare (che magari non dovrebbe poter succedere)
Il BPMN è uno standard, altri linguaggi simili come l'activity diagram di UML che vedremo dopo non sono accetttati, perché il suo uso è ristretto per software object-oriented



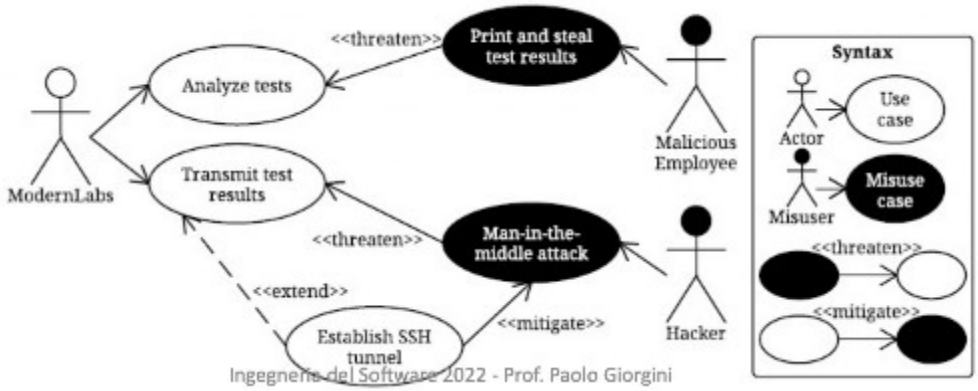
Linguaggi di modellazione in SRE

Parlando di sicurezza, noi possiamo fare dei sistemi sicuri, con una chiave “importante”, ma poi lasciamo la chiave a disposizione, quindi si può sfruttare questa vulnerabilità della persona fisica (la donna delle pulizie perde la chiave del nostro ufficio). Importante è usare alcuni linguaggi che ci permettono di catturare alcuni aspetti della sicurezza

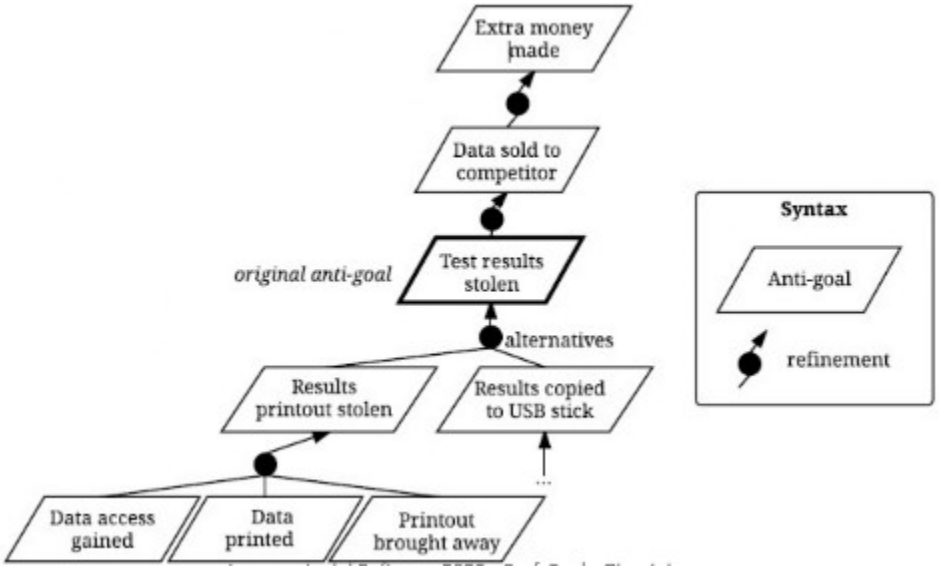
Molti approcci SRE (Security Requirements Engineering) usano linguaggi di modellazione per progettare la sicurezza, con una precisa documentazione e con sistemi di auto analisi. Facciamo qualche esempio:

- **UML, immaginando i casi di uso improprio**
  Possiamo cercare di capire come un hacker potrebbe intervenire, senza parlare del codice ma ancora in fase di analisi. Ovviamente dobbiamo analizzare se ha senso spendere risorse per risolvere un buco di sicurezza che magari non “merita” spreco di

risorse per tapparlo (i rischi non meritano di spendere tante risorse per eliminarli).



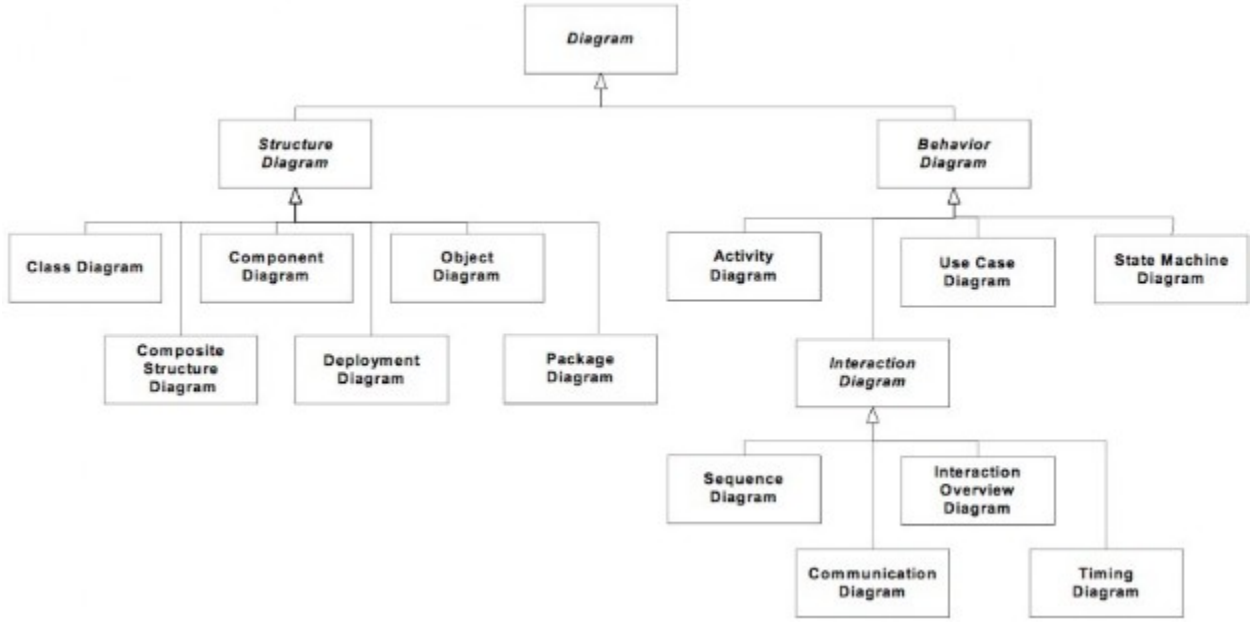
- **Obiettivo dell'attacco**
  Invece di pensare all'attacco, possiamo pensare all'obiettivo dell'attacco, ovvero perché sta attaccando.



Esistono quindi modelli che oltre ai requisiti funzionali evidenziano anche i requisiti di sicurezza, che diventano un aspetto su cui fare ragionamenti anche a livello di analisi e progettazione

UML

UML è lo standard principale per la creazione di diagrammi per progetti.
Abbiamo vari tipi di diagrammi in base a cosa vogliamo ad andare a descrivere, le due divisioni principali sono la rappresentazione della struttura oppure la rappresentazione del comportamento:



I **class diagram** mostrano i vari tipi di associazioni tra le classi in un programma Object Oriented
Gli **object diagram** rappresentano delle possibili istanziazioni di un class diagram.
I **component diagram** vengono utilizzati quando si adopera uno sviluppo a componenti.
I **deployment diagram** si usano al momento della distribuzione del software, mostrando ad esempio i sistemi operativi e altre cose più “pratiche”
Gli **use case diagram** sono utili per vedere i vari ruoli che gli utenti possono assumere in un software, avendo chiaro che azioni possono eseguire e cosa possono vedere
Gli **activity diagram** (già accennati prima) sono molto simili ai BPMN
I **sequence diagram** vanno ad individuare le interazioni tra le diverse classi di un software, con la possibilità di vedere i vari “percorsi” che si possono prendere

OCL

Il linguaggio OCL (Object Constraint Language) è puramente funzionale per scrivere espressioni e vincoli sui modelli object oriented. È proposto come estensione di UML dagli standard.
È un linguaggio composto da query:

- Valutazione di **espressioni** (valore)
- Una espressione è una specifica o riferimento ad un valore
- Un vincolo è una restrizione su uno o più valori o parti di un modello orientato agli oggetti

Le **espressioni** possono essere usate in diversi diagrammi UML per:

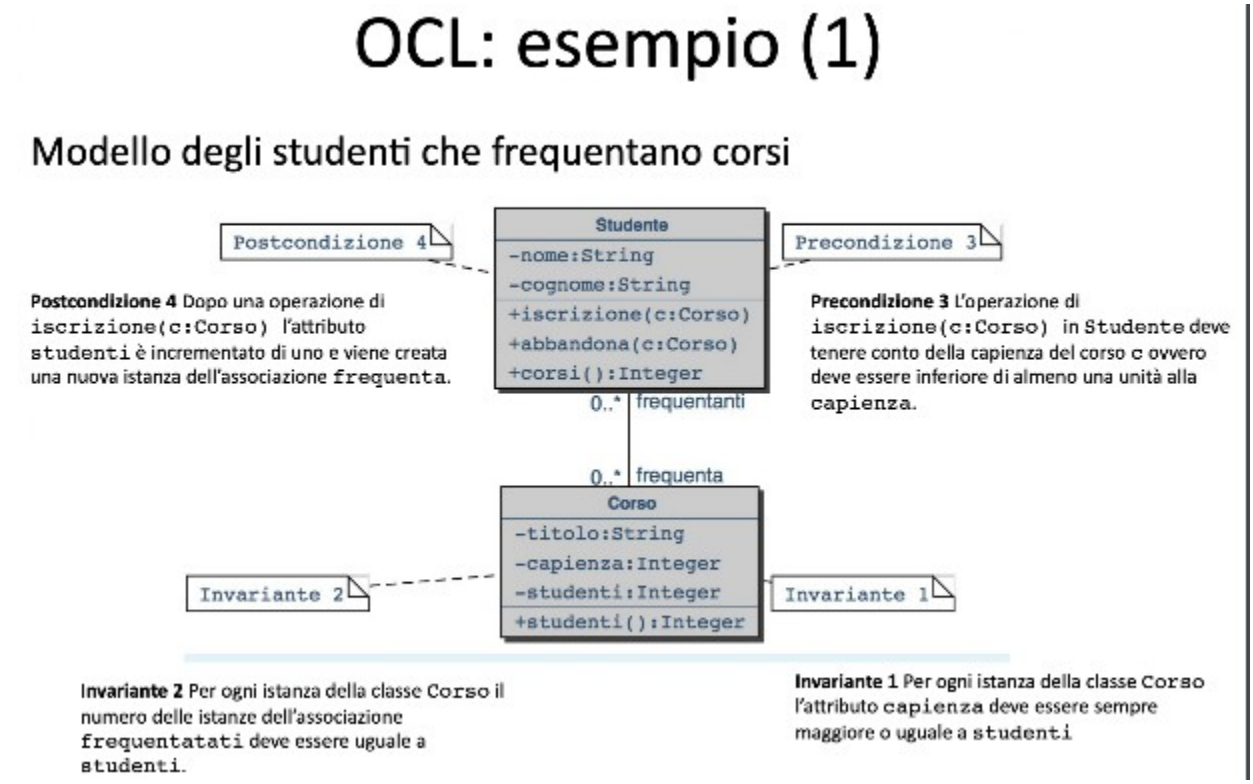
- Specificare un **valore iniziale** di un attributo o associazione
- Specificare un **valore derivato** di un attributo o associazione
- Specificare il **corpo di un’operazione**
- Indicare un’**istanza di un diagramma dinamico**
- Indicare una **condizione in un diagramma dinamico**

- Indicare il **valore attuale di un parametro in un diagramma dinamico**

I **vincoli** invece possono essere:

- Delle **invarianti**
  Delle condizioni che devono essere sempre valide per tutte le istanze di un'espressione o interfaccia
- Delle **precondizioni**
  Delle restrizioni che devono essere vere prima dell'esecuzione di un'operazione
- Delle **postcondizioni**
  Delle restrizioni che devono essere vere subito dopo l'esecuzione di un'operazione
- Delle **guardie**
  Dei vincoli che devono essere veri in una fase della vita del sistema (init, derive)

Esiste anche il **contesto** di un'espressione, ovvero la specificazione della parte (una classe, un'interfaccia, un tipo di dato, un componente) su cui agisce l'espressione.
Molte volte la parte interessata è un'operazione o un attributo, raramente un'istanza.



OCL: esempio (2)

```
-- Invariante 1
   contex Corso
   inv: capienza>=frequentanti.size()

-- Invariante 2
   contex Corso
   inv: studenti=frequentanti.size()

-- Pre e post condizione 3 e 4
   context Studente::iscrizione(c:Corso)
   pre: c.capienza > c.studenti
   post: c.studenti=c.studenti@pre+1
         and c.frequentanti=c.frequentanti@pre+1
```

Requisiti

Un **requisito** viene fuori quando qualcuno ha un obiettivo/un bisogno, e non è il **come** il sistema verrà fatto ma il **cosa** il sistema dovrà fare.
I bisogni di un cliente possono essere espressi come **scopi** (obiettivi di un business).
Per progettare un sistema che permetta di conseguire uno scopo dovremo specificare una serie di **funzioni**, la relazione tra scopi e funzioni che li conseguono è il documento dell'**analisi dei requisiti**.
È molto importante capire anche il contesto, l'ambiente; per poter soddisfare un **requisito**, noi dobbiamo mettere assieme le **assunzioni sull'ambiente** e **le specifiche del sistema**.
Ovviamente capita che potrei creare un sistema che soddisfa la specifica ma che però non soddisfa il requisito (vuol dire che la specifica è stata scritta male), e ovviamente questo non va bene visto che i requisiti sono alla base del progetto.

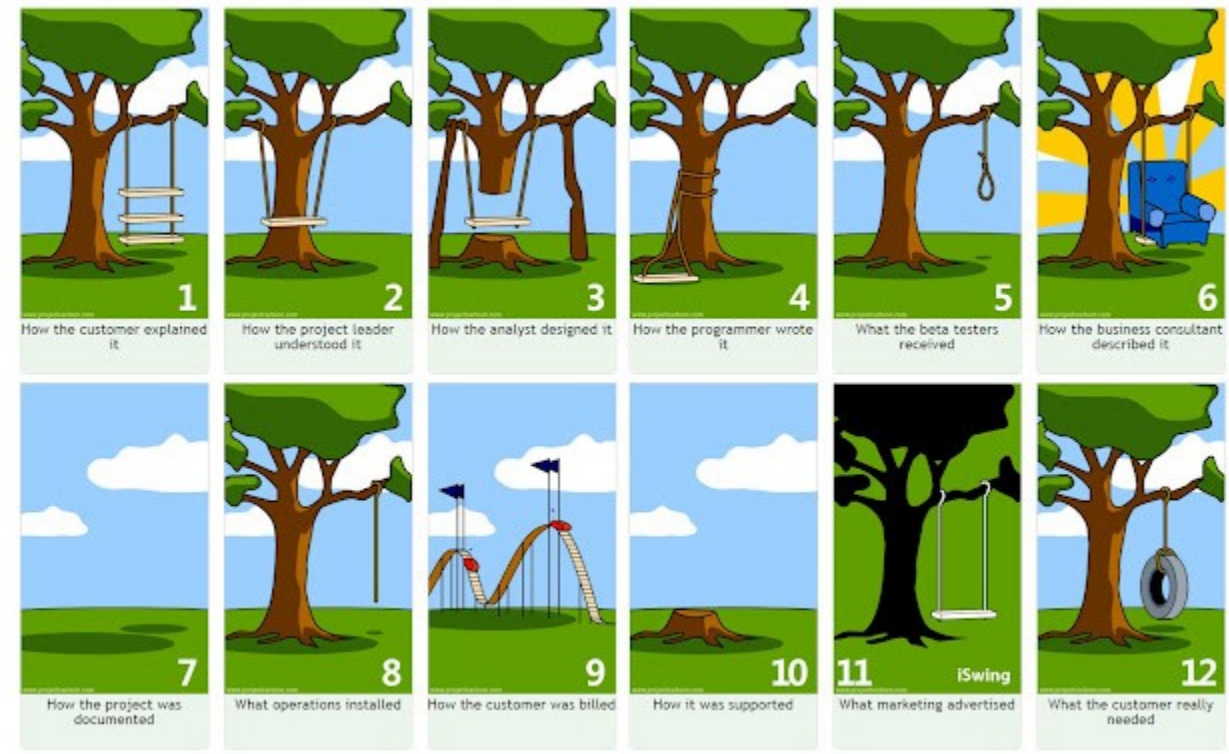
Un esempio di requisiti per un software che esegue il calcolo di Fibonacci sono:

- $F(0)=1$
- $F(1)=1$
- $F(n)=F(n-1)+F(n-2)$

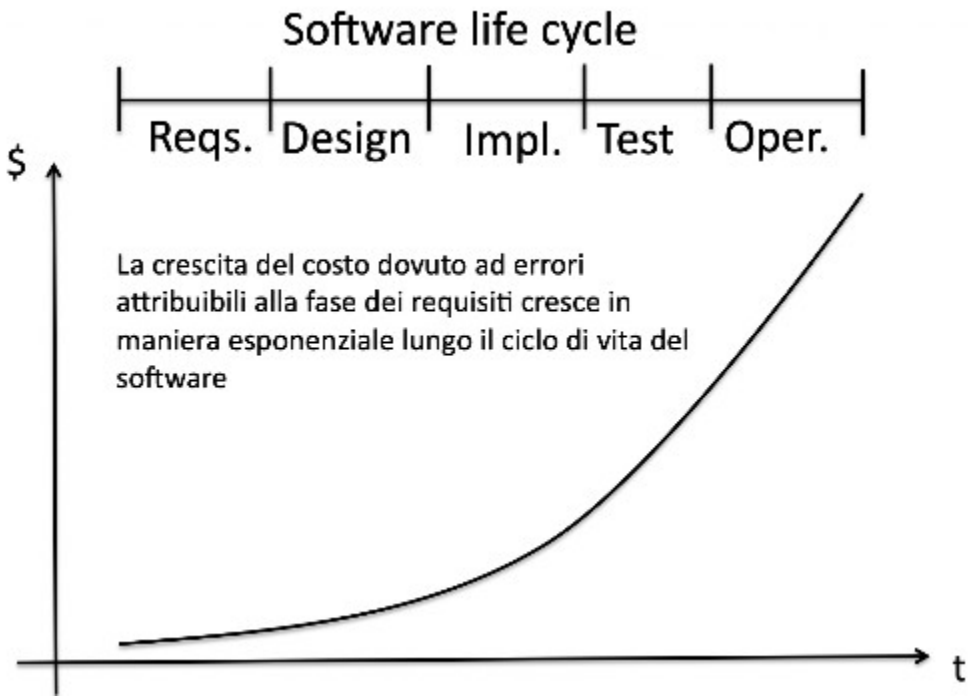
La **definizione dei requisiti** è scritta solitamente in un linguaggio normale, e si usa per mostrarla al cliente, così che dia un'opinione e di conseguenza capire se abbiamo raccolto correttamente i requisiti.

La **specifica dei requisiti** invece dettaglia i servizi attesi dal sistema (specifiche tecniche e funzionali) e viene scritta per darla in mano agli sviluppatori che poi procedono, per l'appunto, con la fase di sviluppo.

Il processo di **elicitazione dei requisiti** è molto complesso e delicato e serve per stabilire **cosa** richiede il cliente da un sistema software (**scopo**). Questo processo può coinvolgere diverse persone e può anche comportare l'analisi della legislazione (ovviamente non si possono fare cose illegali) e l'analisi di realizzazioni già esistenti.
Tutte le persone che vengono coinvolte in questo processo sono gli **stakeholders** (le parti interessate, coloro che mettono i paletti). Gli stakeholders **difficilmente hanno un'idea chiara** di quello che vogliono e usano un loro **linguaggio**, tecnico rispetto al loro campo.
In più i diversi stakeholders hanno richieste diverse che possono andare in **conflitto**.
I requisiti e gli stakeholder possono, per vari motivi, cambiare durante le fasi di analisi



Commettere un errore nei requisiti può avere conseguenze importanti, come ad esempio il superamento di budget, oppure delle feature che non verranno mai utilizzate (quindi spreco di risorse per funzionalità "inutili")



Il 32% dei difetti di un software sono dovuti a errori nei requisiti, i quali vengono rilevati solamente quando il software è già operativo

Capiamo quindi meglio cosa è un requisito per poter definire e individuare per bene:

- Descrivere un requisito
  - Cosa deve fare il software
  - Che interfacce ha con i suoi utenti, con l'hw o con altri prodotti sw
  - Che prestazioni deve esibire il software
  - Quali attributi dovrà avere
  - Quali vincoli deve soddisfare
- Cosa non è un requisito
  - L'architettura del sistema
  - Vincoli tecnologici (quale linguaggio di implementazione usare)
  - Il processo di sviluppo
  - L'ambiente di sviluppo
  - Il sistema operativo di riferimento
  - Aspetti di portabilità e riusabilità

Categorie

Possiamo dividere i requisiti in due categorie: i requisiti **funzionali** e i requisiti **non funzionali**.

I **requisiti funzionali** descrivono le **funzionalità** del sistema software, in termini di **servizi** che il sistema software deve fornire, di come il sistema software **reagisce** a specifici tipi di input e di come si **comporta** in situazioni particolari. Un requisito funzionale quindi definisce una funzione di sistema, inclusi il suo ingresso e la sua uscita.

Non dobbiamo fare l'errore di iniziare a pensare alla implementazione/realizzazione perché è sbagliato farlo in fase di analisi dei requisiti; ed è importante in fase di realizzazione annotare in qualche modo che requisito soddisfa quella sezione, così da aumentare la tracciabilità (se cambio il requisito so dove mettere le mani)
I requisiti funzionali determinano lo sviluppo del codice, ed è su di loro che si baserà la fase di **verifica**
Le specifiche dei requisiti funzionali devono essere **complete** (tutti i servizi richiesti devono essere definiti) e **coerenti** (non devono avere definizioni contraddittorie, molto probabile quando ci sono più stakeholders). Questo crea una difficoltà non da poco nel caso di sistemi grandi e complessi.

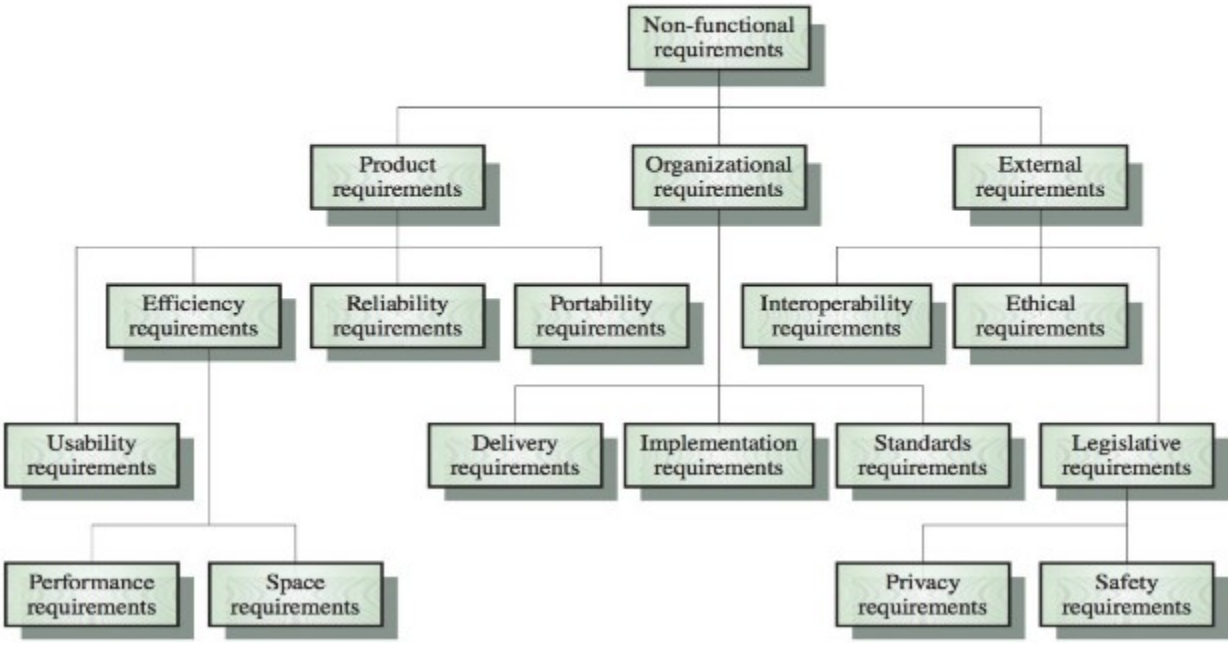
I **requisiti non funzionali** (non stiamo parlando di una funzione specifica) descrivono le **proprietà** (qualità) del sistema software, e non riguardano le specifiche funzioni del sistema.
Degli esempi di requisiti non funzionali sono:

- Caratteristiche di efficienza (affidabilità, security, safety)
- Caratteristiche del processo di sviluppo (standard di processo, linguaggi di programmazione, metodi di sviluppo, …)
- Caratteristiche esterne (interoperabilità con sistemi di altre organizzazioni, vincoli legislativi)

Durante lo sviluppo ci sarà una verifica costante dei requisiti non funzionali.

Importante notare l'importanza dei requisiti non funzionali, perché un utente può anche fare a meno di una funzionalità mancante/sbagliata, ma se non si soddisfa un requisito non funzionale si rischia che l'intero sistema diventi inutilizzabile in certe situazioni (se il sistema di gestione del traffico aereo non soddisfa i requisiti di affidabilità non potrà ottenere la certificazione di sicurezza necessaria per operare)

Classificazione Reqs. non funzionali



I requisiti non funzionali sono difficili da verificare, a differenza di quelli funzionali visto che la verifica di una funzione è oggettiva, o è stata implementata correttamente oppure no.
Bisogna quindi definire delle metriche (successivamente una tabella con le varie misure) con il cliente per capire se è soddisfatto o meno del requisito non funzionale. Le metriche però non sono facili da definire quando gli obiettivi sono troppo **generali** o **vaghi**, visto che si lascia l'interpretazione agli sviluppatori, che spesso non coincide con quella del cliente.
Ecco alcune delle proprietà non funzionali con le rispettive unità di misura:

Proprietà	Misura
Velocità	Transazioni elaborate al secondo Tempi di risposta a utenti/eventi Tempo di refresh dello schermo
Dimensione	Kbyte, Mbyte, Gbyte Numero di circuiti RAM
Facilità d'uso	Tempo di addestramento Numero di maschere di aiuto
Affidabilità	Tempo medio di malfunzionamento Probabilità di indisponibilità Tasso di malfunzionamento Disponibilità
Robustezza	Tempo per il riavvio dopo il malfunzionamento Percentuale di eventi causanti malfunzionamento Probabilità di corruzione dei dati dopo malfunzionamento
Portabilità	Percentuali di dichiarazioni dipendenti dall'architettura di destinazione Numero di architetture di destinazione

Documento di analisi dei requisiti

Il **documento di analisi dei requisiti**, o anche detto anche **documento di specifica**, è un documento ufficiale che descrive in dettaglio le caratteristiche del sistema da sviluppare, includendo sia la definizione dei requisiti che la loro specifica.
Questo documento descrive il **cosa** il sistema deve fornire (**dominio del problema**) e non **come** il sistema deve essere sviluppato (**dominio della soluzione**).
Ci sono degli standard, dei template da utilizzare, così si è forzati a esplicitare le parti interessate dallo standard o dal tipo di software.

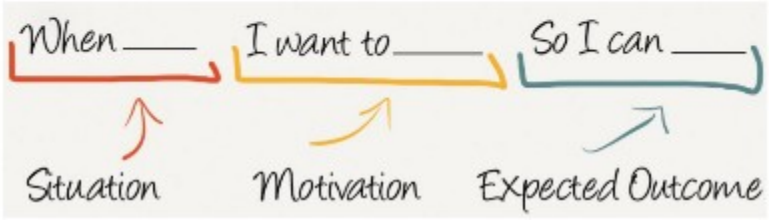
I requisiti possono essere definiti in varie forme:

- **Classica**: "l'app permetterà di…"
- **User story**: "come studente voglio…"
- **Use case**: "uno studente … per …, l'app chiederà …"

In una modalità **user story** (usate nei modelli agile) devo **immedesimarmi in un utente** che hanno la seguente forma:

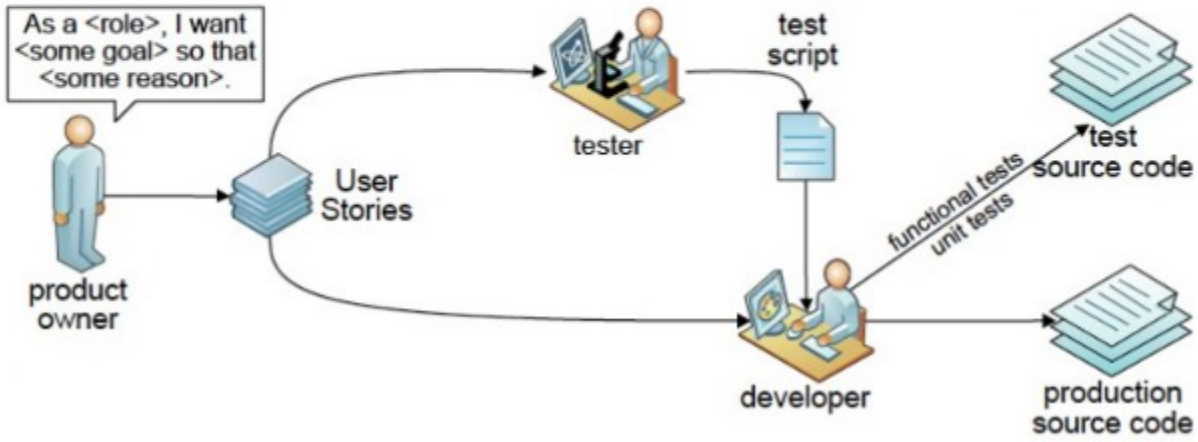
As a <*user type*>
I want <*to do something*>
so that <*I get some value*>

Come *studente UNITN (ruolo)*
Voglio *richiedere autorizzazione (scopo o funzione)*
Per *posso accedere all'aula (valore otttenuto)*



Il Product Owner scrive le storie (di solito sufficientemente piccole da poter essere realizzate in una/due settimane) e le loro "Definition of Done", ovvero il criterio di accettazione.

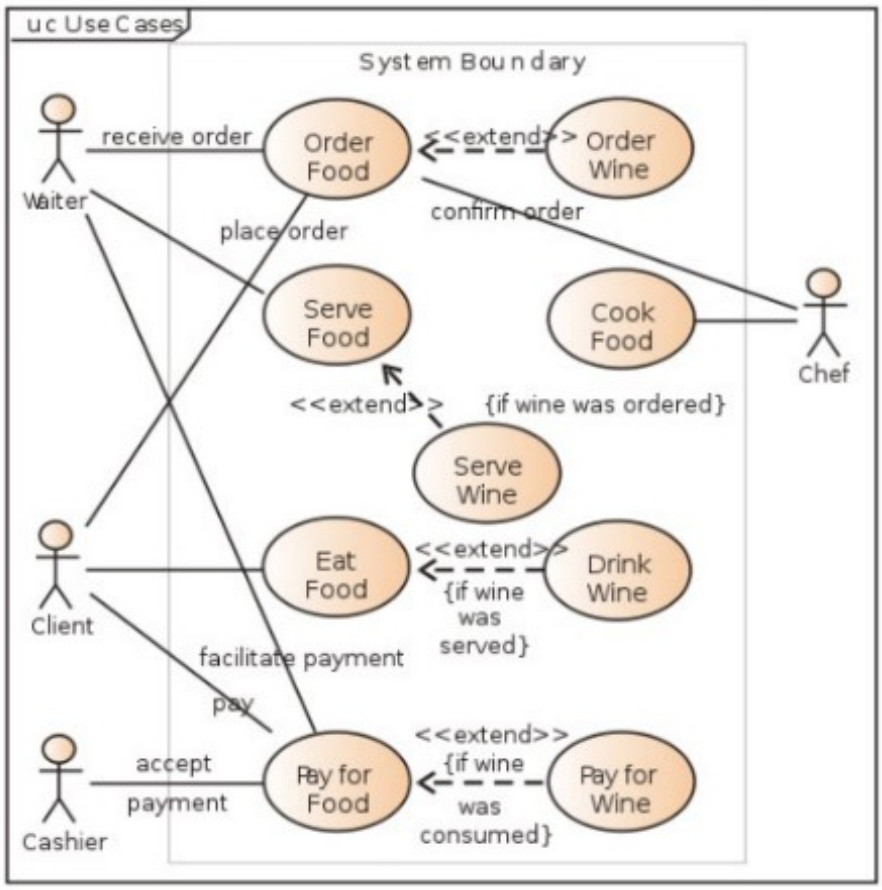
User story nel processo agile



In una modalità **use case** (casi d'uso) si fa uno studio degli **scenari operativi** (possibili modi di utilizzo) del sistema. Un caso d'uso rappresenta un requisito **funzionale**. Quindi descrivono una sequenza di azioni **visibili** dalla visuale di un **attore**.
Gli use case si utilizzano sia per descrivere i requisiti d'utente sia per controllare il sistema (testing e verifica).

Ogni scenario (che viene rappresentato da uno use case) rappresenta **l'interazione di entità esterne**, dette **attori**, con il sistema e le sue componenti, separando il flusso principale dalle varianti alternative.
L'attenzione quindi si **focalizza sull'interazione** e non sulle attività interne al sistema.

Esistono **scenari base** che prevedono il successo del caso d'uso, con uno svolgimento lineare. Poi ci sono gli **scenari alternativi** che possono essere di successo o fallimento, con complicazioni varie (non bisogna analizzare tutti i possibili scenari, ma solamente le varianti che comportano trattamenti particolari)



La gestione dei requisiti è complicata per quanto riguarda:

- Conflitti
- Relazioni
- Tracciabilità
- Cambiamenti
- Documentazione

Esistono diversi tool avanzati che semplificano questa gestione.

Documento di progetto

Il documento di progetto è un documento che descrive gli obiettivi generali del sistema software da realizzare e comprende:

- Requisiti funzionali
- Requisiti non funzionali
- Front end
- Back end

Come già detto i **requisiti funzionali** si riferiscono ad una funzionalità del sistema, descrivono un comportamento.
I **requisiti non funzionali** invece, si riferiscono ad una proprietà del sistema, descrivono delle caratteristiche di prestazione (ad esempio requisiti di portabilità, compatibilità, capacità, affidabilità, operatività, localizzazione, usabilità, accessibilità).

È molto importante chiedersi il **perché** di un requisito non funzionale, e motivare la scelta di includerlo. I requisiti non funzionali sono spesso legati ai requisiti funzionali

Lo scopo del **front end** è quello di **disegnare e illustrare testualmente** il funzionamento del sistema dal punto di vista dell'utente. Il **mockup** non deve essere completo, ma deve illustrare le schermate più significative e gli aspetti più critici dei requisiti funzionali.
Bisogna pensare al front end come un complemento alla descrizione dei requisiti

Lo scopo del **back end** è quello di descrivere i sistemi esterni con cui il sistema da realizzare si dovrà interfacciare per poter funzionare. Può anche comprendere uno schema che illustra le interazioni.

L'**analisi dei requisiti** viene solitamente eseguita in un linguaggio naturale usando un editor di testo (Microsoft Word, Libreoffice Writer).
Ci sono dei problemi tipici in questa fase:

- Problemi nel capire come funziona il sistema
- Ambiguità e incompletezze nelle specifiche
- Gestione dei requisiti (impatto dei requisiti, tracciabilità, etc)

Pensiamo quindi ad un nuovo approccio con l'obiettivo di definire i requisiti del prodotto ma producendo il Documenti di analisi dei requisiti con diagrammi UML (non solo ma sono i più utilizzati) per i requisiti funzionali (use case o sequenziale) e tabelle per i requisiti non funzionali.

Documento di specifica

Use case diagram

In un diagramma use case per l'analisi dei requisiti funzionali dobbiamo:

- Identificare gli attori
  L'insieme di "ruoli" che l'utente può avere. Gli attori non sono necessariamente esseri umani. Non bisogna pensare che gli attori siano istanze, bensì ruoli appunto (una persona può ricoprire diversi ruoli in un sistema).

Gli attori sono **esterni** al sistema e aiutano a definire i “confini” del sistema, di conseguenza il sistema non può essere un attore

- Identificare gli use case
  L’insieme degli scenari descrivendo le funzionalità in maniera coerente **dal punto di vista dell’utente**
- Disegnare il diagramma UML - Use Case Diagram
  Un diagramma che comprende gli use case, gli attori e le relazioni tra gli attori e gli use case (in base a quali attori prendono parte agli use case)

A volte è necessario rappresentare delle funzionalità che si svolgono regolarmente (come un backup), ci sono due modi per scegliere l’attore a cui collegare questi use case:

- Chi beneficia dall’esecuzione di questa azione regolare
- Cosa innesca l’azione (tipo l’orologio)

Esistono vari tipi di **relazioni** in uno use case diagram:

- Relazioni con attori
  - Associazione
    La partecipazione di un attore in uno use case. Questa è l’unica relazione esistente tra attori e use case.



  - Generalizzazione
    Se A generalizza B allora A può “eseguire” tutti gli use case di B



- Relazioni tra use case
  - Inclusione
    Se A include B allora uno step di A è l’esecuzione di B



- Estensione
  Se C estende B allora C fa qualcosa in più di D



Per limitare l'uso degli use case dobbiamo organizzarli e classificarli, ad esempio dividendoli in sottosistemi. Quindi avremo una visione generale più ad alto livello (utile per comunicare con gli stakeholder) e poi avremo i vari sottosistemi che presi singolarmente mostreranno gli use case di basso livello.

Per ogni use case bisogna scrivere la sequenza di eventi che vengono effettuati, descritti dal punto di vista dell'attore. Bisogna scrivere in dettaglio cosa il sistema dovrà eseguire e fornire agli attori quando viene eseguito quel use case. I punti da definire solitamente sono:

- Come lo use case inizia e come finisce
- Il normale flusso di eventi
- I flussi alternativi di eventi
- I flussi di eventi quando c'è una eccezione

C'è un formato per la descrizione degli use case ma non è obbligatorio:

Title
Summary
Description
Exceptions
Extension

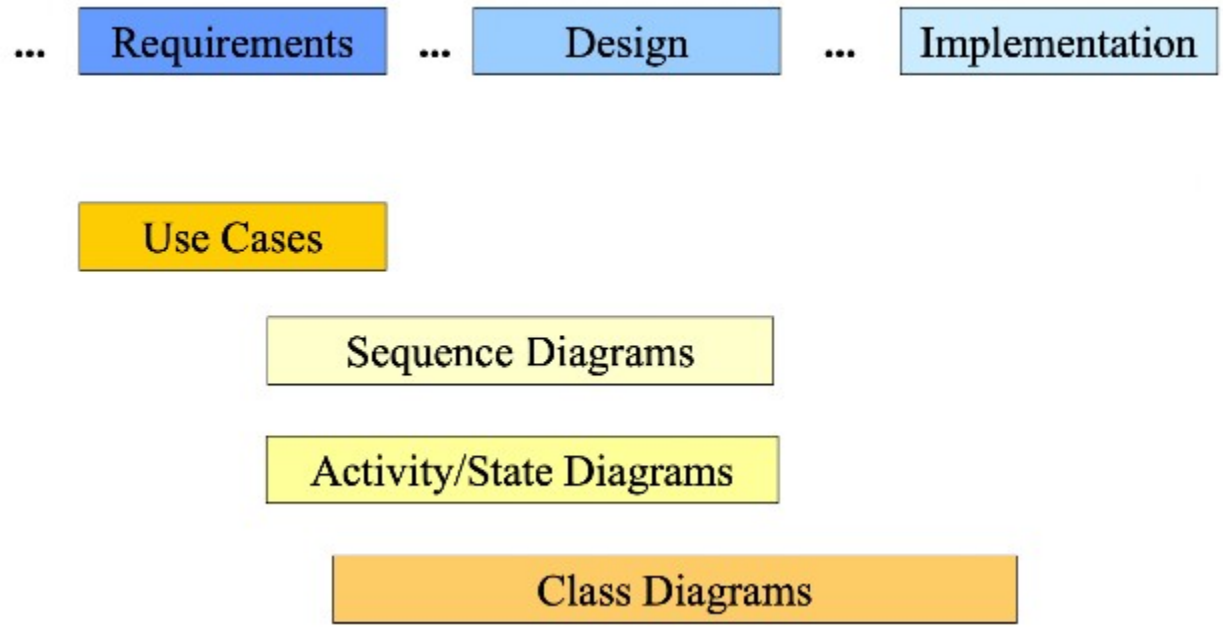
Tabella requisiti non funzionali

I requisiti non funzionali definiscono le proprietà, le qualità ed i vincoli sulle funzioni di un software (ad esempio performance, affidabilità, sicurezza, etc). È importante che le proprietà siamo **misurabili**
Ecco alcuni esempi:

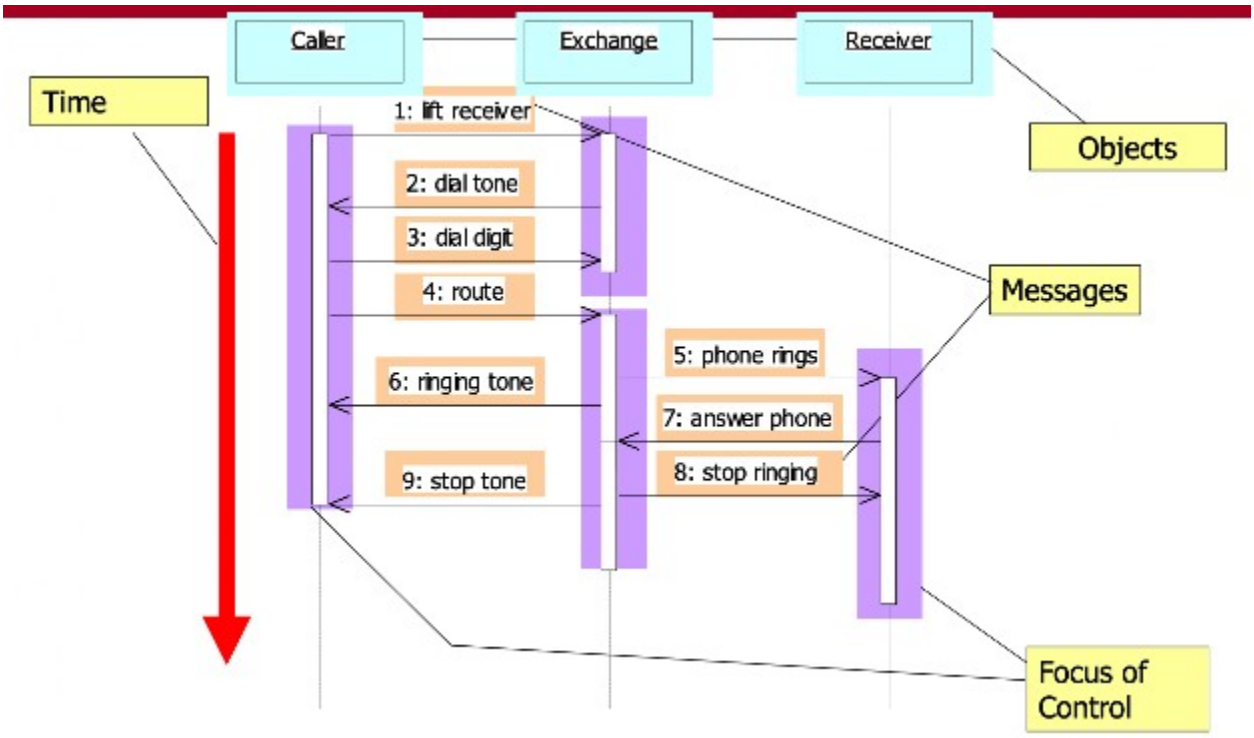
Proprietà	Misura
Velocità	Transazioni elaborate al secondo Tempi di risposta a utenti/eventi Tempo di refresh dello schermo
Dimensione	Kbyte, Mbyte, Gbyte Numero di circuiti RAM
Facilità d'uso	Tempo di addestramento Numero di maschere di aiuto
Affidabilità	Tempo medio di malfunzionamento Probabilità di indisponibilità Tasso di malfunzionamento Disponibilità
Robustezza	Tempo per il riavvio dopo il malfunzionamento Percentuale di eventi causanti malfunzionamento Probabilità di corruzione dei dati dopo malfunzionamento
Portabilità	Percentuali di dichiarazioni dipendenti dall'architettura di destinazione Architetture di destinazione

Descrivere lo use case diagram

Ecco uno schema che descrive i vari diagrammi usati nelle varie fasi di un progetto:



Gli **use case diagram** descrivono il sistema da un **punto di vista esterno**.
I **sequence diagram** sono ottimi per rappresentare come gli oggetti **collaborano** e si **interfacciano**.
Ecco un esempio di sequence diagram:



Notiamo che i sequence diagrams hanno degli elementi fondamentali che lo caratterizzano:

- Gli **oggetti** (quelli in blu)
  Si possono vedere come gli attori, sono le entità che compiono qualcosa nel nostro sistema
- Il **tempo** (la freccia in rosso)
  Cosa non presente negli use case diagrams, così facendo possiamo descrivere la sequenza di operazioni che vengono eseguite.
  Il tempo può essere non specificato (quindi conta solo la sequenza), oppure diviso in unità fisse.
- I **messaggi** (quelli numerati in arancione)
  Si possono vedere come gli use case, sono la comunicazione che avviene tra due oggetti che devono scambiarsi qualcosa (anche semplicemente l'informazione di aver finito un'operazione)
- I **focus di controllo** (le linee verticali in viola)
  Determinano il periodo in cui un oggetto sta compiendo azioni, è attivo.

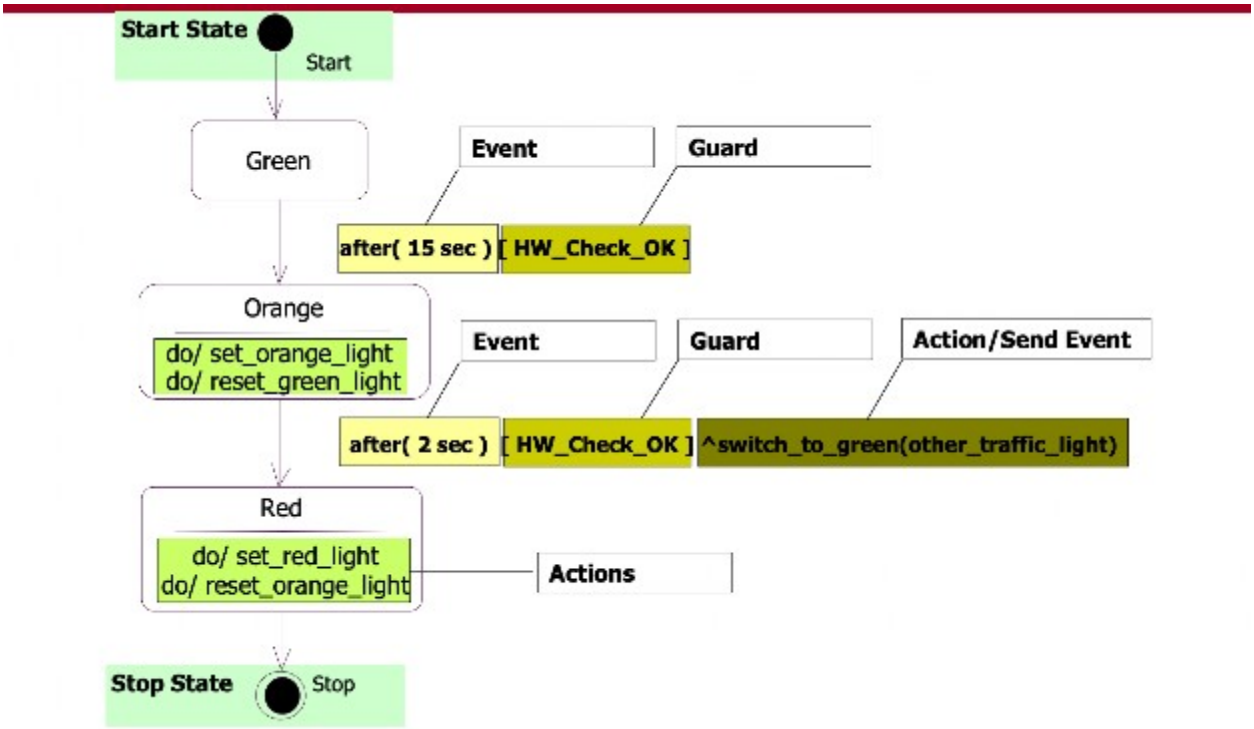
Durante le specifiche e lo sviluppo capita molto spesso di dover incontrare la necessità di rappresentare **comportamenti dinamici** (ad esempio per rappresentare workflow, use case complessi, i cicli di vita degli oggetti etc).
UML mette a disposizione vari diagrammi per rappresentare questi comportamenti dinamici, tra cui:

- State Machine Diagram
- Activity Diagram (possono essere visti come un tipo speciale di State Diagram)

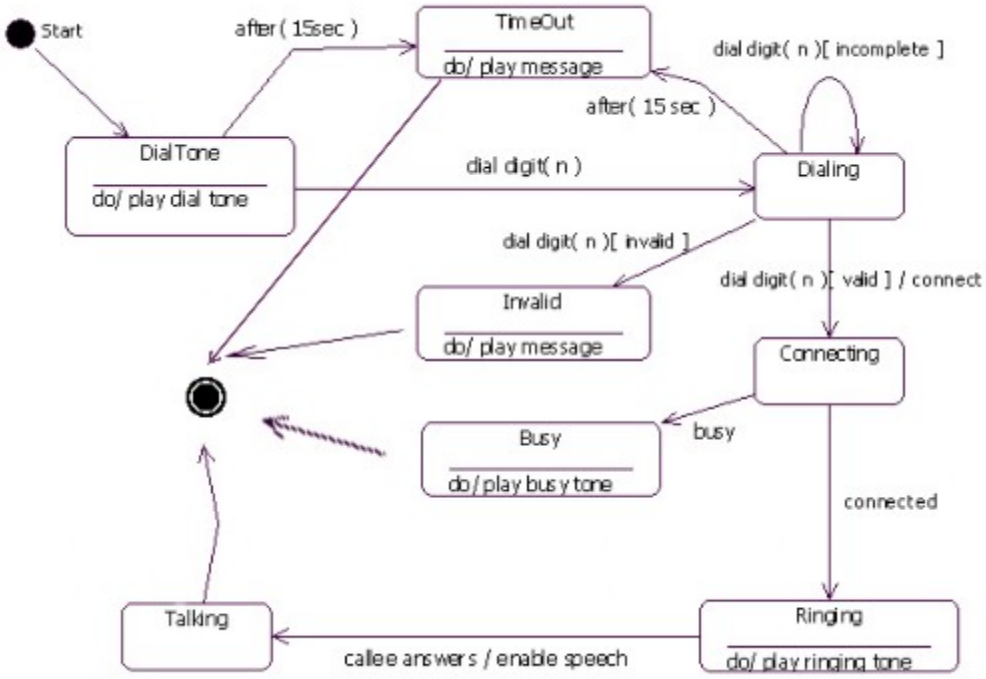
Uno **State Machine Diagram** descrive la sequenza degli **stati** e delle **azioni** attraverso cui un elemento può passare durante il suo ciclo di vita, ad esempio come reazione ad altri eventi.
Quindi gli elementi chiave di questo diagramma sono:

- Gli **stati** (semplici e compositi)
    - entry (fai un'azione quando si entra in questo stato)
    - do (continua a fare un'azione fino a che è in questo stato)
    - exit (fai un'azione quando si esce da questo stato)
- Le **transizioni**
    - Eventi (cosa è che causa la transizione)
    - Guardia (controlla gli eventi, se è vero consente la transizione, altrimenti no)
    - Eventi di azione/invio (se c'è una transizione, allora esegui questi eventi)

Ecco un esempio:



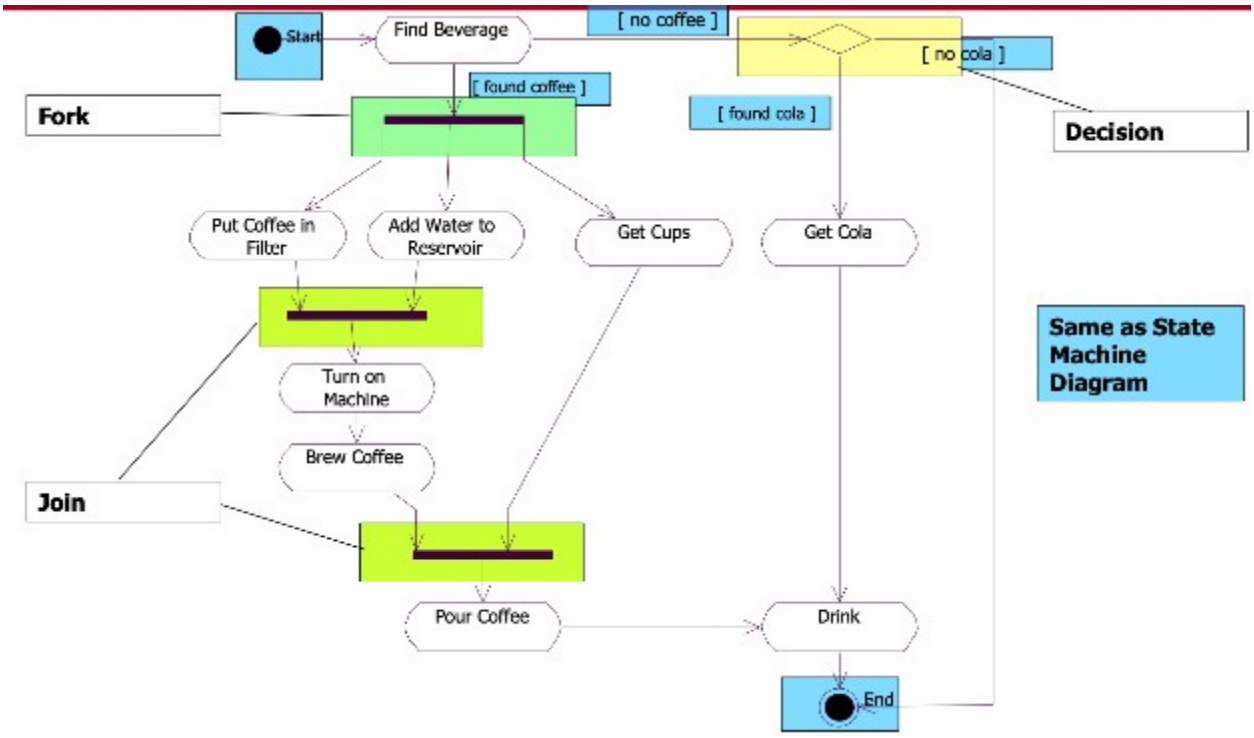
Abbiamo elementi come **stato di partenza** e **stato finale**.
Un problema di questo diagramma è che le **informazioni strutturali** vengono **perse**.
Altro esempio di state diagram:



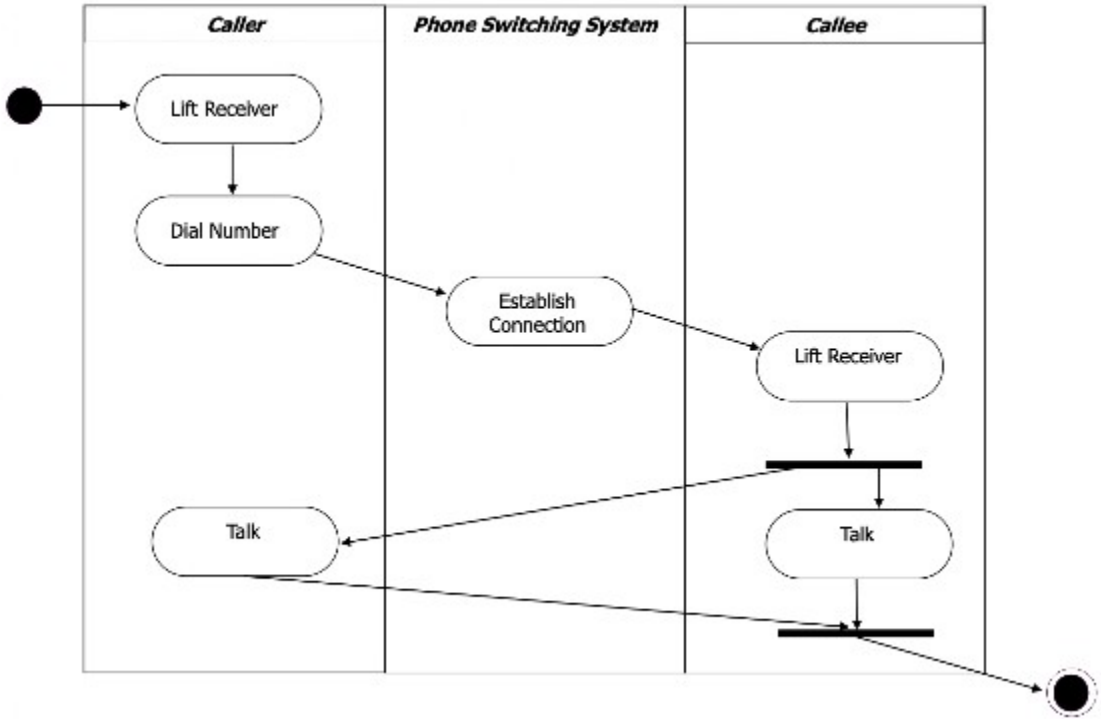
Vediamo ora cosa è un **activity diagram**, che come abbiamo detto è una specie di variazione di uno state diagram nel quale gli stati rappresentano lo **svolgimento delle azioni** e le transizioni sono "**innescate**" dal **completamento delle azioni**
Quindi gli elementi chiave di questo diagramma sono:

- Le attività (gli stati)
- Le transizioni (come negli state diagram)
- Merge
- Fork (permette la decomposizione parallela di eventi, quindi da un evento far partire n eventi)
- Join (permette di sincronizzare attività differenti, quindi far partire un evento dopo che sono stati effettuati n eventi)
- Decisioni

Ecco un esempio:



Spesso è utile poter evidenziare la **responsabilità** di una attività, ovvero chi la compie.
Gli activity diagram permettono di fare questo con le **swimlanes**, ad esempio:



La differenza principale tra **sequence diagram** e **activity diagram** è che i sequence diagram descrivono l'**intero sistema** (infatti gli oggetti sono gli attori e i messaggi sono gli use case).
Invece gli activity diagram descrivono **una parte del sistema** (solitamente uno **use case**).

Gli state e gli activity diagram dobbiamo usarli solo quando è **strettamente necessario** (ovvero quando diventa difficile descrivere uno use case con una spiegazione a parole).

In sintesi, per descrivere uno use case diagram:

- Italiano
  Se la spiegazione è semplice
- Sequence diagram
  Se il processo è lineare (non ci sono condizioni, la sequenza è quella ideale “senza eccezioni”)
- State diagram
  Permettono di descrivere processi ciclici e con condizioni per scegliere un percorso/variante diverso
- Activity diagram
  Permette di esprimere più task eseguiti in parallelo, e aggiungere il fatto che n tasks siano propedeutiche per cominciare la successiva e permette di inserire le “decisioni”
- Swimlanes
  È un sequence diagram con le fork e le propedeuticità, e anche le condizioni

Diagramma di contesto

Gli use case diagram e i sequence diagram rappresentano le **interazioni con gli utenti** e esiste una relazione tra **utenti e casi d’uso**.
Ora vediamo delle rappresentazioni che si concentrano sulle fasi di progettazione e sulle **interazioni del software** con gli altri sistemi **esterni**. Questo viene fatto in parte dall'analisi del back-end (già fatta) e in parte dal **diagramma di contesto**.

Il diagramma di contesto modella l’interazione tra il software e le **entità esterne**, definendone il flusso di informazioni. Viene usato infatti per definire le **interfacce esterne** del software.
Nell’analisi del contesto mettiamo da parte i requisiti per concentrarci a definire con chi **interagirà** il nostro sistema.
Questo ci serve perché sul testo dei requisiti il significato delle varie interazioni è interpretabile, invece in un diagramma di contesto l’interpretazione è univoca.

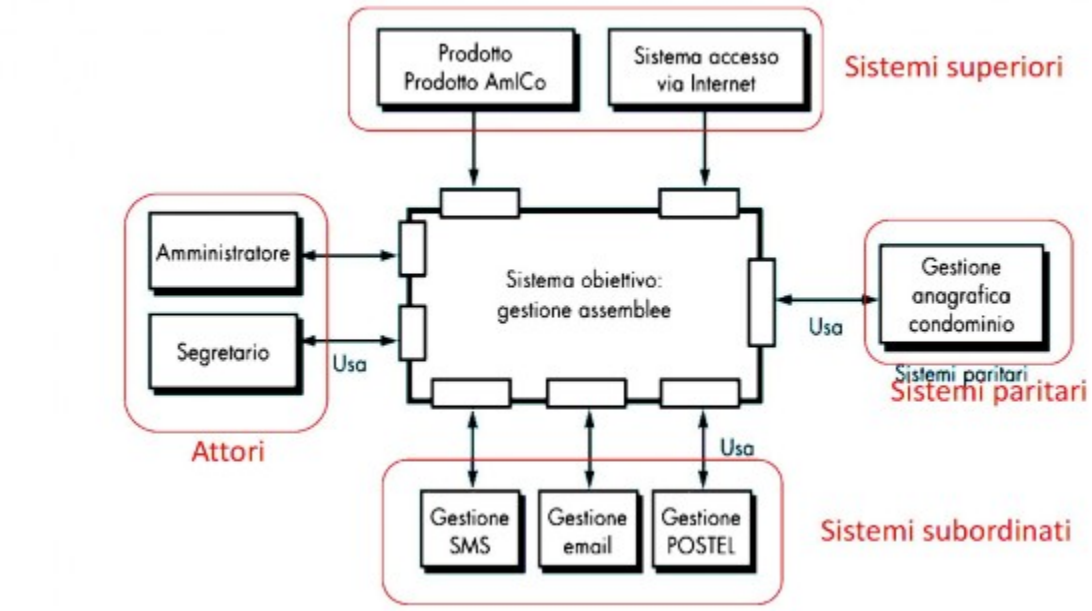
I diversi sistemi con cui il nostro software può interagire sono:

- **Sistemi superiori**
  Sono i sistemi “sopra” al nostro, ovvero che utilizzeranno le funzionalità il nostro software per fare altre cose
- **Sistemi subordinati**
  Sono i sistemi “sotto” al nostro, ovvero che il nostro software utilizzerà per fare altre cose
- **Sistemi paritari**
  Sono i sistemi che fanno parte del sistema più grande a cui appartiene il nostro software, dal punto di vista del sistema più grande questo sistema è allo stesso livello

del nostro. Solitamente sono delle parti del software del nostro sistema che non siamo noi a realizzare.

- **Attori**
  Sono le entità che producono o consumano informazioni necessarie per l'elaborazione delle richieste

Vediamo un esempio (i rettangoli sul bordo del sistema sono le interfacce che servono per comunicare con i sistemi esterni):



Per costruire un diagramma di contesto bisogna seguire alcuni punti importanti:

1. Identificare i **flussi di informazione** necessari al sistema, elencandoli
   (Ad esempio serve salvare i dati da qualche parte)
2. Identificare le **entità** esterne tramite l'individuazione delle sorgenti e dei destinatari dei flussi di informazione trovati al punto 1
   (Ad esempio un DBMS)
3. **Disegnare** ed etichettare il **sistema da sviluppare** (con un cerchio)
4. **Disegnare** ed etichettare le **entità esterne** attorno al sistema (con dei rettangoli)
5. Collegare i **flussi di informazione** tra il sistema e le entità esterne, specificando il tipo di dato che viene scambiato e la direzione
   NB: Gli scambi di informazioni interni al sistema vengono ignorati

Vediamo un semplice esempio di questa procedura:

Diagramma di contesto per un sito di e-commerce

(1) Identificare i flussi di informazione

- Mostrare i **prodotti** in vendita
- Ricevere gli **ordini** dei clienti
- Verificare la **disponibilità** degli articoli
- Richiedere **rifornimenti** degli articoli
- Ricevere i **pagamenti** dei prodotti

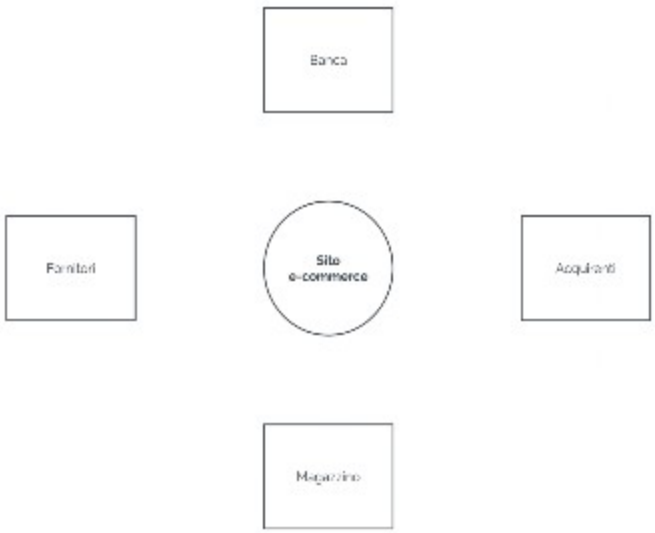
Diagramma di contesto per un sito di e-commerce

(1) Identificare i flussi di informazione

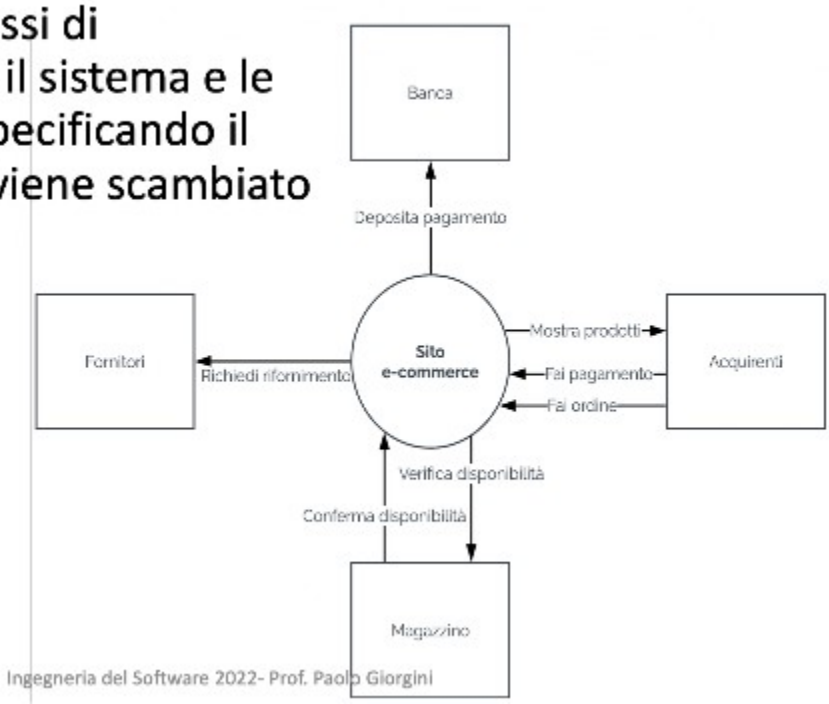
- Mostrare i **prodotti** in vendita → **Acquirente**
- Ricevere gli **ordini** dei clienti → **Fornitore**
- Verificare la **disponibilità** degli articoli
- Richiedere **rifornimenti** degli articoli → **Magazzino**
- Ricevere i **pagamenti** dei prodotti → **Banca**

(2) Identificare le **entità esterne** tramite l'individuazione delle sorgenti e dei destinatari dei flussi di informazione trovati al punto (1)

(3-4) Disegnare ed etichettare il sistema e le entità esterne



(5) Collegare i flussi di informazione tra il sistema e le entità esterne, specificando il tipo di dato che viene scambiato e la direzione



Di che tipo sono?

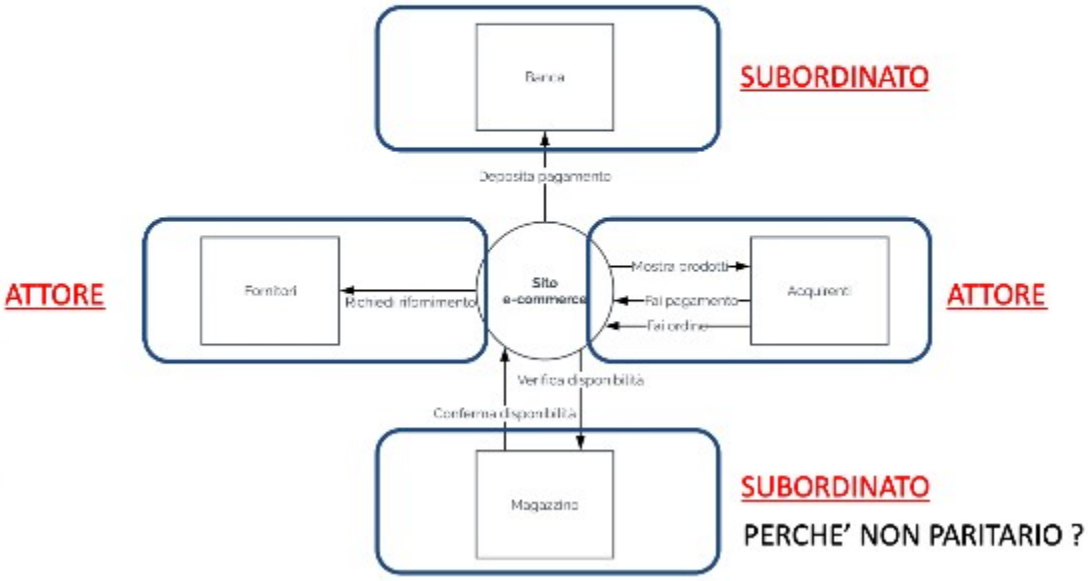
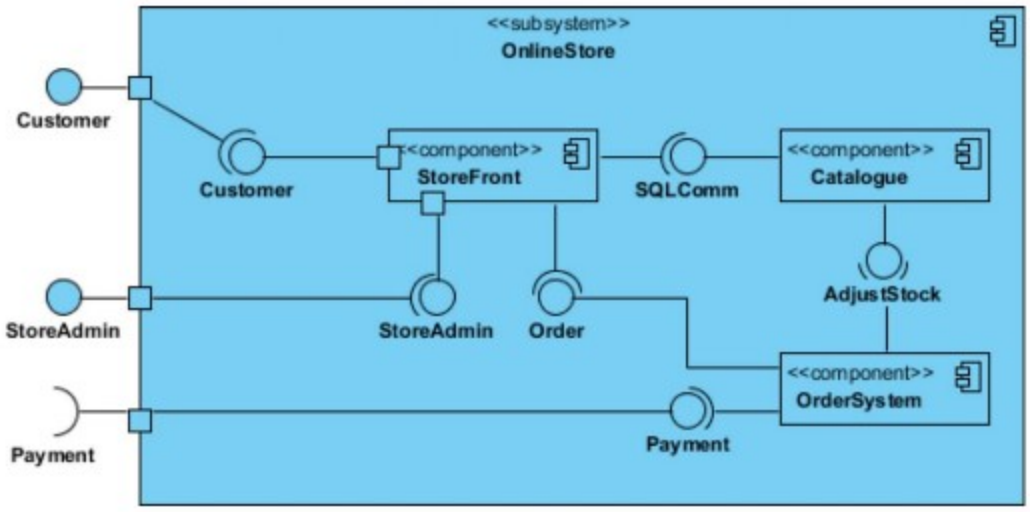


Diagramma dei componenti

Questo diagramma modella le **dipendenze** (le relazioni) esistenti tra i componenti di un sistema.

I **componenti** sono aggregazioni di alto livello di pezzi di software più piccoli e forniscono un approccio modulare (black box) per la costruzione di software.

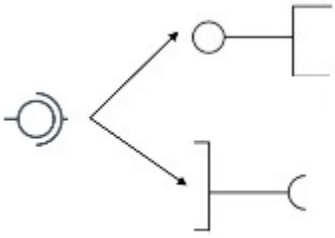
Il diagramma dei componenti consente al designer di sistema di **verificare** che la **funzionalità** richiesta dal sistema sia **implementata** dai componenti
Ecco un esempio di diagramma dei componenti:



I simboli del diagramma dei componenti sono i seguenti e hanno i seguenti significati:



- E' un'entità autonoma all'interno di un sistema o sottosistema.
- Ha una o più interfacce (fornite o richieste), e gli elementi al suo interno sono nascosti ed inaccessibili, se non tramite le sue interfacce.



Interfaccia fornita:
Rappresenta l'interfaccia messa a disposizione da un componente (che produce informazioni)

Interfaccia richiesta:
Rappresenta l'interfaccia richiesta da un componente (che consuma informazioni)

Il termine "**interfaccia**" è assimilabile al concetto di interfaccia di OOP, si tratta di uno scambio di informazioni.

Le fasi della progettazione sono le seguenti:

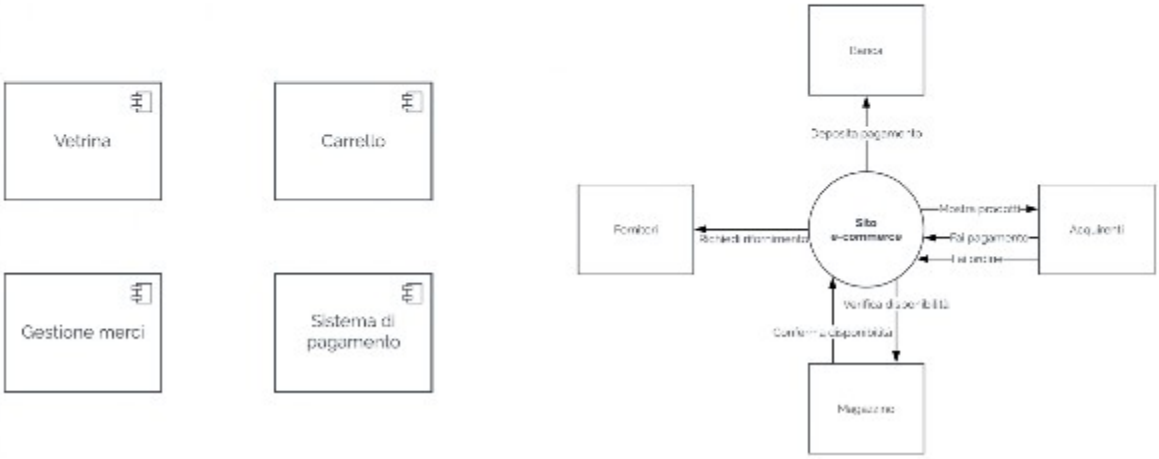
1. **Identificare le componenti facenti parte del sistema**
   I componenti più possibile autonomi all'interno del nostro sistema
   La domanda da farsi è "Quali sono le entità autonome di cui abbiamo bisogno?"
2. **Identificare le interfacce appropriate per ciascun componente**
   La domanda da farsi è "Quali informazioni deve comunicare il componente?" e "Quali informazioni ha bisogno di ricevere da un altro componente?"

3. **Collegare le interfacce dei componenti che interagiscono tra di loro**
4. **Caratterizzare il dato trasmesso attraverso le interfacce, etichettandolo**

Ecco un esempio:

Rappresentare con un diagramma dei componenti le funzionalità richieste da un sito di e-commerce con questo digramma di contesto.

1. Identificare le componenti facenti parte del sistema

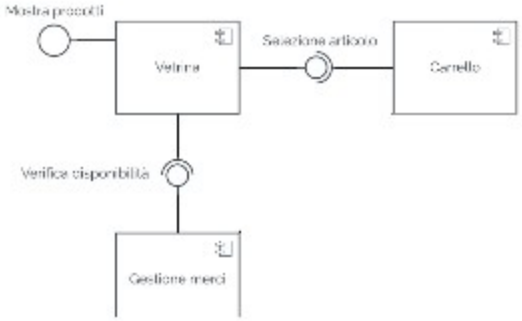


2. Identificare le interfacce appropriate per ciascun componente

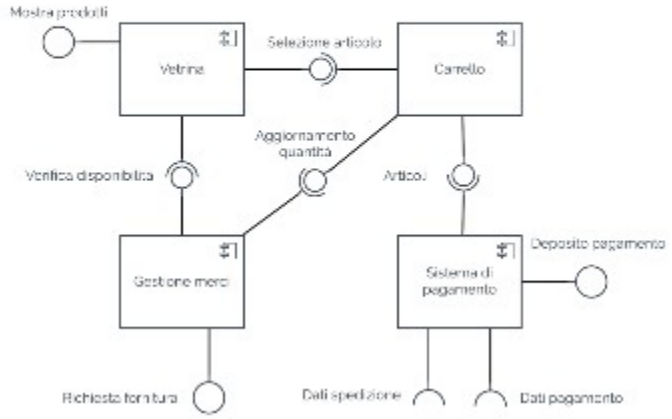


3. Collegare le interfacce dei componenti che interagiscono tra di loro

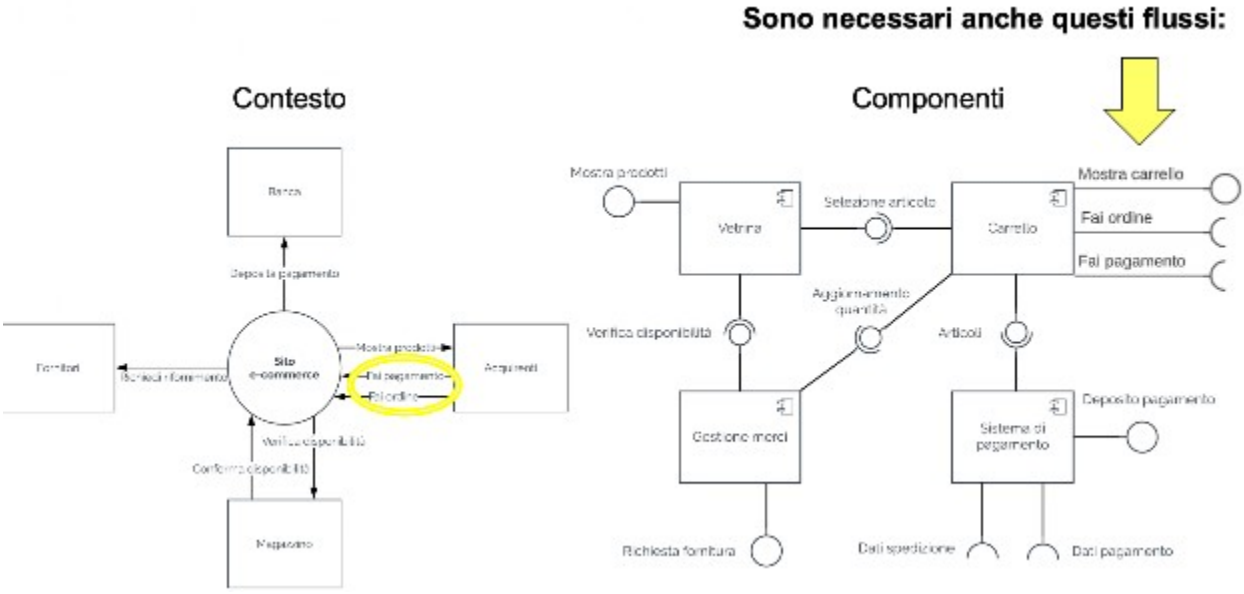
4. Caratterizzare il dato trasmesso attraverso le interfacce, etichettandolo



Risultato finale:



Bisogna adesso confrontarlo col diagramma di contesto e notiamo che ci mancano alcune informazioni:



Progettazione

La **progettazione** è la fase in cui si decidono le modalità in cui fare il **passaggio da** il “che **cosa**” deve essere realizzato nel sistema software al “**come**” la realizzazione deve aver luogo. Questa fase prende in input il **documento di specifica** (analisi dei requisiti) e produce un **documento di progetto** che guida la fase di codifica.

La fase di progetto può venir suddivisa in due sotto-fasi:

- La fase di **progettazione architetturale** (o preliminare)
  In questa fase il sistema complessivo viene suddiviso in più sottosistemi

- La fase di **progetto dettagliato**
  In questa fase ogni sottosistema identificato viene progettato in dettaglio, scegliendo algoritmi e strutture dati specifiche

Quindi iniziamo con una visione di insieme (progettazione architetturale) e poi scendiamo nel dettaglio (progetto dettagliato).

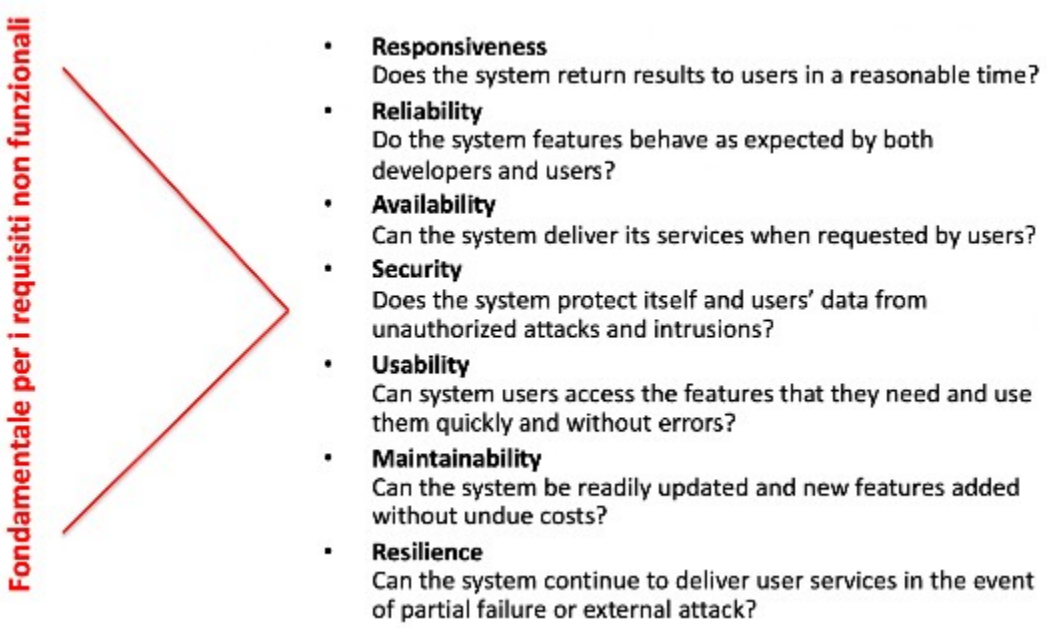
Progettazione architetturale

L'architettura di un sistema descrive la sua forma e struttura: quali sono i suoi componenti e il modo in cui interagiscono
L'architettura del software di un programma o di un sistema di calcolo è la struttura o le strutture del sistema, che comprendono i componenti software, le loro proprietà visibili e le relazioni fra di essi

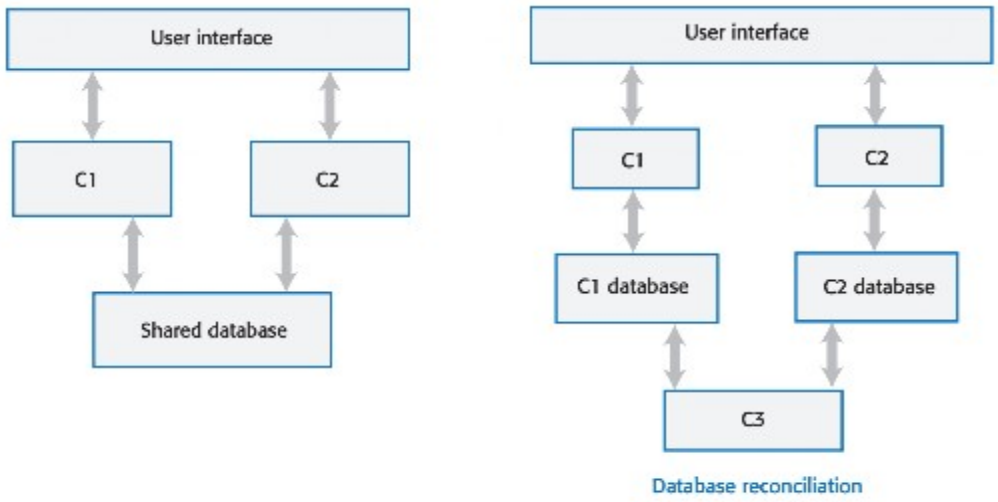
L'**architettura** è un'**astrazione** che permette di:

1. valutare le **alternative** in una fase in cui ogni cambiamento è ancora relativamente semplice (per esempio analizzando l'efficacia del design nel rispondere ai requisiti)
2. definire l'**organizzazione** del software, stabilendo ruoli e relazioni (tramite componenti e sottocomponenti)
3. avere un **modello di riferimento** per il **progetto**
4. avere un modello di riferimento per gli **stakeholders** e consentire la comunicazione:
    - evidenzia le **decisioni progettuali** e permette di motivarle rispetto ai requisiti
    - tutte le parti in gioco possono esprimere la propria opinione (**design partecipativo**)
    - si mostra il legame concettuale tra i requisiti e l'implementazione
    - si ottiene una **riduzione dei rischi** del progetto

L'architettura è fondamentale per i requisiti non funzionali:



Un esempio di scelta della progettazione architetturale è scegliere se avere un database centralizzato oppure un database distribuito (ognuno con i suoi vantaggi e svantaggi)



Performance vs Maintanability

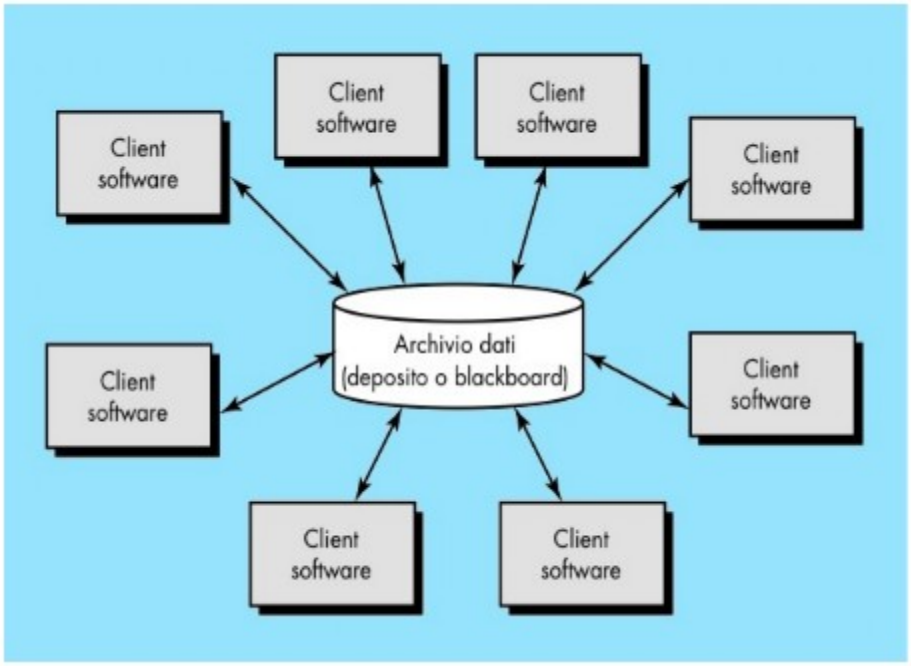
Il database di sinistra ad esempio è meno performante ma non ha bisogno di troppa manutenzione, il database di destra invece ha più performance ma ha bisogno di più manutenzione

Lo scopo di questa fase è di trovare i giusti compromessi (tradeoff):

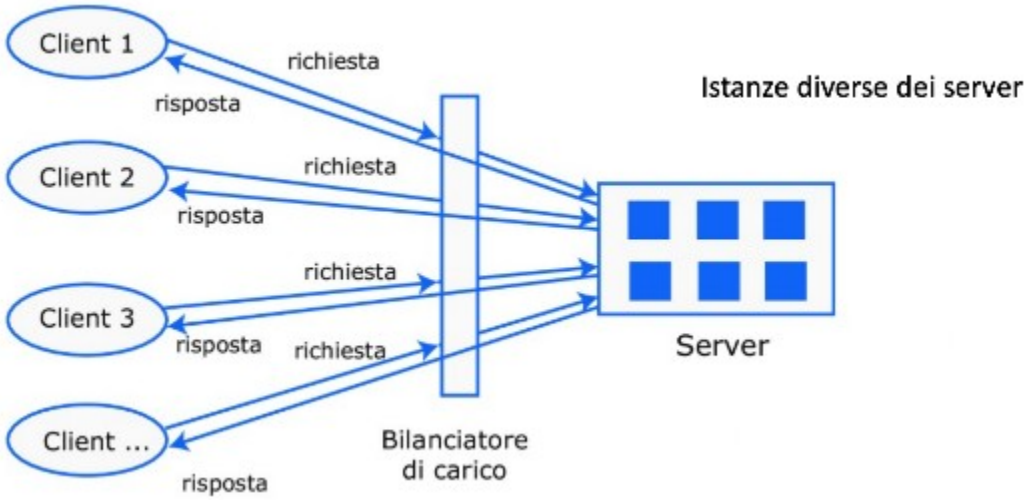
- Performance vs manutenzione
- Sicurezza vs usabilità
- Disponibilità vs time-to-market

Esistono vari tipi di architetture “standard”:

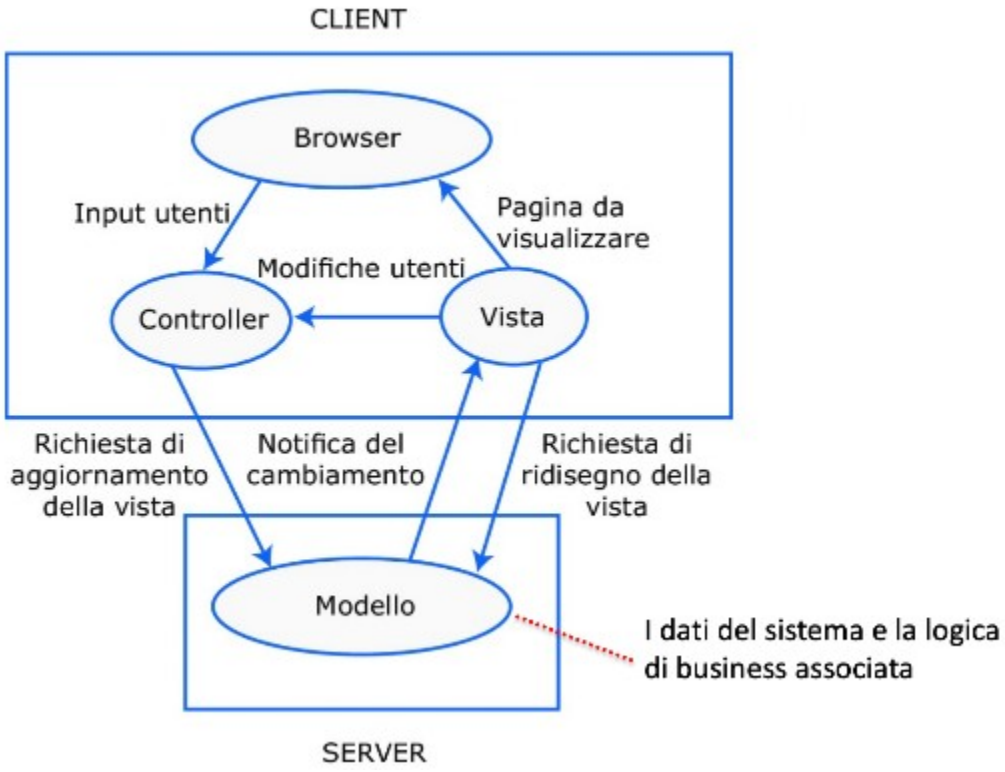
- Architettura basata sui dati
  In questa architettura vengono scambiati solo dati (non dati dell’applicazione ma dati e basta), il software è tutto sul client



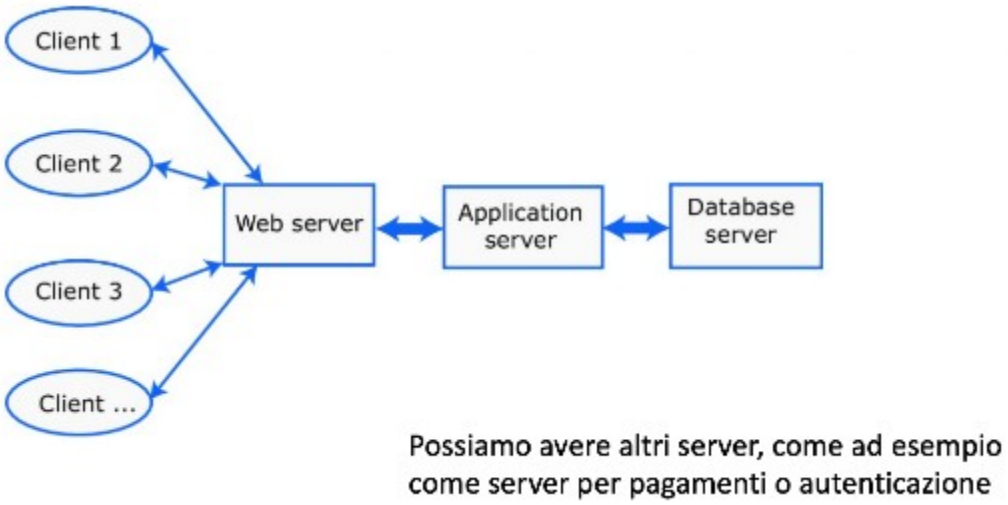
- Architettura client-server
  In questa architettura il client ha implementato solo l’interfaccia e le funzionalità sono distribuite dal server (sul quale opera la parte principale del software)



- Architettura Model-View-Controller
  Questa architettura viene usata per le app dei cellulari

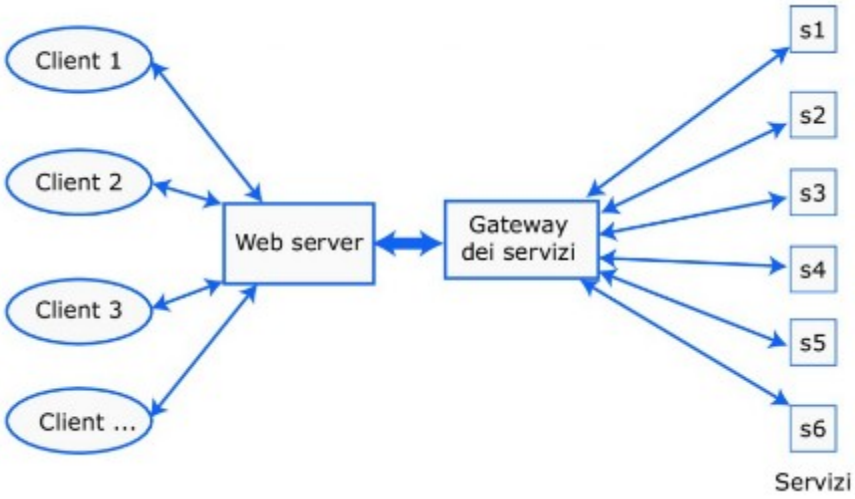


- Architettura multi-tier
  Ci sono più server comunicanti, ognuno con funzionalità diverse (tipicamente di 3 tipi)



- Architettura Service-Oriented
  Invece di avere tanti server che comunicano tra di loro, abbiamo un unico server che

comunica con tutti gli altri



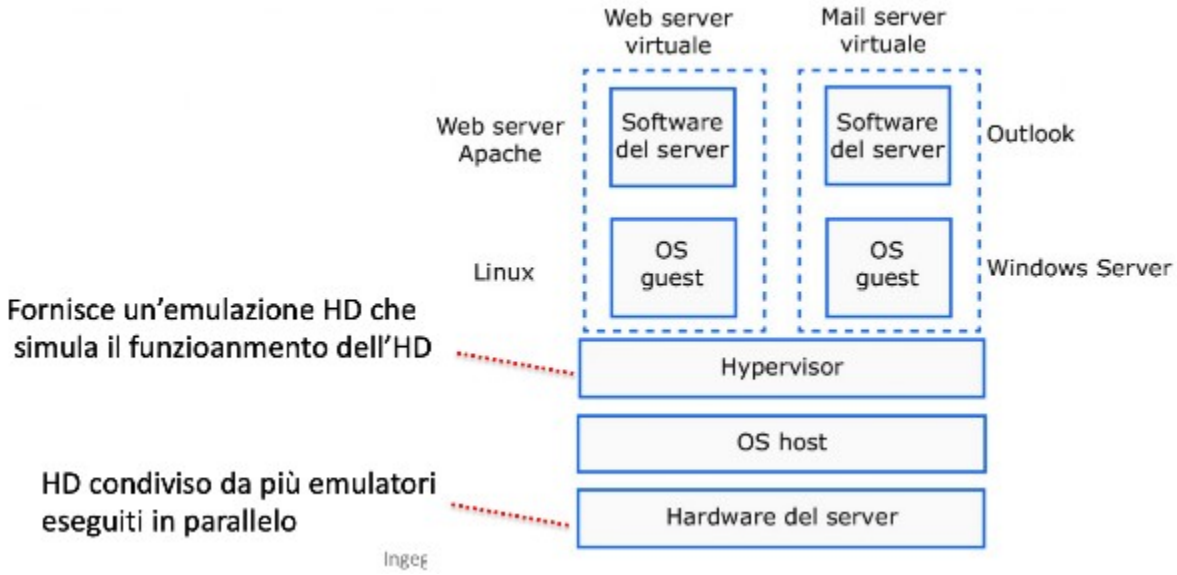
Le tecnologie che scegliamo di usare (database, server etc) possono influire sull'architettura complessiva del sistema

Tecnologia	Decisione progettuale
Database	Meglio usare un database relazionale SQL o uno di tipo NoSQL?
Piattaforma	Il prodotto dovrebbe essere rilasciato su app mobile e/o su una piattaforma web?
Server	Meglio usare server dedicati in sede o progettare il sistema per l'esecuzione su un cloud pubblico? Nel caso del cloud, meglio usare Amazon, Google, Microsoft o altre opzioni?
Open source	Esistono componenti open-source che possono essere utilizzati?
Strumenti di sviluppo	Gli strumenti di sviluppo utilizzati limitano la libertà delle scelte architetturali?

Le architetture che utilizzano **software sul cloud** hanno i vantaggi di scalabilità, elasticità e resilienza



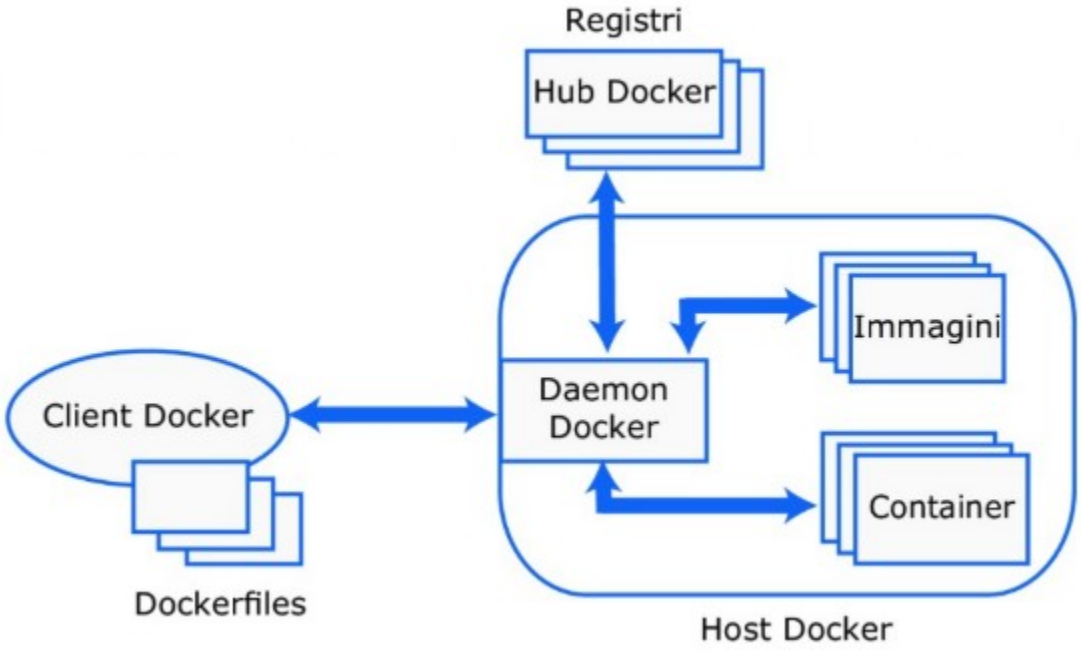
I server possono venire implementati tramite **macchine virtuali**, quindi esiste un solo server hardware (molto potente) su cui faccio girare tanti server SW (anche su diversi sistemi operativi)



Ovviamente per creare una macchina virtuale bisogna caricare e avviare un sistema operativo grande e complesso (che riesca a gestire tutti gli altri) e ad esempio se bisogna reagire istantaneamente alle variazioni delle richieste non si può avviare e spegnere la VM perché richiede diversi minuti.

Per questo motivo vengono usati i **container**, i quali rendono più veloce il processo di deployment dei server virtuali sul cloud.
I container sono una tecnologia di virtualizzazione dei sistemi operativi che permettono a più server indipendenti di condividere lo stesso sistema operativo (possono venir avviati e spenti in pochi secondi).

Un metodo standard per la gestione dei container in maniera veloce e di facile impiego è il **docker**

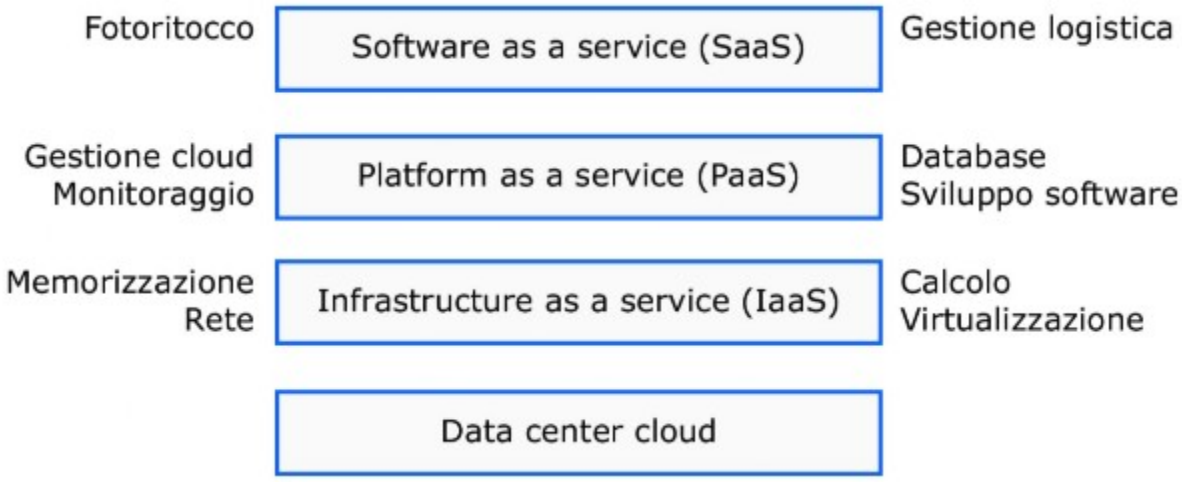


Elemento	Funzione
Daemon Docker	È un processo eseguito su un server host ed è utilizzato per impostare, avviare, fermare e monitorare i container, nonché per costruire e gestire immagini locali.
Client Docker	Questo software è utilizzato da sviluppatori e system manager per definire e controllare i container.
Dockerfiles	Definiscono le applicazioni eseguibili (immagini) come una serie di comandi di impostazione che specificano il software da includere in un container. Ogni container deve essere definito da un Dockerfile associato.
Immagine	Un Dockerfile viene interpretato per creare un'immagine Docker, ovvero un insieme di directory con il software e i dati specificati installati nei posti giusti. Le immagini sono configurate in modo da essere applicazioni Docker eseguibili.
Hub Docker	È un registro delle immagini create, che possono essere usate per configurare container o come punti di partenza per definire nuove immagini.
Container	I container eseguono le immagini: un'immagine viene caricata in un container e l'applicazione definita dall'immagine inizia l'esecuzione. I container possono essere spostati da un server all'altro senza modifiche, e possono essere replicati su più server. È possibile apportare modifiche a un container Docker (modificare file, ad esempio), ma le modifiche devono essere confermate per creare una nuova immagine e riavviare il container.

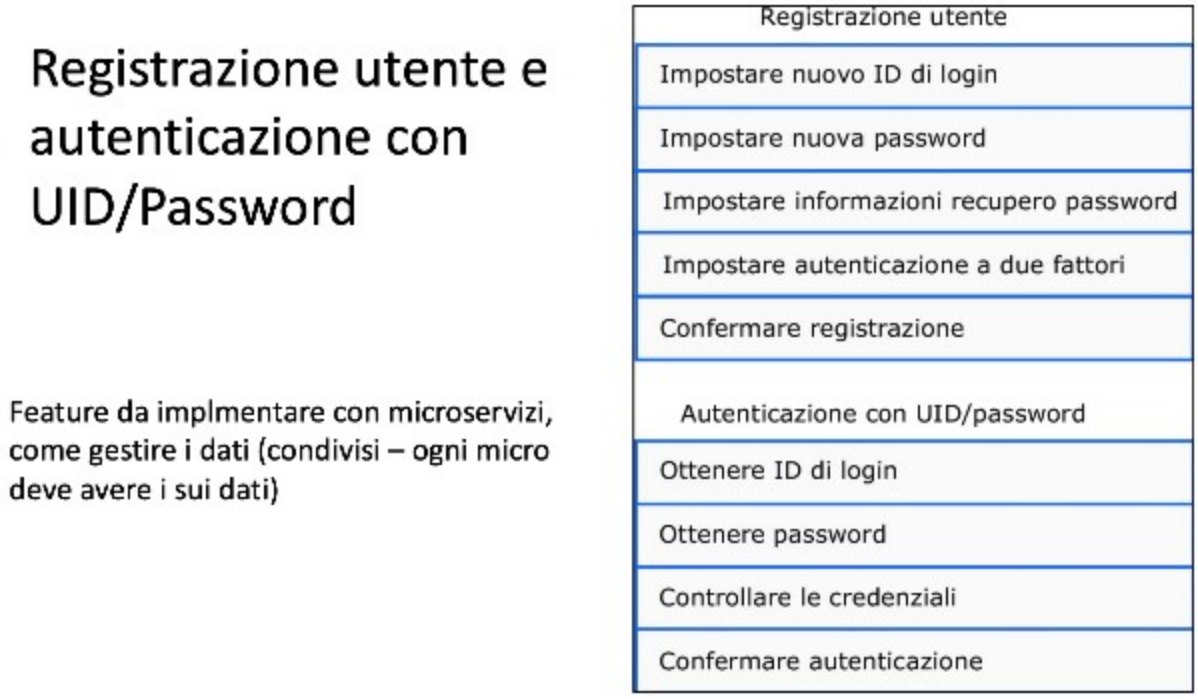
Per quanto riguarda le architetture a servizi abbiamo 3 diversi tipi di funzionalità messe a disposizione:

- Software as a Service (Saas)

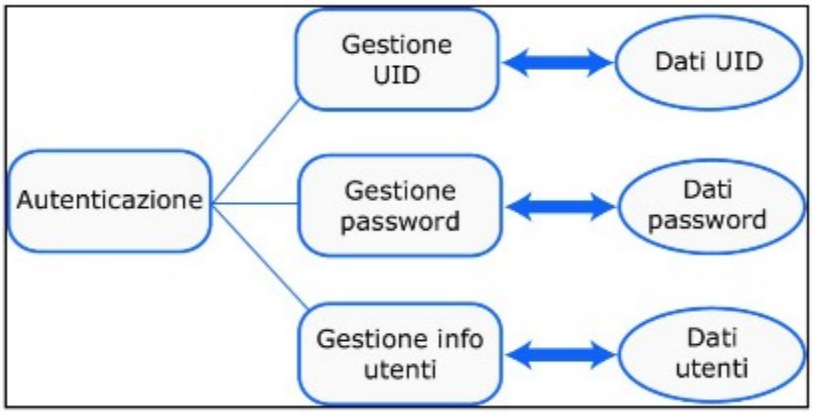
- Platform as a Service (Paas)
- Infrastructure as a Service (Iaas)



Lo sviluppo orientato ai servizi (web services) ha dei protocolli standard XML (come SOAP e WSDL), i messaggi XML sono lunghi e complessi da analizzare
I **microservizi** sono delle singole funzionalità operative e sono completamente indipendenti (hanno una loro interfaccia e un loro database). Questi microservizi sono semplici da sostituire senza cambiare il sistema.
Ecco un esempio di microservizi:



Si può identificare un microservizio in ogni dato logico da gestire:



Progettazione dettagliata

La progettazione architetturale è una rappresentazione del sistema nel **contesto** (deve definire le entità esterne con cui il software interagirà).
Una volta che la progettazione architetturale è stata fatta (ovvero il contesto è stato modellato) si passa alla **specifica del sistema** in cui vengono definiti e raffinati i componenti software che implementano l'architettura.
Viene fatto quindi un **processo iterativo** fino alla definizione completa dell'architettura.

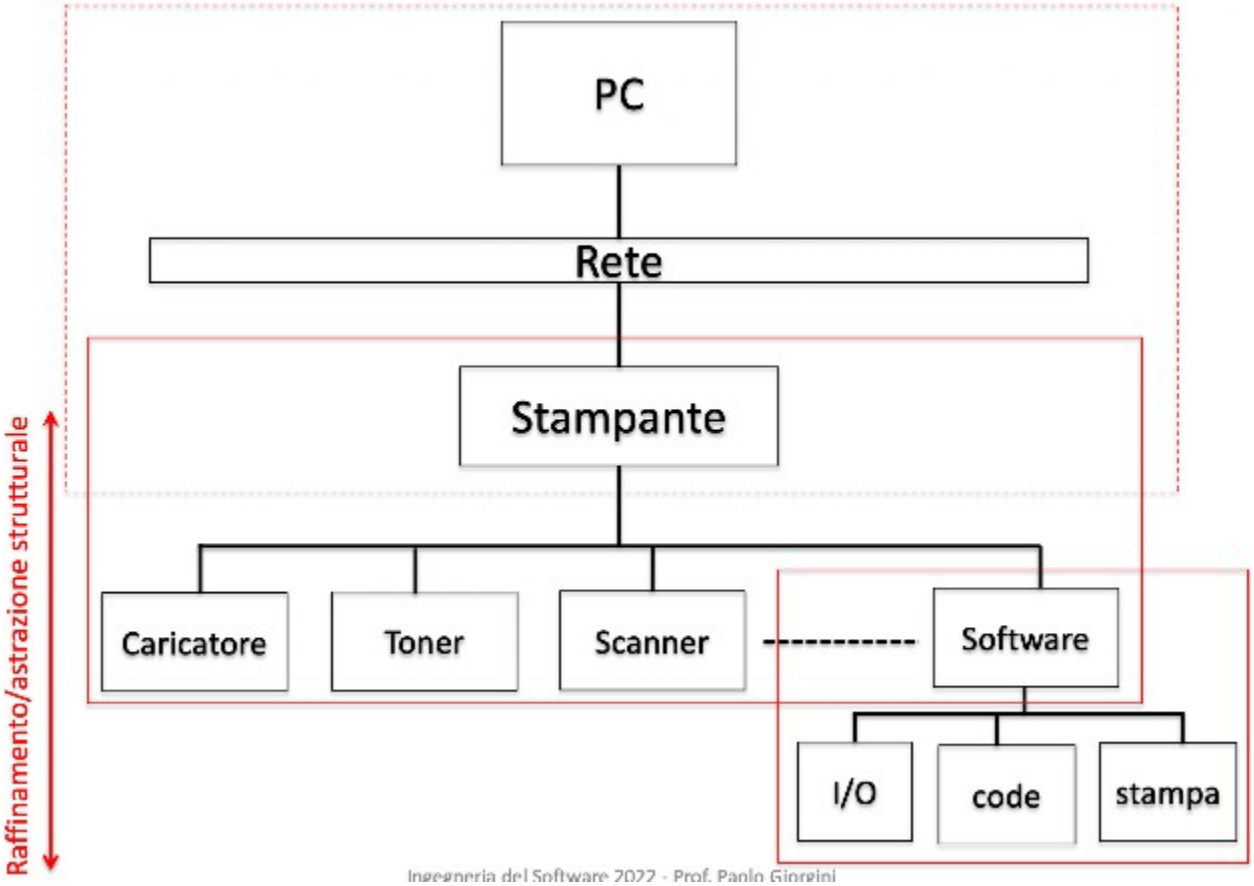
In questa fase viene fatta una **scomposizione del sistema**, identificando le seguenti categorie:

- Un **servizio** è un'entità coesa di funzionalità
  Ad esempio un servizio di posta elettronica (creare, memorizzare, leggere e inviare messaggi)
- Un **componente** è un'entità software che offre uno o più servizi
  Quando usata da altre componenti, viene utilizzata tramite API
  I componenti possono a loro volta impiegare altri componenti per implementare i loro servizi
- Un **modulo** è un insieme di componenti
  I componenti di un modulo devono avere qualcosa in comune, ad esempio fornire un insieme di servizi correlati

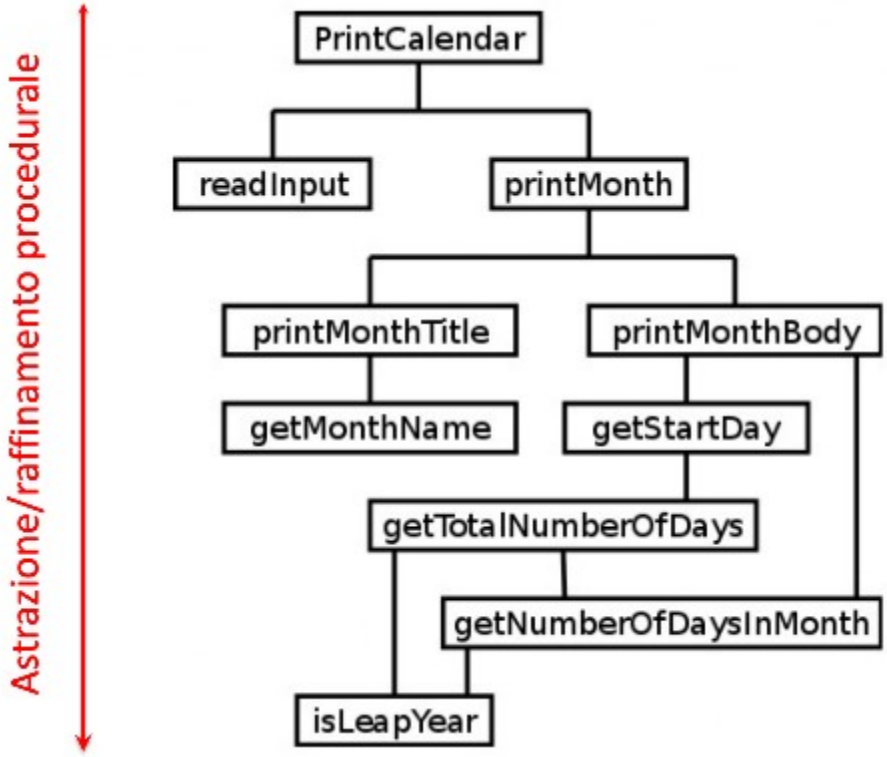
L'**astrazione** consente ad un progettista di specificare le procedure ed i dati sopprimendo i dettagli di basso livello.
Il **raffinamento** aiuta il progettista a rivelare i dettagli di basso livello a mano a mano che progredisce il design. È sbagliato passare direttamente al dettaglio saltando i passi di raffinamento, perché porta a tanti errori e omissioni

Ci sono diversi tipi di **astrazione**:

- Astrazione **strutturale**
  Come i vari componenti di un sistema sono aggregati



- Astrazione **procedurale**
  La sequenza di istruzioni di una funzione, specifica e limitata



- Astrazione **dei dati**
  Collezione dei dati che descrivono un progetto

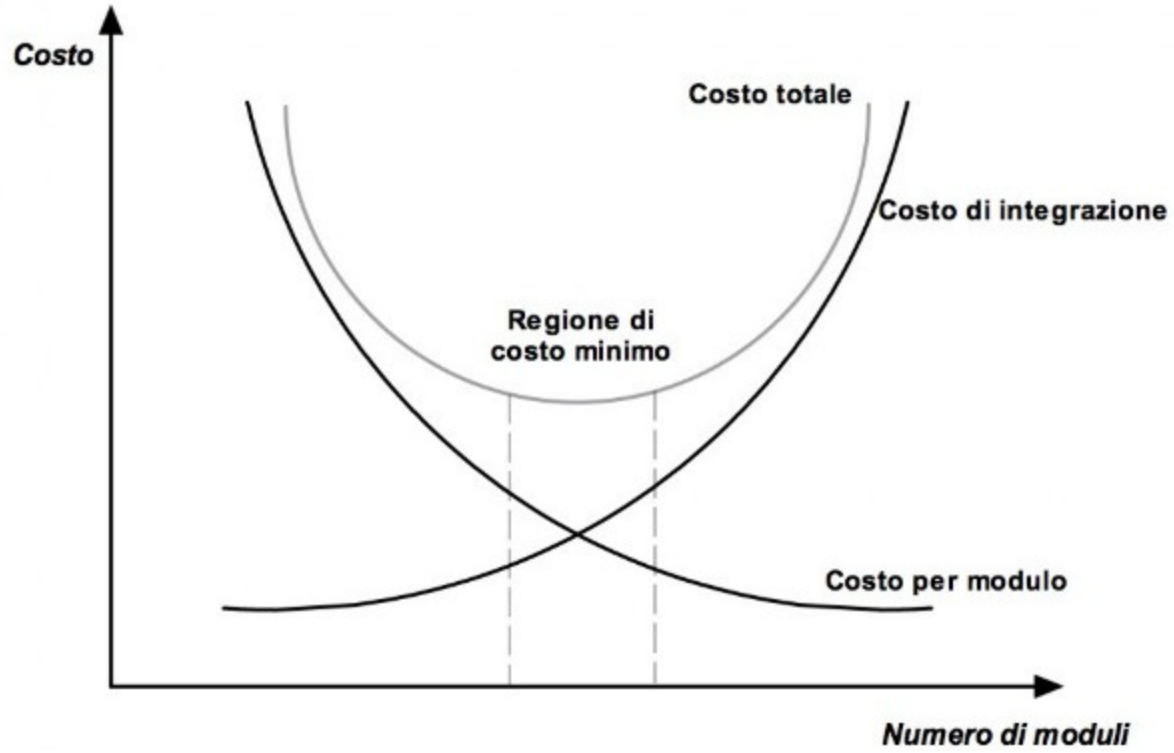


I prodotti software fatti da un unico blocco di codice sono difficili da mantenere, da correggere, da capire e da riusare.
Per questo motivo si usa la **modularità**, ovvero suddividere il prodotto software in segmenti più piccoli detti **moduli** (componenti) che poi vengono integrati per soddisfare i requisiti del problema.
Un modulo è un elemento software che:

- Contiene istruzioni, logica e strutture dati
- Può essere compilato separatamente e memorizzato all'interno di una libreria software
- Può essere incluso in un programma

- Può essere usato invocando segmento di modulo identificato da un nome e da una lista di parametri
- Può usare altri moduli

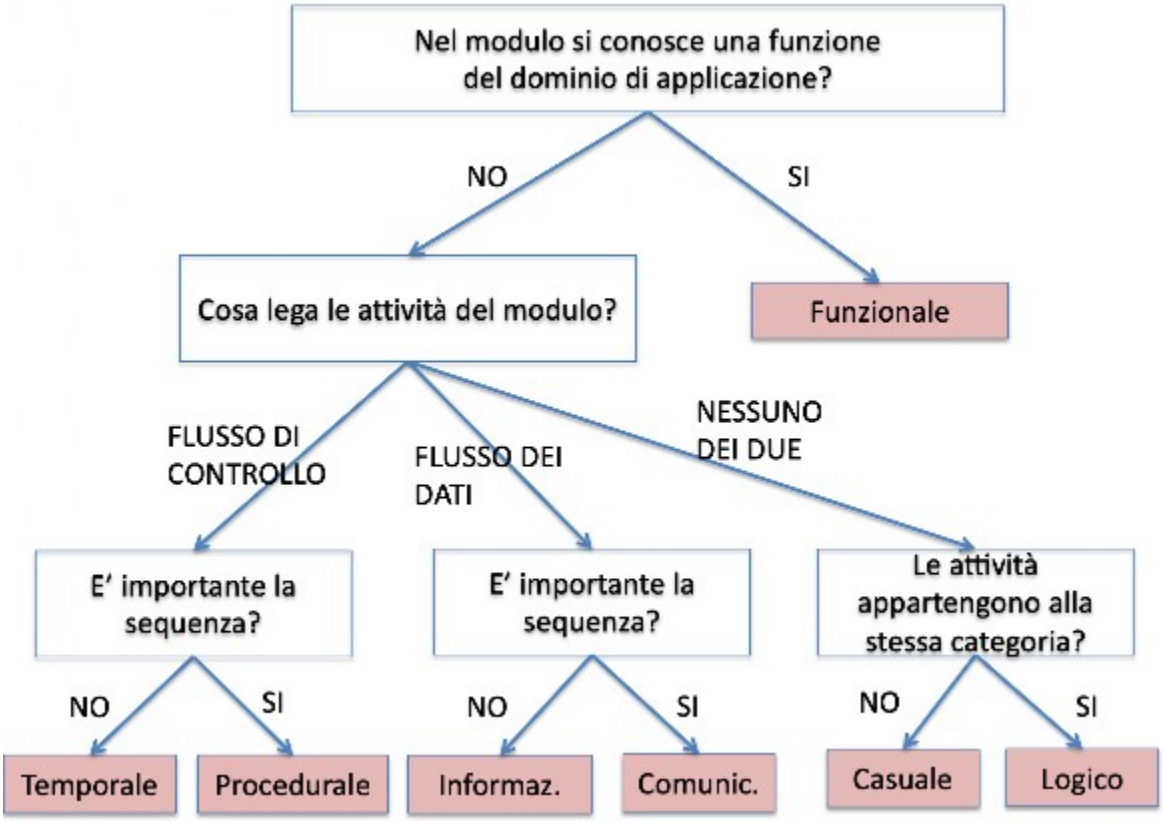
La suddivisione dei moduli in un sistema software produce come risultato l'identificazione di un **architettura dei moduli**, la quale descrive la **struttura dei moduli** e il modo in cui **interagiscono** (strategia "divide et impera").



Una buona divisione di un prodotto software in moduli è quella che permette di ottenere l'**indipendenza funzionale**, ovvero la **massima coesione interna** ai moduli, riducendo al **minimo il grado di accoppiamento** tra i moduli.
Una massima coesione e un minimo accoppiamento permettono di incrementare comprensibilità, manutenibilità, estensibilità e riusabilità.
Per eseguire una **funzione** sono necessarie varie azioni, queste azioni possono essere concentrate in un singolo modulo oppure essere sparse tra tanti; la **coesione** è la misura in cui il modulo esegue **internamente** le azioni per una data funzione.
La coesione dunque misura il grado di interazione interna al modulo tra le altre azioni di una funzione, esistono 7 livelli di coesione:

1. **Casuale** (nessuna relazione tra gli elementi del modulo)
2. **Logica** (elementi correlati, di cui uno viene selezionato dal modulo chiamante)
3. **Temporale** (relazione di ordine temporale tra gli elementi)
4. **Procedurale** (elementi correlati in base ad una sequenza predefinita di passi da eseguire)
5. **Comunicazionale** (elementi correlati in base ad una sequenza predefinita di passi che vengono eseguiti sulla stessa struttura dati)
6. **Informazionale** (ogni elemento ha una porzione di codice indipendente e un proprio punto di ingresso ed uscita; tutti gli elementi agiscono sulla stessa struttura dati)

7. **Funzionale** (tutti gli elementi sono correlati dal fatto di svolgere una singola funzione)

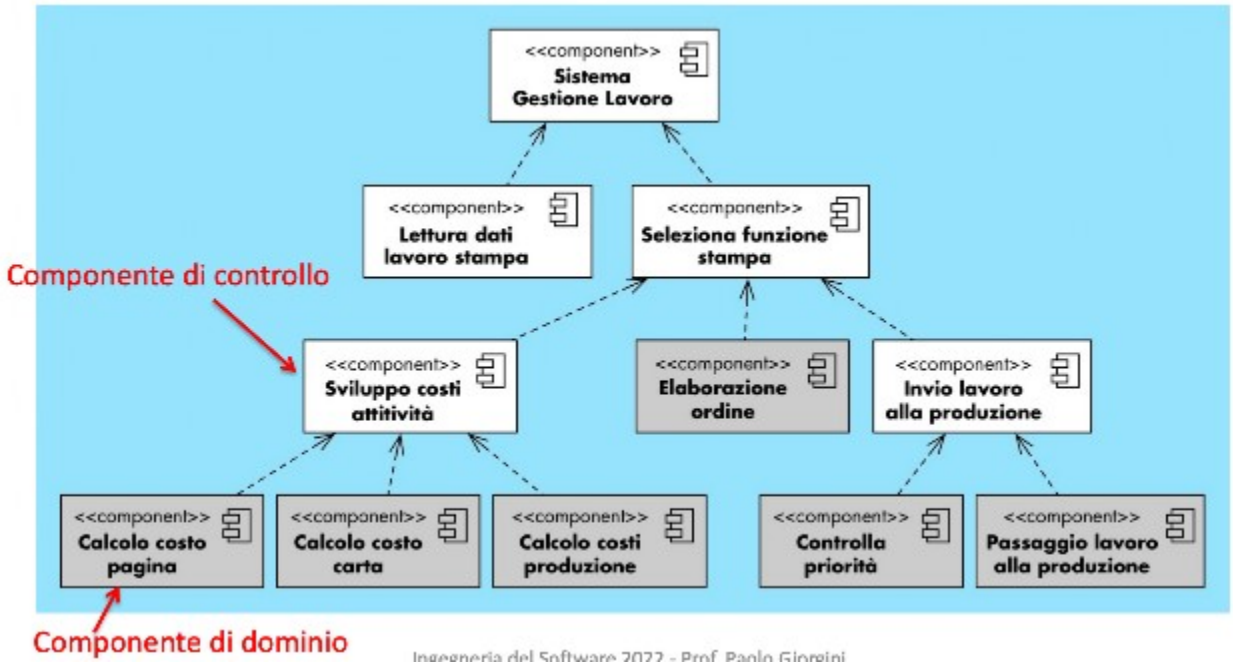


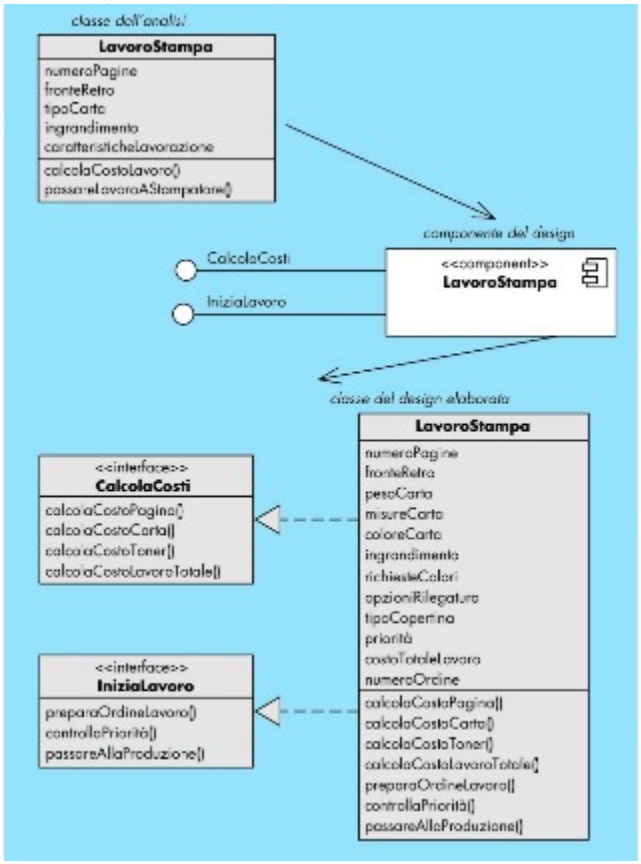
L'**accoppiamento** esprime la forza di inter-connessione tra moduli (maggiori connessioni implicano maggiore dipendenza).
L'obiettivo è quello di minimizzare l'accoppiamento, semplificando il tipo di connessione (solo tramite interfaccia) e semplificando la complessità dell'interfaccia (numero e tipo di parametri).
Il **coupling** misura il grado di accoppiamento tra moduli:

1. **Content** (un modulo fa diretto riferimento al contenuto di un altro modulo)
2. **Common** (due moduli che accedono alla stessa struttura dati)
3. **Control** (un modulo controlla esplicitamente l'esecuzione di un altro modulo)
4. **Data** (due moduli che si passano come argomento una struttura dati, della quale si usano solo alcuni elementi)
5. **Stamp** (due moduli che si passano argomenti omogenei, ovvero argomenti semplici o strutture dati delle quali si usano tutti gli elementi)

Gli obiettivi della progettazione modulare sono quelli di **massimizzare la coesione interna** (per migliorare la comprensibilità del modulo e la sua modificabilità) e di **minimizzare il grado di accoppiamento** (per migliorare la comprensibilità di ciascun modulo)

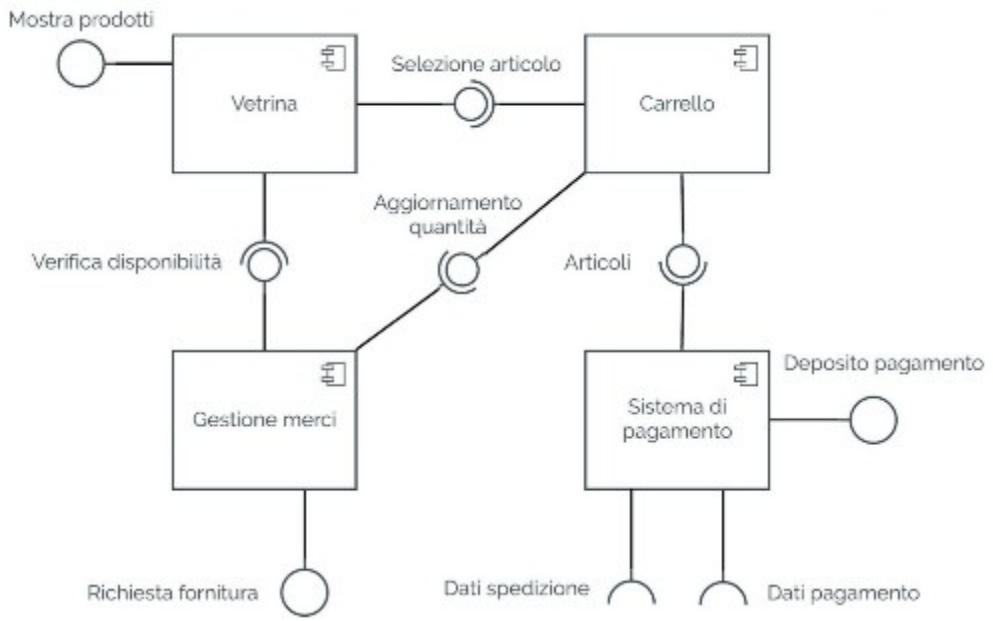
In UML esistono i component diagram, che rappresentano proprio queste proprietà:



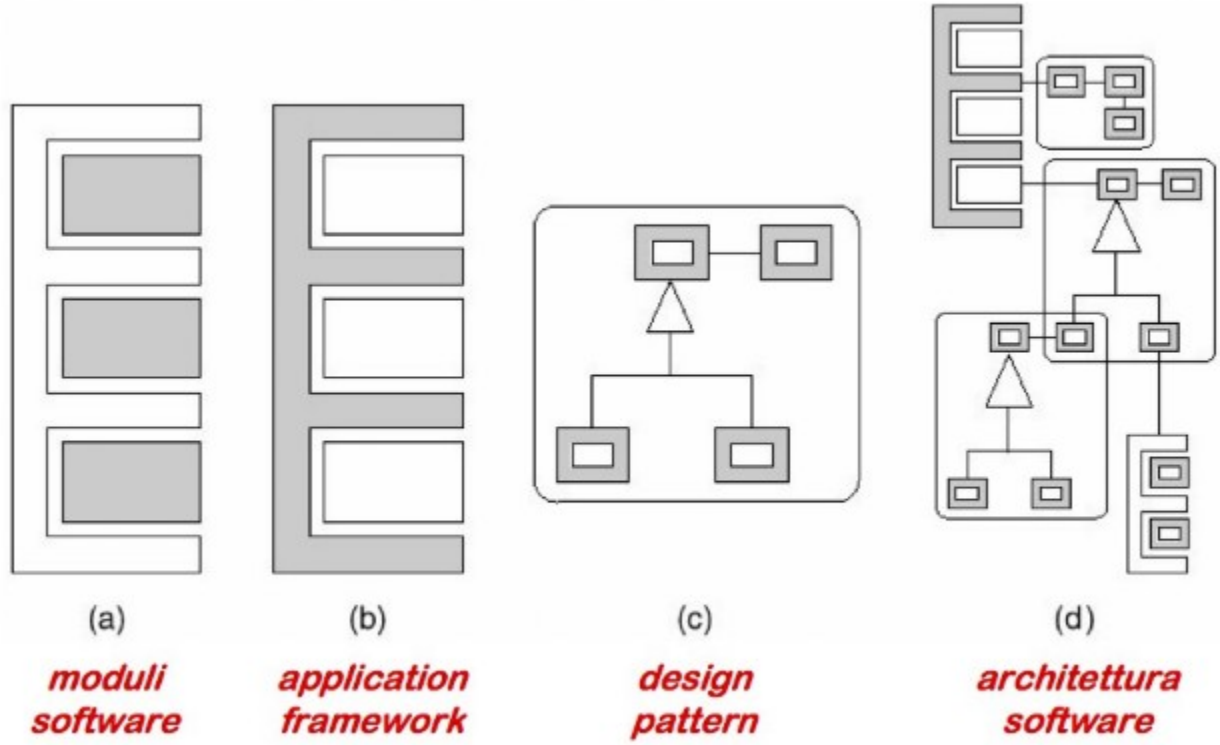


I dettagli di `LavoroStampa` sono elaborati

- Informazioni più dettagliate ed una descrizione estesa delle operazioni necessarie per implementare la classe
- Le interfacce implicano comunicazioni e collaborazioni con altri componenti
  - `calcolaCostoPagina()` deve collaborare con un componente `TabellaPrezzi`
  - `controllaPriorita()` deve collaborare con una componente `CodaLavori`

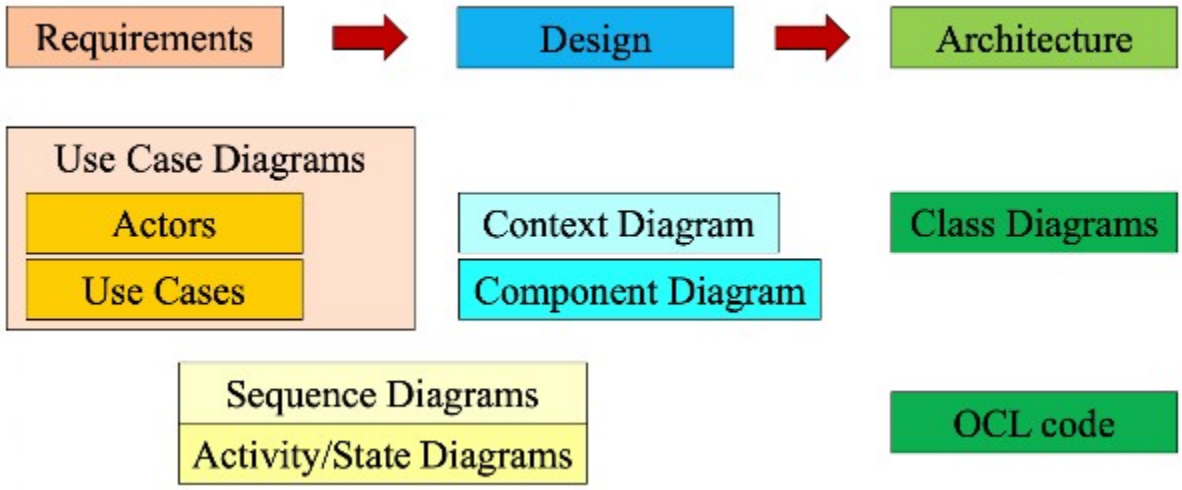


La **riusabilità** fa riferimento all'utilizzo di componenti sviluppati per un prodotto all'interno di un prodotto differente. Per **componente riusabile** si intende non solo un modulo o un frammento di software, ma anche un progetto intero.
I vantaggi della riusabilità sono una netta **diminuzione di costi e tempi** dovuti alla produzione del software e un **incremento dell'affidabilità** dovuto all'uso di componenti già convalidati e verificati.



Architettura

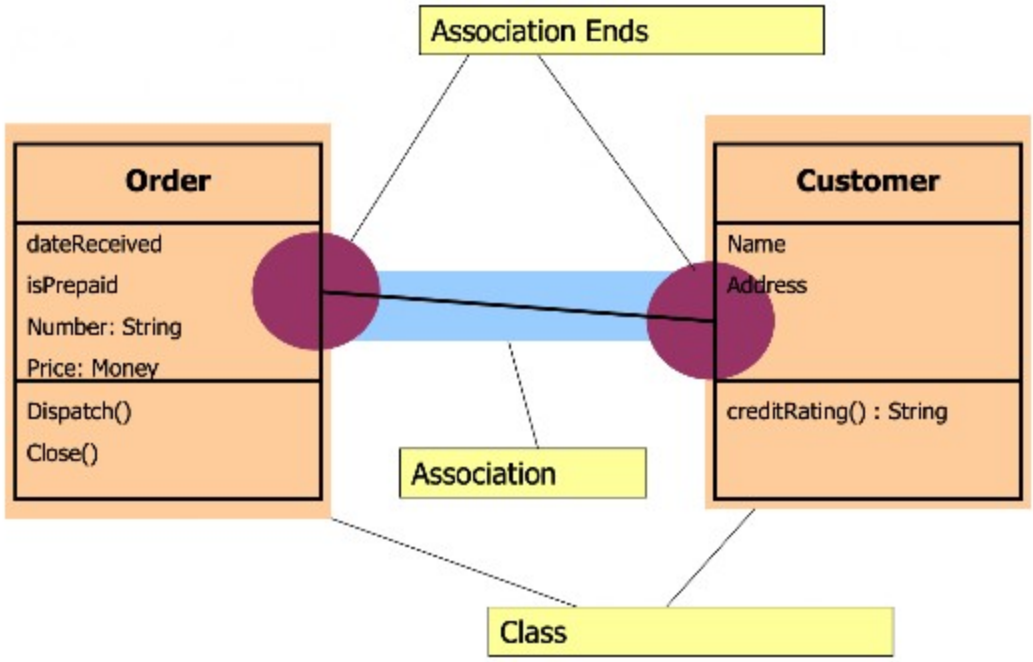
Con lo use case diagram non si definisce ancora come fare le cose, ma si definisce solo **cosa**. Poi con lo sviluppo di diagramma di contesto e componenti iniziamo già a entrare nell'ottica del **come**, senza però entrare nello specifico.
Il **come** viene effettivamente specificato nel **diagramma delle classi**



L'obiettivo della progettazione architetturale è quello di definire l'architettura (ovvero la struttura) del sistema, e viene prodotto come output il **documento di specifica architetturale**

Diagramma delle classi

Partiamo subito con un esempio per capire bene il diagramma delle classi:



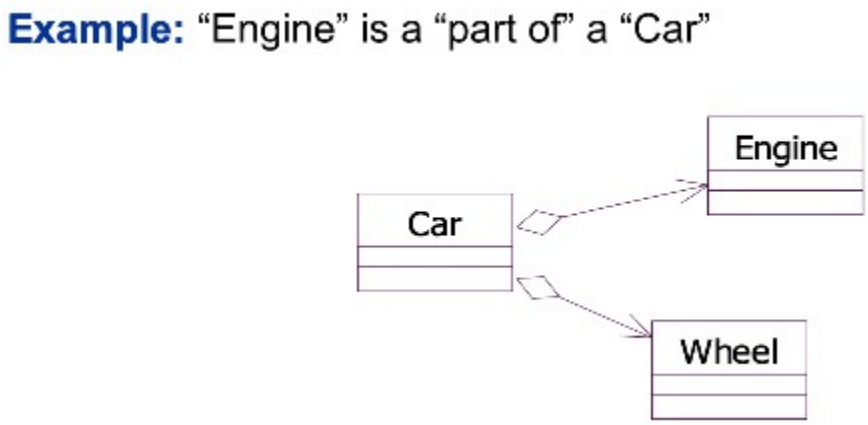
Notiamo che è composto dai seguenti elementi:

- **Classi**
  Le classi hanno un **nome**, degli **attributi** (delle caratteristiche identificative della classe) e delle **operazioni** (dei metodi, funzioni)
  Di un attributo dobbiamo sapere come minimo il nome, ma possiamo specificare anche il tipo, la visibilità e il valore iniziale.
- **Associazioni**
  Che collegano due classi e hanno un nome
- I **termini** dell'associazione
  Abbiamo la **navigabilità** (che serve per dare una chiave di lettura), il **ruolo** (che ruolo ha quella classe nell'associazione, il perchè è presente nell'associazione) e la **molteplicità** (o cardinalità).

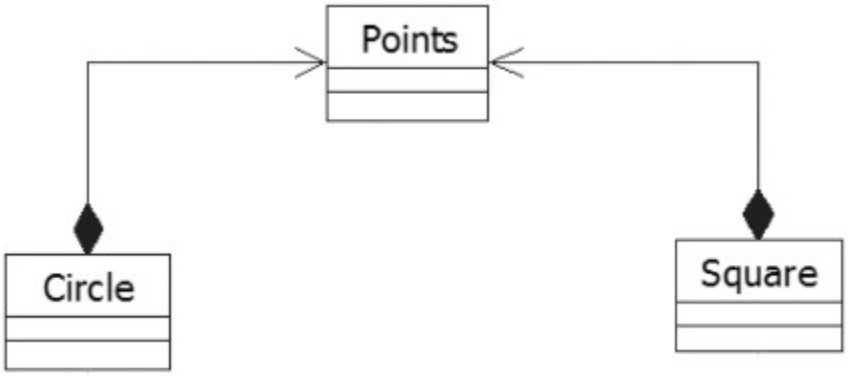
Generalmente non esiste un mapping diretto tra una componente del diagramma dei componenti e una classe nel diagramma delle classi, ma solitamente un componente "genera" più classi.

Abbiamo altri concetti avanzati che possono essere aggiunti in caso di necessità:

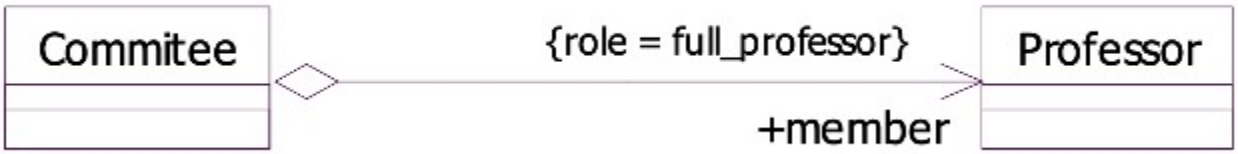
- **Aggregazione**
  Per rappresentare una relazione "è parte di"



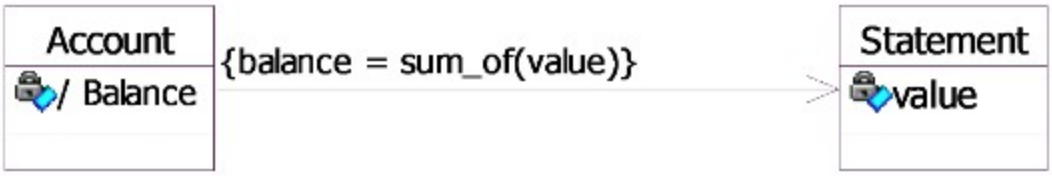
- **Composizione**
  È una aggregazione ma più forte, ovvero l'oggetto "contenuto" esiste e vive solo assieme alla classe "container"



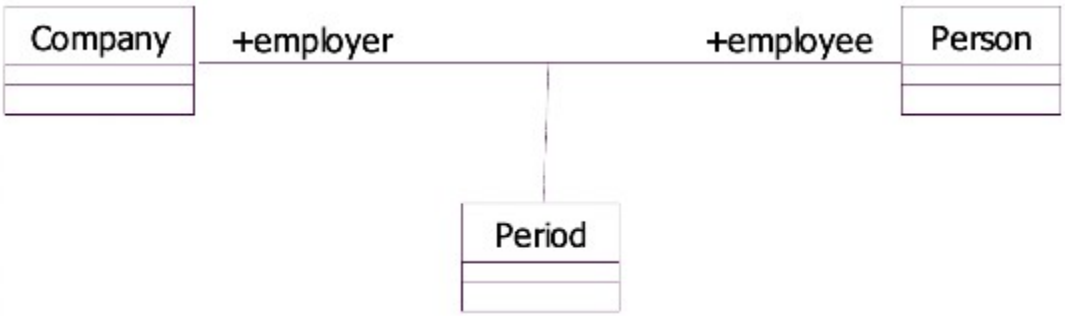
- **Vincolo**
  Per definire qualche condizione semantica che deve essere preservata mentre il sistema è in uno stato costante



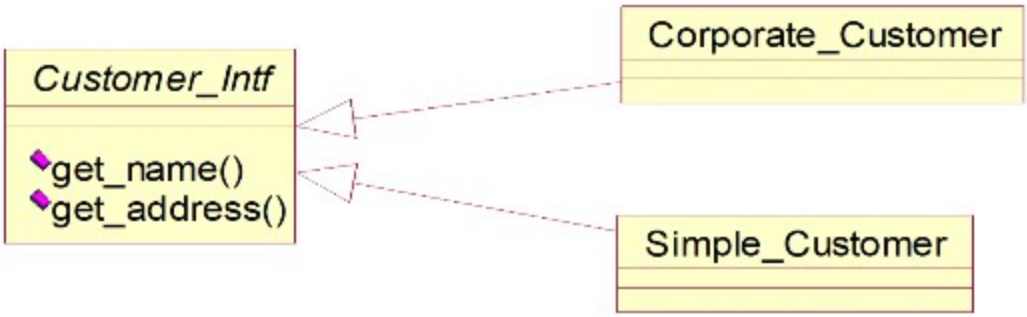
- **Attributi derivati**
  Per sottolineare le informazioni che possono essere derivate da elementi del modello



- **Classi di associazione**
  Permettono di arricchire e specificare meglio le associazioni tra le classi



- **Classi astratte**
  Provvedono una specifica per una interfaccia, è come l'ereditarietà



Nel diagramma delle classi dobbiamo comprendere tutti i dati del nostro progetto e tutte le operazioni che vengono eseguite (sottoforma di metodi)
Per **ridurre la complessità** di lettura del diagramma delle classi possiamo eseguire una divisione in vari livelli di **astrazione**:

- **Concettuale**
  Il diagramma che rappresenta i **concetti** nel dominio di studio

- **Specifica**
  Il diagramma che rappresenta le **interfacce** del sistema da sviluppare
- **Implementazione**
  Il diagramma che rappresenta l'**implementazione** del sistema da sviluppare

OCL code

Il codice **OCL** (Object Constraint Language) permette di arricchire il diagramma delle classi aggiungendo dei **vincoli**. È parte di UML.

Un **vincolo** è una restrizione su uno o più valori (di una parte) di un modello/sistema object-oriented.

Un'**invariante** è un vincolo che deve essere verificato/rispettato per un oggetto durante tutto il suo ciclo di vita.

Syntax:

```
context <classifier>

inv [<constraint name>]: <Boolean OCL expression>
```

Un esempio di invariante è il seguente:

```
context asta inv: fine-asta > inizio-asta
```

or

```
context asta inv: self.fine-asta > self.inizio-asta
```

Dove *self* viene usato per identificare l'oggetto dal quale il vincolo viene valutato.

Ci sono poi le **precondizioni** e le **postcondizioni**, e sono dei vincoli che specificano l'applicabilità e l'effetto di un'operazione senza specificare un algoritmo o un implementazione per esso.
Le precondizioni e le postcondizioni vengono "attaccate" alle operazioni in un diagramma delle classi.
Queste condizioni permettono di avere una specifica più completa del sistema.

Una **precondizione** è un vincolo che deve essere verificato/rispettato prima di poter eseguire una operazione

Syntax:

```
context <classifier>::<operation> (<parameters>)
pre [<constraint name>]:
<Boolean OCL expression>
```

Ecco alcuni esempi di precondizioni:

```
context conduttore::avvia-asta()
pre : asta.attiva = false

context conduttore::assegna-oggetto()
pre : asta.conteggio = 3

context ordinare_pietanze::modifica-ordine(…)
pre : ((oraattuale-self.data)<5 min) or
      (pietanza.preparata = false)
```

Una **postcondizione** è un vincolo che deve essere verificato/rispettato subito dopo l'esecuzione di una operazione.

Syntax:

```
context <classifier>::<operation> (<parameters>)
post [<constraint name>]:
<Boolean OCL expression>
```

Ecco alcuni esempi di postcondizioni:

```
context conduttore::assegna-oggetto()

post : asta.attiva = false


context conduttore::azzera-conteggio()

post : asta.conteggio = 0


context tavolo::set-riservato()

post : self.riservato = true
```

Da Diagramma della classi a API server

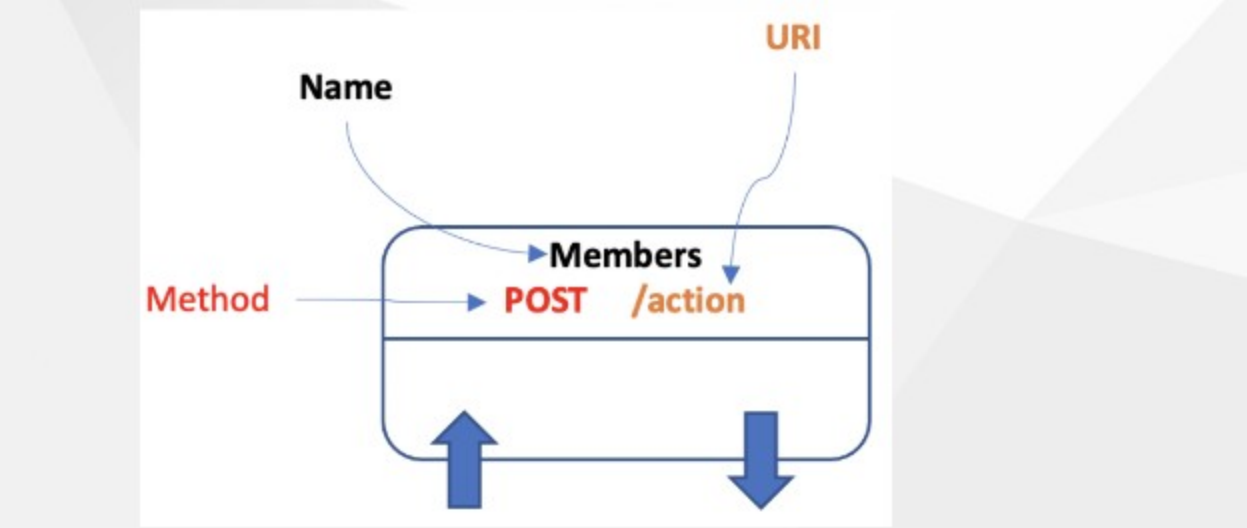
Un passaggio critico dell'ingegneria del software è quello che va dal diagramma delle classi al **server API**. Vedremo tutti gli step da fare partendo dal diagramma delle classi e arrivando alla specifica e alla implementazione delle API.
Un obiettivo è rendere le componenti più autonome possibili, così di conseguenza diventano anche più riutilizzabili

Per progettare una API possiamo disegnare un **class diagram** che rappresenta la **risorsa**, la **richiesta** e la **risposta**.
Una **risorsa API** è l'unità fondamentale di un API, è un oggetto con un URI, un metodo di richiesta HTTP e i parametri associati.

Ogni risorsa API rappresenta uno specifico servizio, disponibile nel path specificato dal suo URI.

La **risorsa** quindi è composta da:

- **URI**
  Ogni risorsa ha il proprio URI, gli utenti accedono all'URI per accedere alla risorsa API
- **Metodo**
  Specifica l'azione da compiere sulla risorsa
- **Nome**
  Il nome della risorsa



Ci sono tanti tipi di metodo, ognuno per uno scopo diverso:

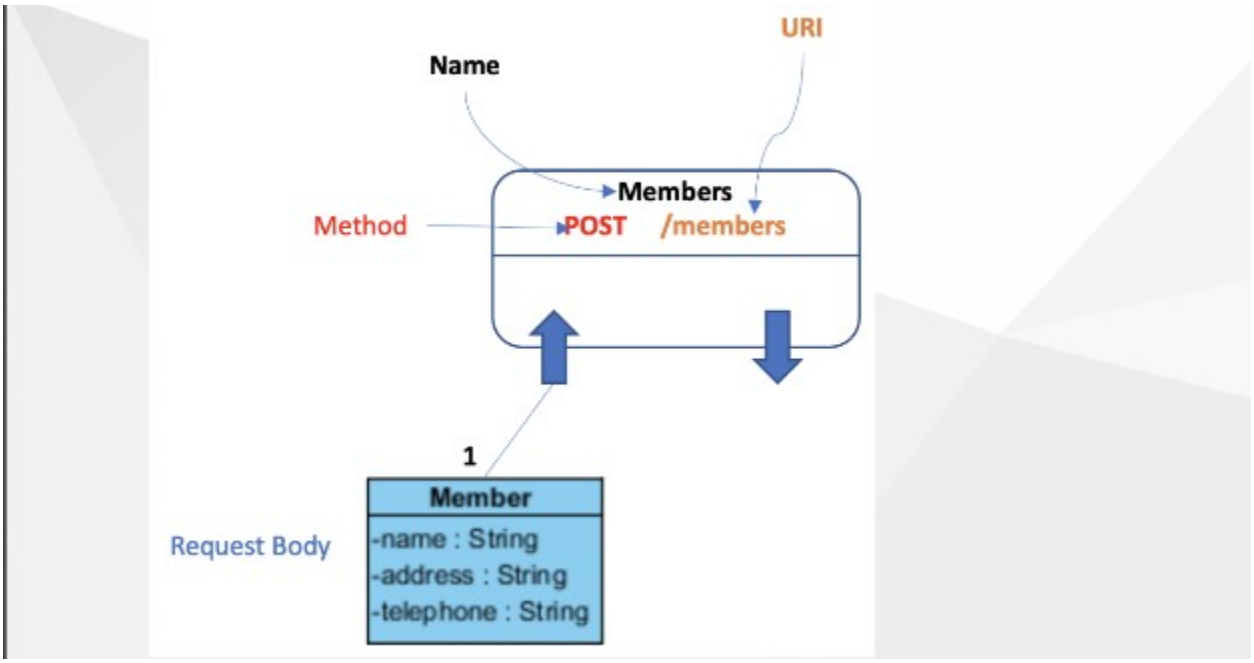
Methods (HTTP methods)

HTTP methods, or sometimes known as HTTP verbs, specify the action to act on a resource. The most commonly used HTTP methods are GET, PUT, POST and DELETE, which correspond to read, update, create and delete operations, respectively.

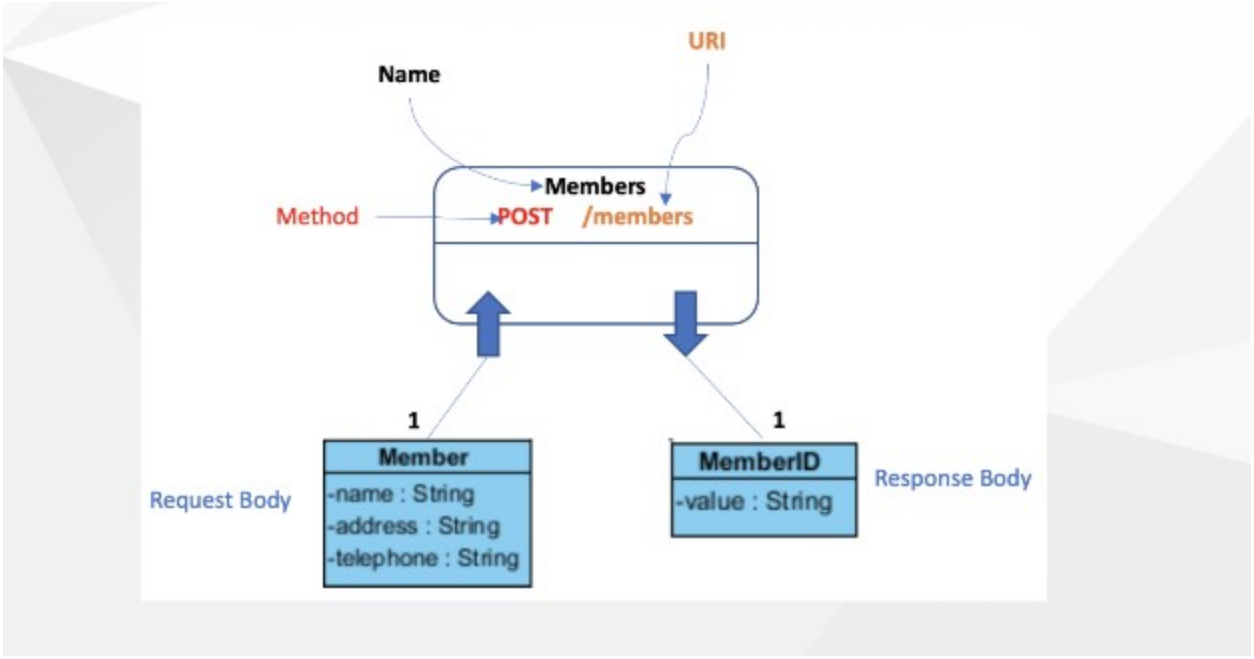
Method	Description
GET	A GET method (or GET request) is used to retrieve a representation of a resource. It should be used SOLELY for retrieving data and should not alter.
PUT	A PUT method (or PUT request) is used to update a resource. For instance, if you know that a blog post resides at http://www.example.com/blogs/123, you can update this specific post by using the PUT method to put a new resource representation of the post.
POST	A POST method (or POST request) is used to create a resource. For instance, when you want to add a new blog post but have no idea where to store it, you can use the POST method to post it to a URL and let the server decide the URL.
PATCH	A PATCH method (or PATCH request) is used to modify a resource. It contains the changes to the resource, instead of the complete resource.
DELETE	A DELETE method (or DELETE request) is used to delete a resource identified by a URI.

Description of different HTTP methods

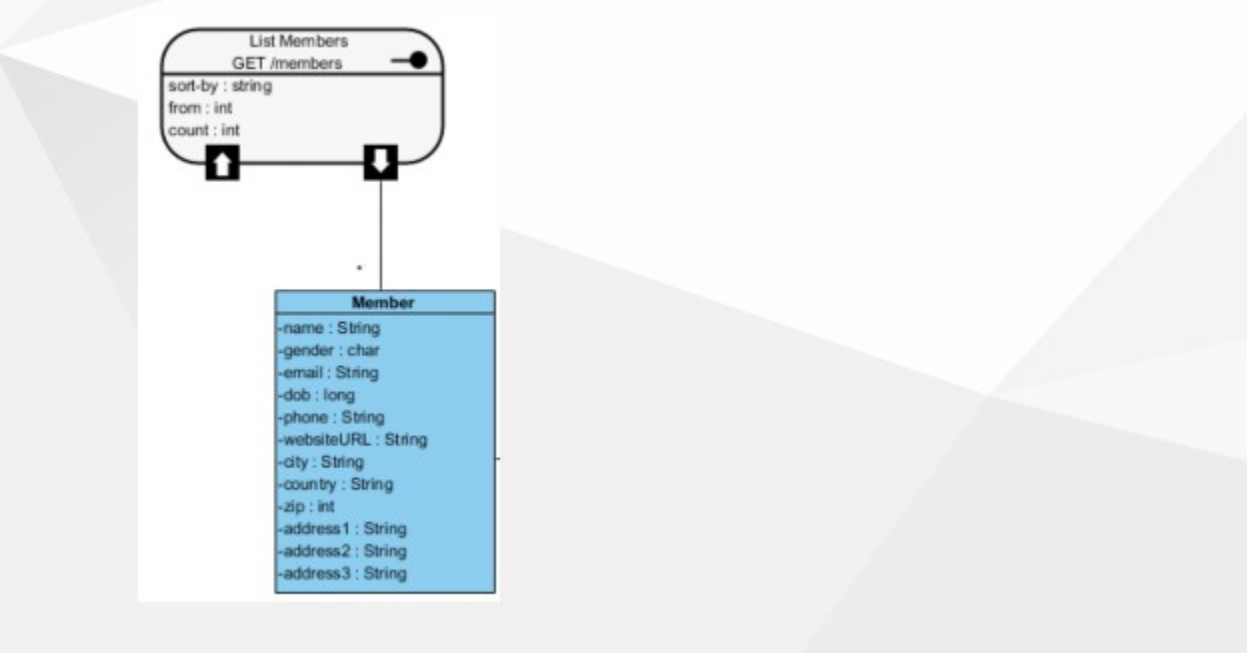
L’API **request body** viene utilizzato quando la risorsa usa POST, PUT, PATCH o DELETE oppure se sono richiesti dei parametri per utilizzare la risorsa:



L'API **response body** viene utilizzato quando la risorsa ritorna un valore o un oggetto:



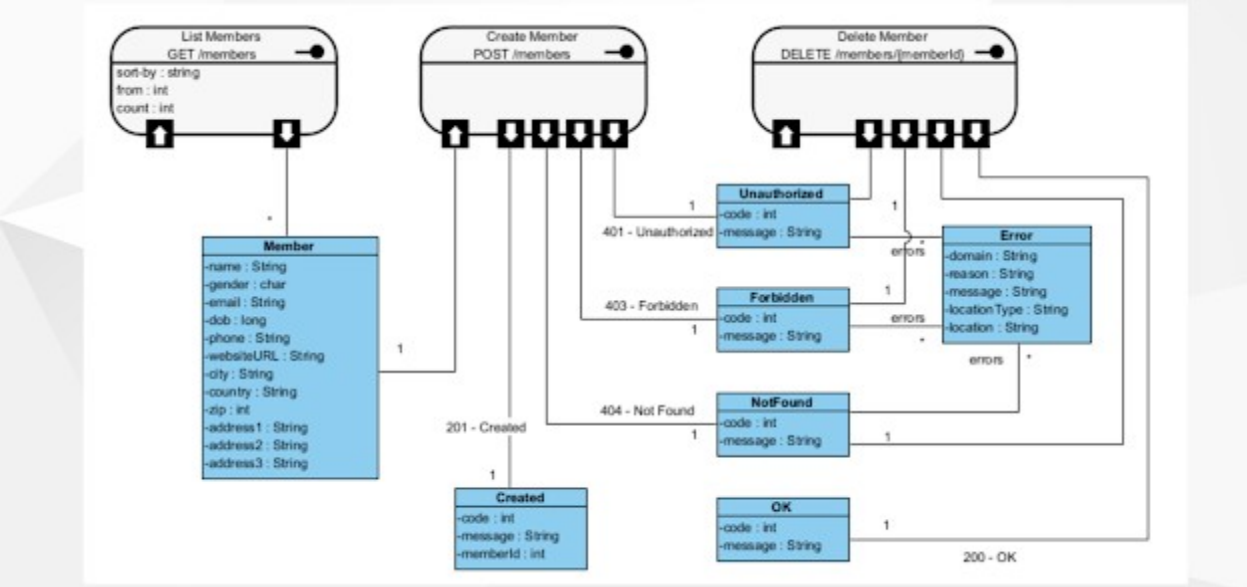
Nei GET si possono usare dei parametri per specificare la richiesta dell'oggetto:



Molto spesso bisogna modellare **scenari diversi** dove ci possono essere più responde bodies (ad esempio se vogliamo definire i diversi status code di HTTP)

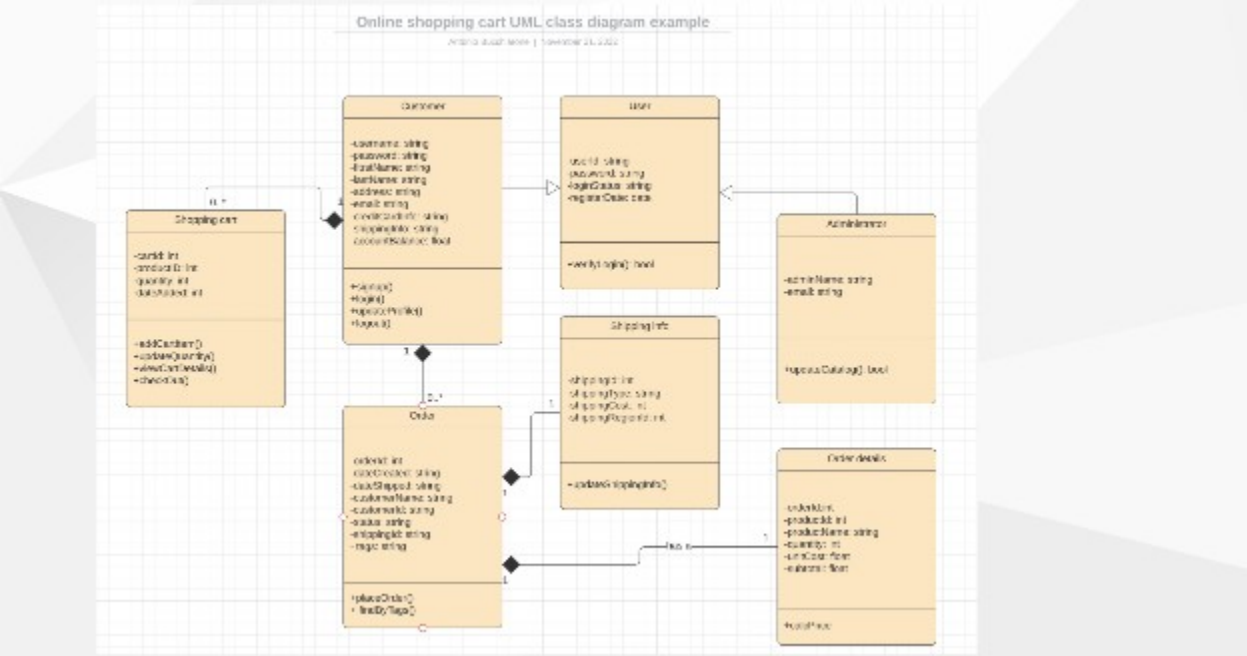
Response code	Description
200 OK	Request accepted, response contains result. This is a general purpose response code that can be returned from any request. For GET requests, the requested resource or data is in the response body. For PUT or DELETE requests, the request was successful and information about the result (such as new resource identifiers, or changes in resource status) can be found in the response body.
201 CREATED	This response code is returned from PUT or POST, and indicates that a new resource was created successfully. The response body might for example contain information about a new resource, or validation information (for example, when an asset is updated).
204 NO CONTENT	Indicates that the request was accepted but that there was nothing to return. This is returned when the request was processed, but no additional information about the result has been returned.
400 BAD REQUEST	The request was not valid. This code is returned when the server has attempted to process the request, but some aspect of the request is not valid, for example an incorrectly formatted resource or an attempt to deploy an invalid event project to the event runtime. Information about the request is provided in the response body, and includes an error code and error message.
401 UNAUTHORIZED	Is returned from the application server when application security is enabled, and authorization information was missing from the request.
403 FORBIDDEN	Indicates that the client attempted to access a resource which they do not have access to. This might be encountered if the user accessing the remote resource does not have sufficient privileges; for example, by having the WBERestApiUsers or WBERestApiPrivilegedUsers role. Users who attempt to access private event projects owned by other users might also see this error, but only if they have the WBERestApiUsers role rather than the WBERestApiPrivilegedUsers role.
404 NOT FOUND	Indicates that the targeted resource does not exist. This might be because the URI is malformed, or the resource has been deleted.
405 METHOD NOT ALLOWED	Returned when the targeted resource does not support the requested HTTP method; for example, the functions resource only allows GET operations.
406 NOT ACCEPTABLE	The data format requested in the Accept header or accept parameter is not supported by the targeted resource. That is, the client has requested that data is returned in a particular format, but the server is unable to return data in that format.
409 CONFLICT	Indicates that a conflicting change has been detected during an attempt to modify a resource. The response body provides further information.
415 UNSUPPORTED MEDIA TYPE	The data format of the request body, specified in the Content-Type header, is unsupported by the targeted resource.
500 INTERNAL SERVER ERROR	An internal error occurred in the server. This might indicate a problem with the request, or might indicate a problem in the server side code. Error information can be found in the response body.

Ecco un esempio dove consideriamo scenari diversi, nelle response body, appunto in base ai status code HTTP:

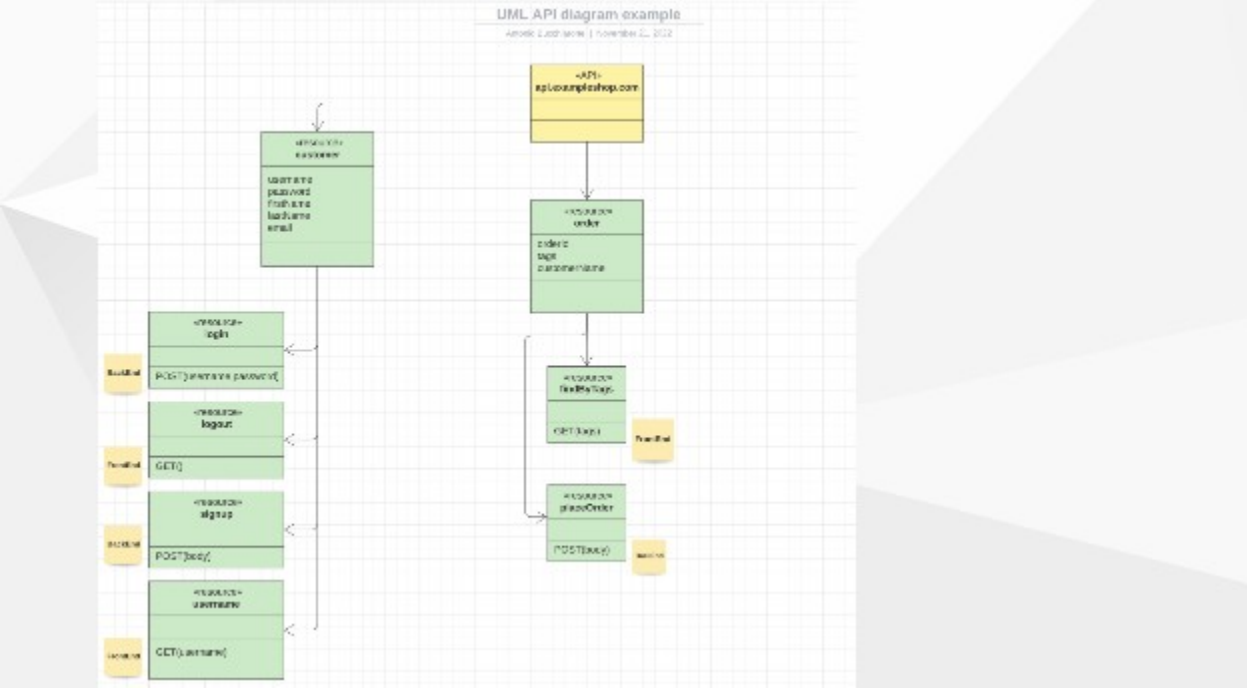


Bisogna mettere la cardinalità (soprattutto sulla response)

Vediamo un esempio di come si passa da un class diagram alla specifica API:

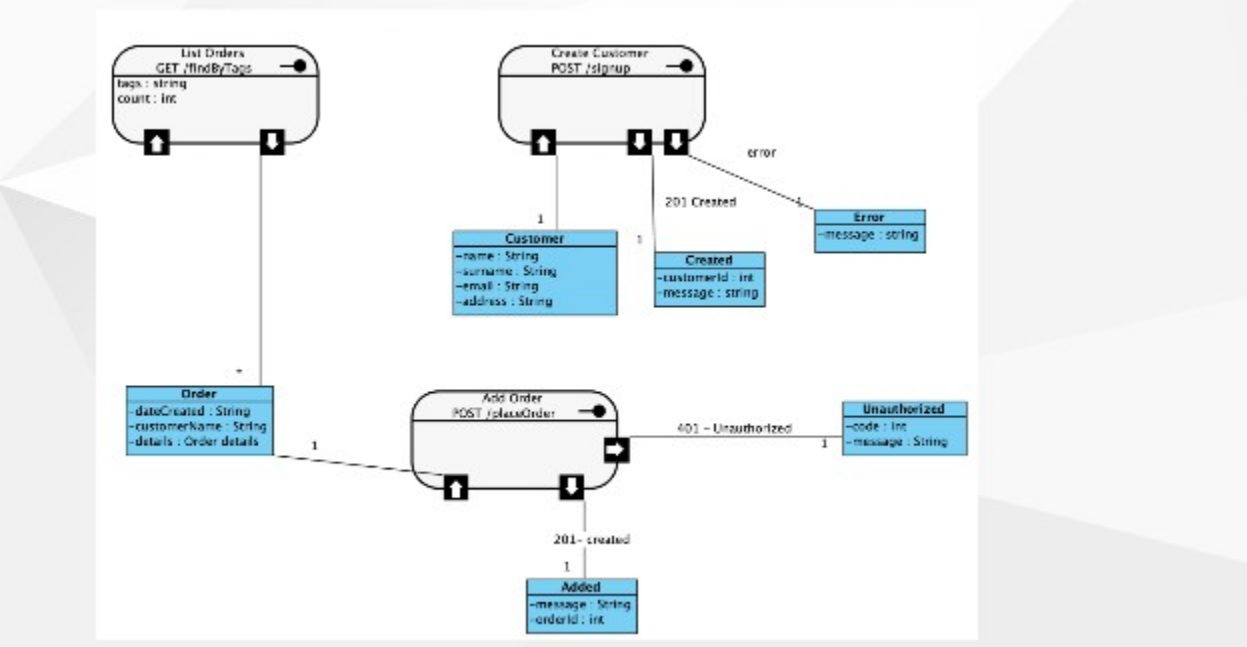


Identifichiamo ora le risorse e le API che vanno ad elaborare queste risorse:

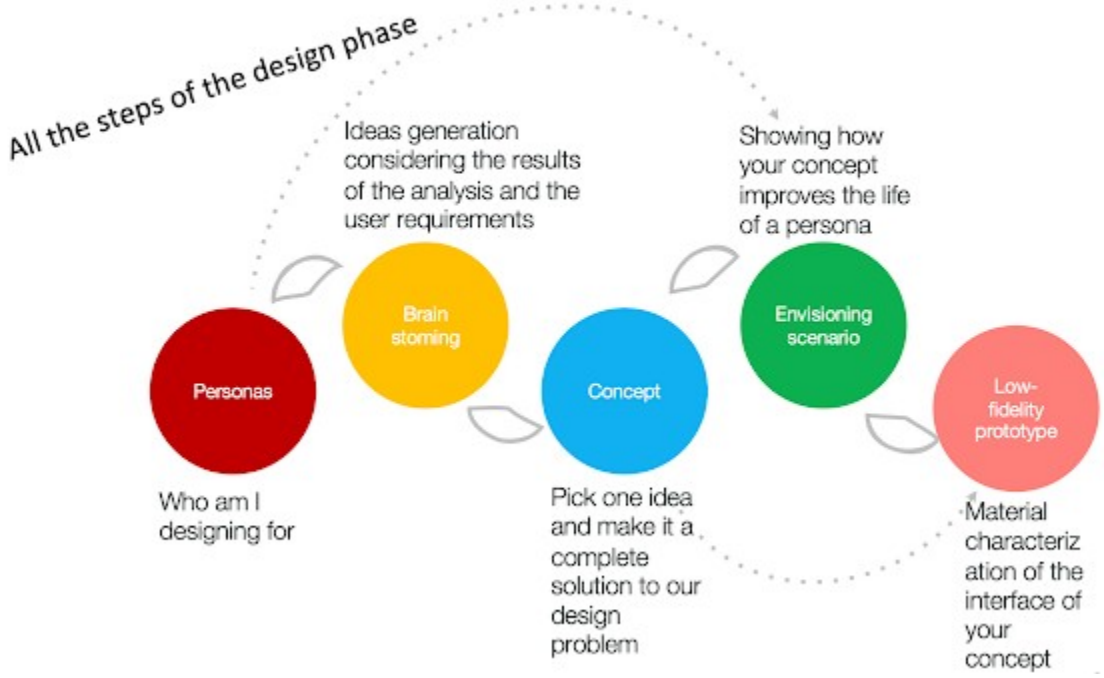


Successivamente vado a crearmi il modello delle risorse (in cui effettivamente descrivo come funzionano):

1. Per ogni API identificare request bodies e responde bodies
2. Identificare i parametri per le API get
3. Identificare i messaggi che servono

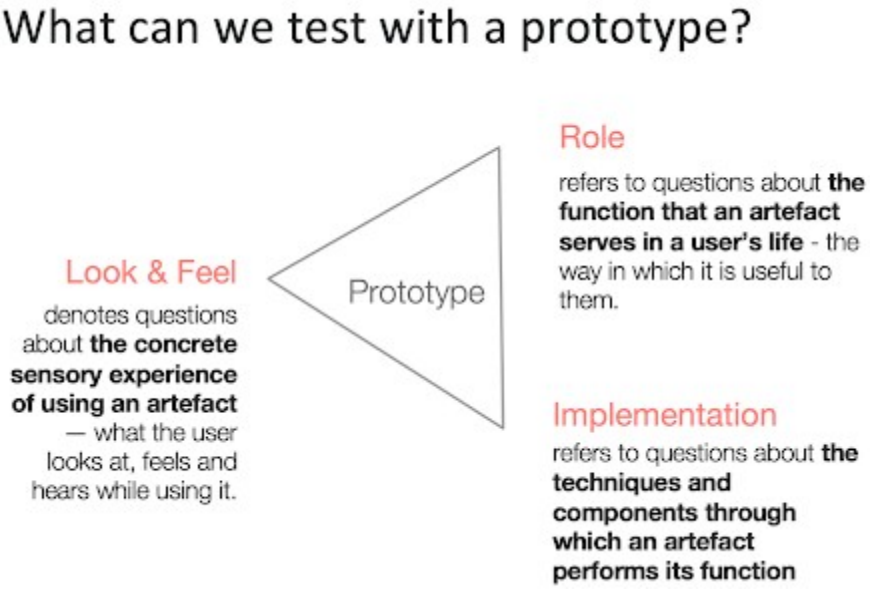


Progettazione dell’applicazione



Un **prototipo** è una rappresentazione di un’idea progettuale che viene fatta prima del prodotto finale. È una cosa che sta in mezzo a l’idea concettuale e il prodotto finale.
Il prototipo quindi è un **modello** e non un prodotto definito e finito.

Decidere su cosa concentrarsi su un prototipo è l’arte di identificare le più importanti domande in sospeso sulla progettazione.



I prototipi ci servono per ottenere una valutazione e un feedback sulle idee e in più gli stakeholders possono interagire più facilmente con un oggetto visivo e tangibile rispetto ad una descrizione o un documento di progettazione

I prototipi quindi sono usati per suscitare reazioni:

- Dai designers
  Per incoraggiare la comunicazione e riflettere sui materiali, per trovare risposte sulle domande e per scegliere tra le alternative
- Dagli utenti
  Per testare le idee generiche o gli aspetti specifici immediatamente, collezionando feedback e chiarendo i requisiti

Tramite i prototipi possiamo rappresentare:

- Problemi tecnici
- User flows
- Compiti di progettazione
- Layout
- Le idee difficili / controverse / cruciali

E possono essere rappresentati in diversi modi:

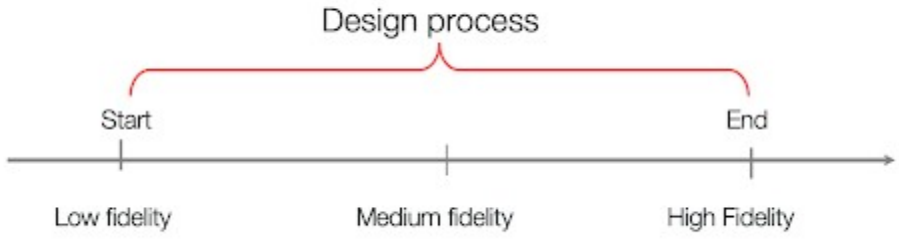
- Una serie di disegni
- Con una storyboard e gli scenari
- Con un powerpoint
- Mockup di cartone
- Un video che simula l'uso del sistema
- Pezzi di software con funzionalità limitate

In base a cosa vogliamo rappresentare il prototipo:

- Lavorerà come il sistema finale
- Assomiglierà esteticamente al sistema finale
- Avrà un'esperienza simile al sistema finale

Il **livello di fedeltà** di un prototipo è continuo, ovvero cresce sempre di più:

The level of *fidelity* of prototypes is a continuum



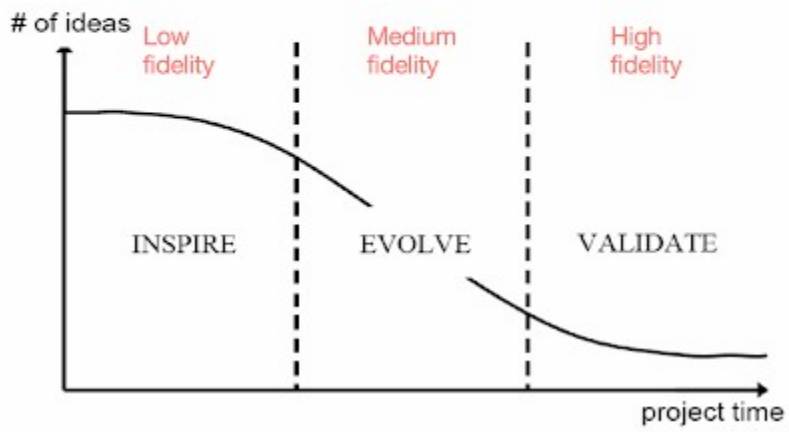








Prototype fidelity evolution



I due compromessi principali su cosa un prototipo può rappresentare sono:

- **Prototipo orizzontale** (overview generale)
  Un sacco di funzionalità ma con pochi dettagli
- **Prototipo verticale** (in maniera dettagliata)
  Poche funzionalità ma con tanti dettagli

I prototipi **low-fidelty** hanno gli obiettivi di:

- Mostrare concetti, non dettagli
- Mostrare i layout e la grafica
- Mostrare la macro funzionalità e i problemi di utilizzo principali
- Le alternative di progettazione

La forma con cui vengono fatti è:

- Veloce, economica e facile da cambiare
- Usare un “media” diverso da quello finale
- Non funzionale

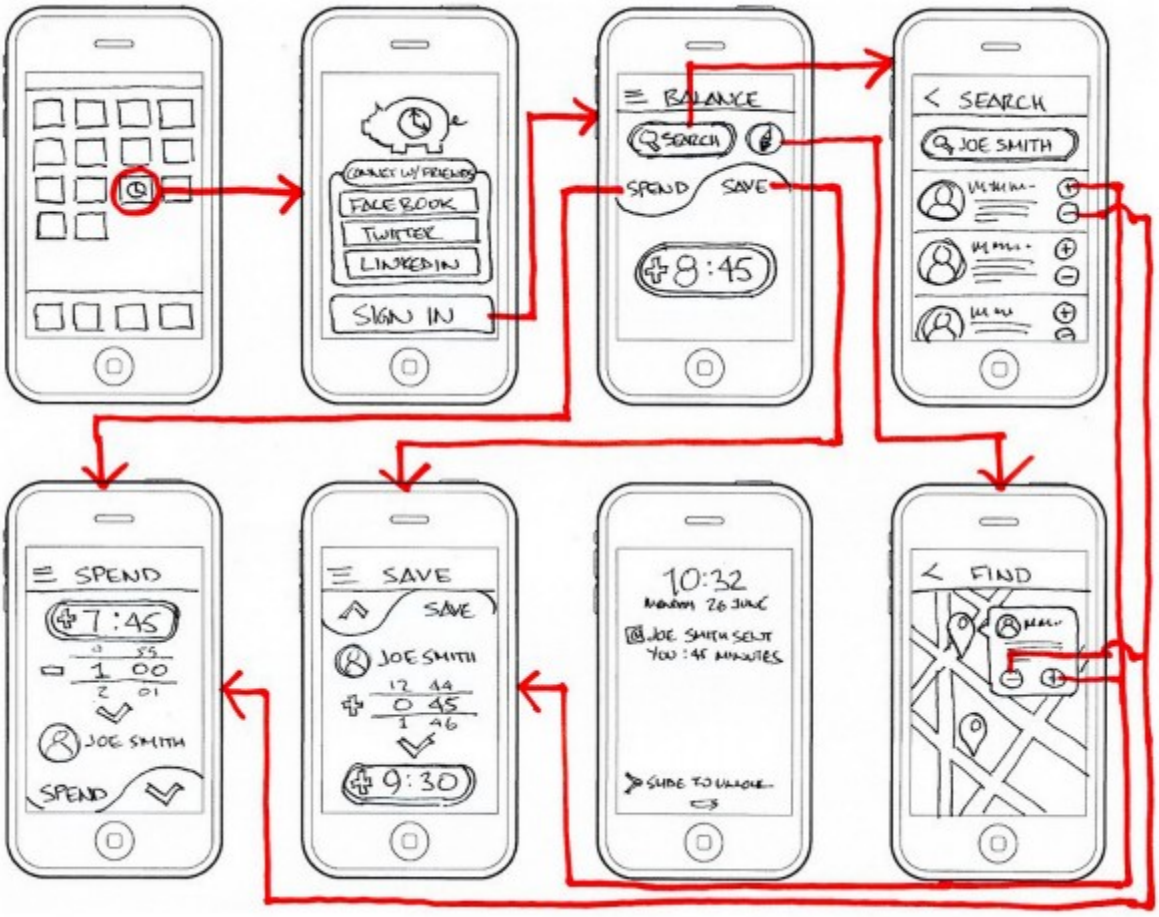
Gli usi sono:

- Il team di design può ragionare sulle differenti opzioni vedendo qualcosa di tangibile
- Per essere presentato al futuro utente del progetto

Degli esempi sono:

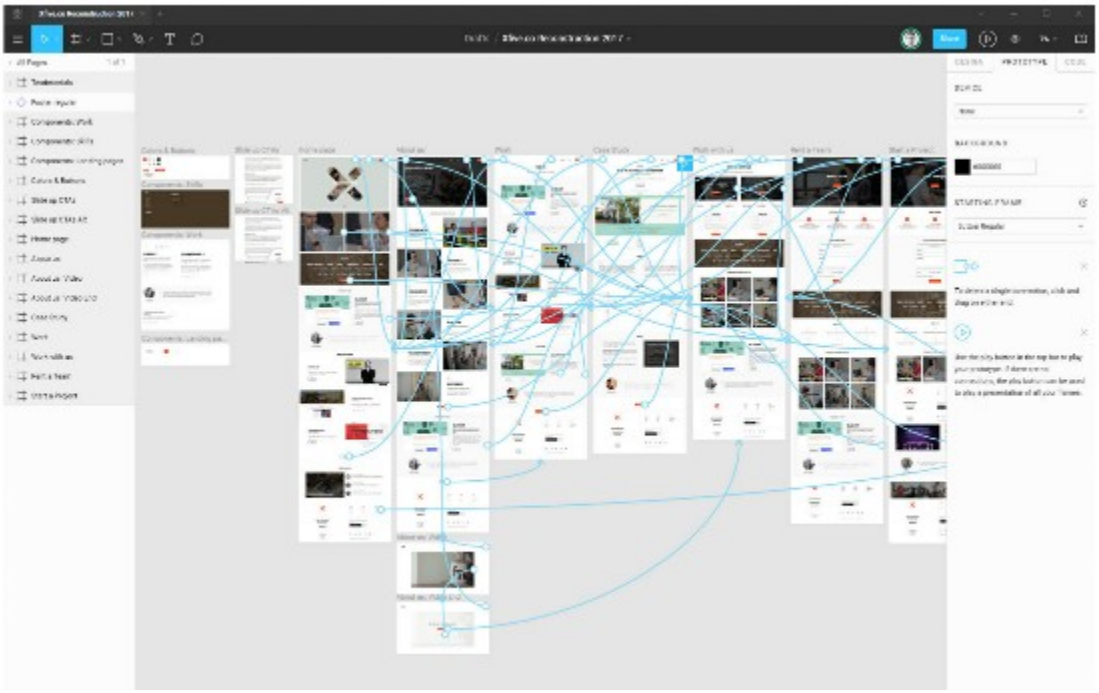
- Layout fatto su carta e penna
  Fornisce un overview senza i dettagli, usando simboli semplici e chiari

- User flows

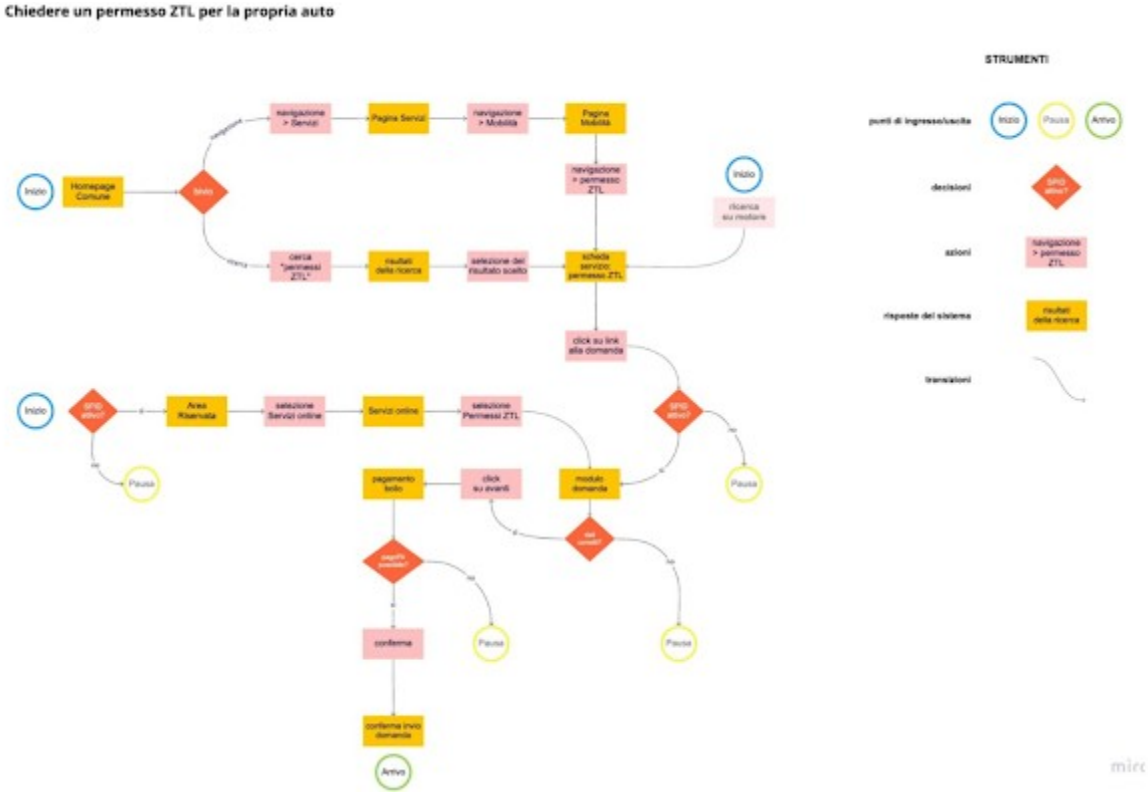


Le interfacce dello user flows mostrano graficamente come un utente navigherà nell'interfaccia utente
Mostra i collegamenti tra le varie pagine e le interazioni per passare da una all'altra.
Rispetto a Process/System flows, questo non descrive come un utente completa un'azione, ma invece descrive come un utente userà fisicamente l'interfaccia del sistema.

I prototipi **medium-fidelty** invece sono interattivi e "puliti" (PowerPoint, Figma etc)



FlowMapp è un tool specifico per fare web design



Piccolo resoconto:
Differenti tipi di prototipi sono usati per scopi differenti e a differenti livelli
I prototipi rispondono a delle domande, quindi bisogna prototipizzare in maniera appropriata
I prototipi e gli scenari sono usati durante la progettazione

Testing

Il **testing** consiste nella verifica del codice implementato tramite una serie di **casi di test**.
Il testing consiste in due fasi:

- Verifica del codice da parte del programmatore
- Verifica del codice da parte di altre persone

Esistono due metodologie principali di testing: l'**analisi statica** e l'**analisi dinamica**.
L'analisi **statica** consiste nella ricerca di eventuali anomalie **analizzando il codice** (anche usando tool appositi) **senza eseguire il software/prodotto**.

L'analisi **dinamica** consiste nella ricerca di eventuali malfunzionamenti del prodotto software tramite l'**esecuzione del codice** stesso, fornendo opportuni dati in ingresso.
Invertiamo quindi la logica delle fasi di Analisi dei Requisiti, Progettazione e Implementazione, nelle quali ci ponevamo domande del tipo:

- Com'è possibile implementare questa funzionalità?
- Come posso indurre l'utente a compiere queste azioni?

Le domande nella fase di testing diventano:

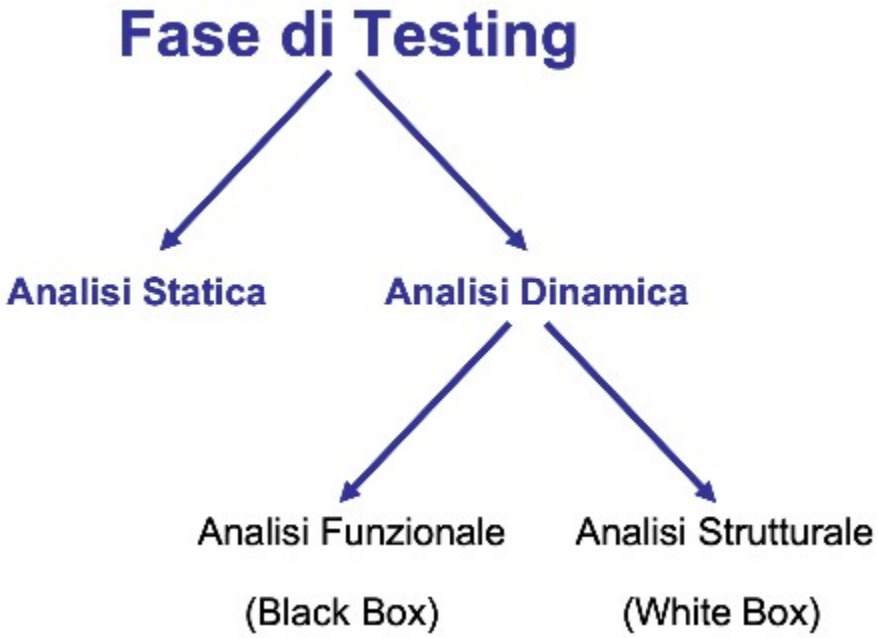
- Cosa può generare un errore in questa funzionalità?
- Cosa accade se l'utente compie queste azioni?

I due **indicatori chiave** (unità di misura) dell'analisi dinamica sono:

- La **quantità di test effettuati**
- La **copertura del codice da testare** (percentuale di linee di codice testate)

Rispettivamente esistono due analisi dinamiche che considerano questi due indicatori:

- L'**analisi funzionale** o **blackbox** (per la quantità di test effettuati)
- L'**analisi strutturale** o **whitebox** (per la copertura del codice da testare)



L’**analisi funzionale** (**black box**) ha lo scopo di sottoporre il software ad una serie di **casi di test** per **verificare i requisiti richiesti** (dato un requisito ci possono essere da 1 a n casi di test).
È importante sottolineare che **il codice non viene considerato** e si tratta di una verifica “sul campo” dei requisiti.
Per ogni caso di test l’analisi deve riportare:

- descrizione caso di test
- dati di input utilizzati per il test
- eventuali precondizioni necessarie a questo test
- eventuali dipendenze con altri casi di test
- risultato atteso
- risultato riscontrato
- nel caso di risultato riscontrato diverso da quello atteso: descrizione dell’anomalia che ha prodotto il malfunzionamento

Il **risultato** dell’analisi funzionale è **positivo** se il risultato attesto è sempre uguale a quello riscontrato per un alto numero di casi di test. È necessario però garantire un adeguato set di casi di test, che prendano in considerazione tutte le possibili varianti.

L’**analisi strutturale** (**white box**) ha lo scopo di sottoporre il software ad una serie di **casi di test** al fine di eseguire la **maggior quantità possibile di righe di codice**.
È importante sottolineare che **il codice viene considerato**, viene garantita la correttezza del software anche in casi estremi e individua eventuali parti del codice non utilizzate.
Questa analisi deve riportare la percentuale di copertura del codice utilizzando un’opportuna serie di casi di test. Questa percentuale sarà più alta per le parti del codice critiche. In ogni caso è molto difficile raggiungere il 100% di copertura.
Il **risultato** dell’analisi funzionale si guarda in base alla percentuale, più alta è la percentuale del codice, più il risultato sarà positivo.

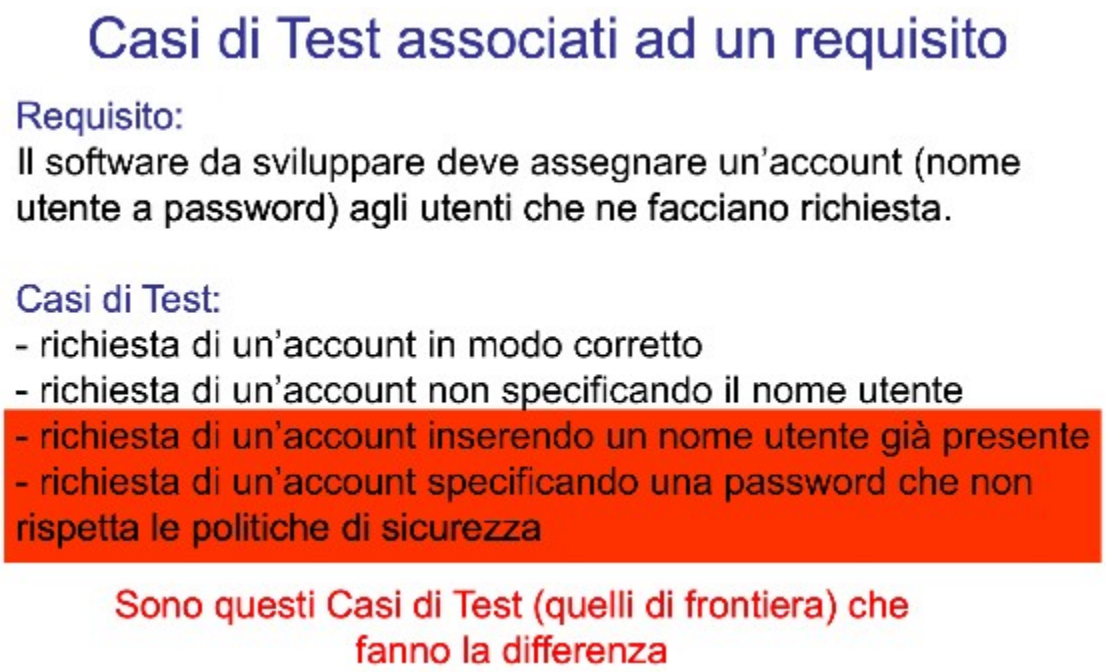
Definizione casi di test

L’analisi strutturale e funzionale sono da farsi ovviamente dopo la fase di sviluppo del codice, ma la definizione dei casi di test deve essere fatta **prima**, principalmente nella fase di Analisi dei Requisiti

Per ogni requisito si individuano dei casi di test, in particolare:

- **almeno un** caso di test con il requisito rispettato
- **almeno un** caso di test con il requisito non rispettato
- **almeno un** caso di test in situazioni di frontiera (quelli più critici)

Ecco un esempio:



Un caso di test deve:

- avere un'alta probabilità di scoprire un'errore
- non essere ridondante
- non deve essere né troppo semplice né troppo complesso (un caso di test molto complicato può essere inutile, perché nel caso in cui fallisca non si capisce dove sia l'errore che porta il caso di test a fallire)

È importante notare che la maggior parte delle volte il caso di test viene eseguito da una persona diversa da quella che ha fatto il progetto, quindi chi legge il caso di test deve essere in grado di sapere cosa fare e sapere cosa aspettarsi, senza conoscere in dettaglio l'applicazione. Ovviamente questa persona deve anche saper valutare se il risultato ottenuto è corretto o meno.

Ecco un esempio di tabella di casi di test:

Esempio di tabella di casi di test:

Numero Test Case	Descrizione Test Case	Test Data	Precondizioni	Dipendenze	Risultato Atteso	Risultato riscontrato	Note:
1	Creazione di un'account in modo corretto	<username> non vuota <password> rispettosa delle politiche di sicurezza	<username> mai inserita prima nel sistema	---	Viene creata l'account specificato. Il sistema risponde con <messaggio_OK>		
1.1	Creazione di un'account specificando uno username già esistente	<username> già inserita nel sistema	<username> già inserita nel sistema	Questo caso di test deve essere fatto dopo il caso di testo numero 1 specificando la stessa <username>	Viene mostrato un messaggio di errore <messaggio errore_user_exists> , l'account non viene creata ed il sistema mostra alcuni username disponibili da utilizzare		
2	Creazione di un'account non specificando lo username	<username> Vuota	---	---	Viene mostrato <messaggio errore_emptyuser> e l'account non viene creato		
3	Creazione di account violando le politiche di sicurezza (password banale, conferma password errata, ...)	<password> non rispettosa delle politiche di sicurezza	---	---	Viene mostrato un messaggio di errore e l'account non viene creata		

- Testing di user interface grafiche
- Testing di architetture client/server
  - Testing della funzione dell'applicazione
  - Testing del server
  - Testing dei database
  - Test delle transazioni
  - Testing delle comunicazioni di rete
- Testing della documentazione e degli strumenti di help
  - Gli errori nella documentazione possono avere lo stesso effetto sul cliente degli errori nei dati o nel codice sorgente
- Testing di sistemi real-time
  - Testing dei task
  - Testing del behaviour
  - Testing intertask
  - Testing di sistema